

École polytechnique de Louvain

Analysis and comparison of post-quantum cryptographic schemes: CROSS, MAYO and MQOM

AVX2 and NEON Implementations

Author: **Alexandre STORDEUR**

Supervisors: **François-Xavier STANDAERT, Thomas PETERS**

Reader: **Tom BARBETTE**

Academic year 2025–2026

Master [120] in Computer Science and Engineering

Acknowledgments

I would like to thank my supervisors, Pr. François-Xavier Standaert and Pr. Thomas Peters, for their guidance and feedback on this work throughout the year.

I would also like to thank Tom Barbette for agreeing to be the reader of this thesis.

My thanks also go to the people who supported me during this work and took the time to proofread it. I am particularly grateful to my family for their support and encouragement throughout my studies.

Enjoy your reading!

Abstract

The development of quantum computers creates a need for new cryptographic schemes that can resist quantum attacks. In this context, NIST launched a competition for additional post-quantum digital signature schemes. This thesis focuses on three candidates from the second round: CROSS, MAYO, and MQOM.

The aim is to analyze these three schemes and compare their performance on x86 and ARM architectures. Their reference implementations, AVX2 and NEON, are tested for key generation, signing, and verification. An internal analysis also compares the key parts of each implementation between AVX2 and NEON.

Since there was no optimized NEON implementation for MQOM v2.1, a primary contribution of this thesis is the development of this implementation. This includes adapting the main optimized components, such as SHA-3, Rijndael, and finite-field operations, to the ARM architecture.

Finally, the three schemes are compared regarding execution time, key sizes, and signature sizes. This shows the differences between the schemes and their implementations on the two architectures.

Contents

1	Introduction	1
1.1	Context	1
1.2	Content	2
2	Supports and Setup	4
2.1	Introduction	4
2.2	x86	4
2.2.1	AVX2 Extension	4
2.2.2	AES-NI Extension	4
2.2.3	SHA	4
2.2.4	GFNI	5
2.3	ARM	5
2.3.1	ARM types	5
2.3.2	NEON, SVE and SVE2 Extensions	6
2.3.3	AES Extension	6
2.3.4	SHA	6
2.4	Comparison of x86 and ARM	7
2.4.1	SIMD	7
2.4.2	AES	7
2.4.3	SHA	7
2.4.4	Final note	8
2.5	Compiling the signature scheme	8
2.6	Benchmark	8
2.6.1	Signature-scheme benchmark	9
2.6.2	Implementation benchmark	10
2.7	Benchmarking Platforms	11
3	CROSS	12
3.1	Description of the CROSS signature scheme	12
3.1.1	Parameters	12
3.1.2	CROSS-ID Protocol	13
3.1.3	Key Generation	14
3.1.4	Signing Algorithm	14
3.1.5	Verification Algorithm	15
3.1.6	SeedTree and Merkle-Tree	15
3.2	Current Implementations	16
3.3	CrossNEON	16
3.3.1	Overview	16
3.3.2	Portability to AVX2	18
3.4	Analysis of internal performance	18
3.4.1	Key Generation	18
3.4.2	Signing Algorithm	20

3.4.3	Verification Algorithm	22
3.5	Conclusion and Future work	25
3.5.1	Conclusion	25
3.5.2	Future work	25
4	MAYO	26
4.1	Description of the MAYO signature scheme	26
4.1.1	Parameters	26
4.1.2	MAYO Principle	27
4.1.3	Key Generation	27
4.1.4	Signing Algorithm	27
4.1.5	Verification Algorithm	28
4.2	Current Implementations	28
4.3	Analysis of internal performance	28
4.3.1	Key Generation	28
4.3.2	Signing Algorithm	30
4.3.3	Verification Algorithm	32
4.4	Conclusion and Future work	33
4.4.1	Conclusion	33
4.4.2	Future work	34
5	MQOM	35
5.1	Description of the MQOM signature scheme	35
5.1.1	Parameters	35
5.1.2	MQOM Principle	36
5.1.3	Key Generation	37
5.1.4	Signing Algorithm	37
5.1.5	Verification Algorithm	37
5.2	Current Implementations	37
5.3	Implementation of MQOMv2.1 NEON	38
5.3.1	Makefile	39
5.3.2	SHA-3	39
5.3.3	Rijndael	42
5.3.4	Fields	49
5.3.5	Known answer test results	51
5.4	Analysis of internal performance	52
5.4.1	Key Generation	52
5.4.2	Signing Algorithm	54
5.4.3	Verification Algorithm	56
5.5	Conclusion and Future work	59
5.5.1	Conclusion	59
5.5.2	Future work	59
6	Comparison	60
6.1	Size Comparison	60
6.2	Performance Comparison	61
6.3	Signature Comparison Recap	64
7	Conclusion and Future Work	66
7.1	Conclusion	66
7.2	Future work	67
7.3	References	68
7.4	AI usage	68

A	Instructions	72
A.1	AES-NI instructions	72
A.2	SHA-NI instructions	72
A.3	AES ARM instructions	73
A.4	SHA ARM instructions	73
B	Mersenne prime	74
C	Measured values	76
C.1	Size	76
C.1.1	Private Key size in Bytes	76
C.1.2	Public Key size in Bytes	76
C.1.3	Signature size in Bytes	77
C.2	Overall Benchmark	77
C.2.1	Benchmark key generation in kilocycles	77
C.2.2	Benchmark signing algorithm in kilocycles	78
C.2.3	Benchmark verification in kilocycles	79
C.3	Internal Benchmark	81
C.3.1	CROSS	81
C.3.2	MAYO	86
C.3.3	MQOMv2.1	87

List of Figures

2.1	ARM-A versions	6
2.2	AES single round encryption AVX2 and NEON	6
2.3	Files added or modified for the Overall benchmark	9
2.4	Internal bench files modification	9
2.5	AVX2 tag	10
2.6	NEON tag	11
3.1	Illustrative tree structures for $t = 11$: balanced and small versions (left) vs. fast version (right). Gray nodes represent leaves.	15
3.2	Comparison of CROSS Key generation, Reference and AVX2 versions (Table C.4) . .	18
3.3	Comparison of CROSS Key generation, Reference and NEON versions (Table C.5) .	18
3.4	CROSS internal analysis of key generation for AVX2 implementation ¹ (Table C.10) .	19
3.5	CROSS internal analysis of key generation for NEON implementation (Table C.11) .	19
3.6	Difference in proportion (NEON - AVX2) for CROSS key generation internal analysis (Tables C.10, C.11)	20
3.7	Comparison of CROSS signing algorithm, Reference and AVX2 versions (Table C.6) .	20
3.8	Comparison of CROSS signing algorithm, Reference and NEON versions (Table C.7)	20
3.9	CROSS internal analysis of signing algorithm for AVX2 implementation (Tables C.12, C.13)	21
3.10	CROSS internal analysis of signing algorithm for NEON implementation (Tables C.14, C.15)	21
3.11	Difference in proportion (NEON - AVX2) for CROSS signing algorithm internal analysis (Tables C.12, C.13, C.14, C.15)	22
3.12	Comparison of CROSS verification algorithm, Reference and AVX2 versions (Table C.8)	22
3.13	Comparison of CROSS verification algorithm, Reference and NEON versions (Table C.9)	23
3.14	CROSS internal analysis of verification algorithm for AVX2 implementation (Tables C.16, C.17)	23
3.15	CROSS internal analysis of verification algorithm for NEON implementation (Table C.18, C.19)	24
3.16	Difference in proportion (NEON - AVX2) for CROSS verification algorithm internal analysis (Tables C.16, C.17, C.18, C.19)	24
4.1	Comparison of MAYO Key generation, Reference and AVX2 versions (Table C.4) . .	28
4.2	Comparison of MAYO Key generation, Reference and NEON versions (Table C.5) . .	29
4.3	MAYO internal analysis of key generation for AVX2 implementation ² (Table C.20) .	29
4.4	MAYO internal analysis of key generation for NEON implementation (Table C.21) .	29
4.5	Difference in proportion (NEON - AVX2) for MAYO key generation internal analysis (Tables C.20, C.21)	30
4.6	Comparison of MAYO signature algorithm, Reference and AVX2 versions (Table C.6)	30
4.7	Comparison of MAYO signature algorithm, Reference and NEON versions (Table C.7)	30
4.8	MAYO internal analysis of signing algorithm for AVX2 implementation (Table C.22)	31
4.9	MAYO internal analysis of signing algorithm for NEON implementation (Table C.23)	31

4.10	Difference in proportion (NEON - AVX2) for MAYO signing algorithm internal analysis (Tables C.22, C.23)	31
4.11	Comparison of MAYO verification algorithm, Reference and AVX2 versions (Table C.8)	32
4.12	Comparison of MAYO verification algorithm, Reference and NEON versions (Table C.9)	32
4.13	MAYO internal analysis of verification algorithm for AVX2 implementation (Table C.24)	32
4.14	MAYO internal analysis of verification algorithm for NEON implementation (Table C.25)	33
4.15	Difference in proportion (NEON - AVX2) for MAYO verification algorithm internal analysis (Tables C.24, C.25)	33
5.1	Overall MQOM directory	38
5.2	Overall NEON implementation directory	38
5.3	MQOM Makefile NEON implementation	39
5.4	MQOM SHA-3 NEON implementation	40
5.5	SHA-3 x4 structure [bits]	41
5.6	Assembly from KeccakP-1600-armv8a-neon.S	42
5.7	rijndael_aes_ni.h and rijndael_neon.h comparison	43
5.8	rijndael_aes_ni.h and rijndael_neon.h comparison, AES Key scheduling	44
5.9	rijndael_aes_ni.h and rijndael_neon.h comparison, AES Key scheduling part 2	44
5.10	rijndael_neon.h, AES Key scheduling part 3	45
5.11	rijndael_aes_ni.h and rijndael_neon.h comparison, AES Round	46
5.12	rijndael_aes_ni.h and rijndael_neon.h comparison, Rijndael Key scheduling	47
5.13	Rijndael-256-256 single round encryption	47
5.14	rijndael_aes_ni.h and rijndael_neon.h comparison, Rijndael Round	49
5.15	rijndael_neon.h , Rijndael Round part 2	49
5.16	MQOM KAT NEON implementation	51
5.17	Comparison of MQOMv2.1 Key generation, Reference and AVX2 versions (Table C.4)	52
5.18	Comparison of MQOMv2.1 Key generation, Reference and NEON versions (Table C.5)	52
5.19	MQOMv2.1 internal analysis of Key generation for AVX2 implementation ³ (Table C.26)	53
5.20	MQOMv2.1 internal analysis of Key generation for NEON implementation (Table C.27)	53
5.21	Difference in proportion (NEON - AVX2) for MQOMv2.1 key generation internal analysis (Tables C.26, C.27)	54
5.22	Comparison of MQOMv2.1 signing algorithm, Reference and AVX2 versions (Table C.6)	54
5.23	Comparison of MQOMv2.1 signing algorithm, Reference and NEON versions (Table C.7)	55
5.24	MQOMv2.1 internal analysis of signing algorithm for AVX2 implementation (Table C.28)	55
5.25	MQOMv2.1 internal analysis of signing algorithm for NEON implementation (Table C.29)	56
5.26	Difference in proportion (NEON - AVX2) for MQOMv2.1 signing algorithm internal analysis (Tables C.28, C.29)	56
5.27	Comparison of MQOM verification algorithm, Reference and AVX2 versions (Table C.8)	57
5.28	Comparison of MQOM verification algorithm, Reference and NEON versions (Table C.9)	57
5.29	MQOMv2.1 internal analysis of verification algorithm for AVX2 implementation (Table C.30)	58
5.30	MQOMv2.1 internal analysis of verification algorithm for NEON implementation (Table C.31)	58
5.31	Difference in proportion (NEON - AVX2) for MQOMv2.1 verification algorithm internal analysis (Tables C.30, C.31)	58
6.1	Comparison of the private key size of CROSS, MAYO and MQOM (Table C.1)	60
6.2	Comparison of the public key size of CROSS, MAYO and MQOM (Table C.2)	60
6.3	Comparison of the signature size of CROSS, MAYO and MQOM (Table C.3)	61

6.4	Comparison of signature schemes key generation by time for the AVX2 optimized implementations of each version run on Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (Table C.4)	61
6.5	Comparison of signature schemes key generation by time for the NEON optimized implementations of each version run on Cortex-A76 (Table C.5)	62
6.6	Comparison of signature schemes signing algorithm by time for the AVX2 optimized implementations of each version run on Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (Table C.6)	62
6.7	Comparison of signature schemes signing algorithm by time for the NEON optimized implementations of each version run on Cortex-A76 (Table C.7)	63
6.8	Comparison of signature schemes verification algorithm by time for the AVX2 optimized implementations of each version run on Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (Table C.8)	63
6.9	Comparison of signature schemes verification algorithm by time for the NEON optimized implementations of each version run on Cortex-A76 (Table C.9)	64

List of Tables

1.1	Main contributions of this master thesis.	3
2.1	AES-128 and AES-256 Comparison	5
2.2	ARM types	5
2.3	SIMD Comparison	7
2.4	AES Comparison	7
2.5	Specifications of the benchmarking platforms	11
3.2	CROSS-ID	14
3.3	Available implementations of CROSS	16
3.4	CROSS: Performance comparison between AVX2 and NEON	25
4.2	Available implementations of CROSS	28
4.3	MAYO: Performance comparison between AVX2 and NEON	33
5.2	Available implementations	38
5.3	Correspondence between AVX2 and NEON functions	50
5.4	MQOM NEON version verified with or without XOFX4 or ENCX8	51
5.5	MQOM: Performance comparison between AVX2 and NEON	59
6.1	Public Key size in Bytes (Table C.2)	65
A.1	AES-NI instructions	72
A.2	SHA-256 (SHA-2) x86	72
A.3	AES ARM instructions	73
A.4	ARM SHA instructions	73
C.1	Private Key size in Bytes	76
C.2	Public Key size in Bytes	76
C.3	Signature size in Bytes	77
C.4	Key generation in kilocycles, Reference and AVX2 implementations	78
C.5	Key generation in kilocycles, ARM Ref and NEON implementations	78
C.6	Signing algorithm in kilocycles, Reference and AVX2 implementations	79
C.7	Signing in kilocycles, ARM Ref and NEON implementations	79
C.8	Verification algorithm in kilocycles, Reference and AVX2 implementations	80
C.9	Verification algorithm in kilocycles, ARM Ref and NEON implementations	81
C.10	Internal Cross Key generation, AVX2 implementation	81
C.11	Internal Cross Key generation, CrossNEON implementation	81
C.12	Internal CROSS signing algorithm, AVX2 implementation – Part 1	82
C.13	Internal CROSS signing algorithm, AVX2 implementation – Part 2	82
C.14	Internal CROSS signing algorithm, CrossNEON implementation – Part 1	83
C.15	Internal CROSS signing algorithm, CrossNEON implementation – Part 2	83
C.16	Internal CROSS verification algorithm, AVX2 implementation – Part 1	84
C.17	Internal CROSS verification algorithm, AVX2 implementation – Part 2	84
C.18	Internal CROSS verification algorithm, CrossNEON implementation – Part 1	85

C.19 Internal CROSS verification algorithm, CrossNEON implementation – Part 2	85
C.20 Internal MAYO Key generation, AVX2 implementation	86
C.21 Internal MAYO Key generation, NEON implementation	86
C.22 Internal MAYO signing algorithm, AVX2 implementation	86
C.23 Internal MAYO signing algorithm, NEON implementation	86
C.24 Internal MAYO verification algorithm, AVX2 implementation	86
C.25 Internal MAYO verification algorithm, NEON implementation	86
C.26 Internal MQOMv2.1 Key generation, AVX2 implementation	87
C.27 Internal MQOMv2.1 Key generation, NEON implementation	88
C.28 Internal MQOMv2.1 signing algorithm, AVX2 implementation	89
C.29 Internal MQOMv2.1 signing algorithm, NEON implementation	90
C.30 Internal MQOMv2.1 verification algorithm, AVX2 implementation	91
C.31 Internal MQOMv2.1 verification algorithm, NEON implementation	92

Chapter 1

Introduction

1.1 Context

With the emergence and development of quantum computing, modern cryptography is at risk, and threatens many public-key algorithms used in modern infrastructures, like RSA and ECC. A quantum computer could efficiently execute Shor's algorithm or Grover's algorithm, which would enable the possibility to break most of the current cryptographic systems. The risk is even greater with the "harvest now, decrypt later" attack model, in which encrypted data is stored now with the intent to decrypt it after the adversary gains the relevant computational capabilities. For all those reasons, there has been a push to develop quantum-resistant algorithms.

In 2016, the National Institute of Standards and Technology (NIST)[1] started the process of post-quantum cryptography (PQC) standardization. This worldwide competition seeks to select and implement new cryptographic algorithm standards protected against attacks by classical and quantum computers.

The first selected post-quantum standards were published by NIST[2]. These are:

⇒ Key-Encapsulation Mechanisms

- **ML-KEM**, short for Module-Lattice-Based Key-Encapsulation Mechanism and previously known as CRYSTALS-Kyber, was selected by NIST for standardization in 2022. Further information can be found in the FIPS 203 document [3].
- **HQC**, short for Hamming Quasi-Cyclic, was selected by NIST for standardization in 2025. An FIPS document concerning this scheme will be released by the NIST in the future.

⇒ Digital Signature Algorithms

- **ML-DSA**, short for Module-Lattice-Based Digital Signature Algorithm and previously known as CRYSTALS-Dilithium, selected by NIST for standardization in 2022. Further information can be found in the FIPS 204 document [4].
- **SLH-DSA**, short for Stateless Hash-Based Digital Signature Algorithm and previously known as SPHINCS+, selected by NIST for standardization in 2022. Further information can be found in the FIPS 205 document [5].
- **FN-DSA**, short for FFT (fast-Fourier transform) over NTRU-Lattice-Based Digital Signature Algorithm and previously known as FALCON, was selected by NIST for standardization in 2022. A FIPS document concerning this scheme will be released by the NIST in the near future and will be called FIPS 206.

This standardization started the transition of post-quantum cryptography from research to deployment, with these constructions now appearing in protocols, software, and national implementations.

As we saw, NIST continued to expand its post-quantum portfolio following the first wave from 2022 because of the large number of lattice-based schemes chosen. And in 2025, NIST selected an alternative key encapsulation algorithm, HQC. It became a backup standard scheme, providing cryptographic diversity through the application of a different mathematical problem.

Following this idea of diversity, the NIST started a new competition for additional digital signature schemes[6]. This competition followed this agenda:

- ▷ **6 September 2022** Request for Additional PQC Digital Signature Schemes
- ▷ **17 July 2023** First Round selection between 40 candidates
- ▷ **25 October 2024** Second Round selection between 14 candidates
- ▷ **14 May 2026** Third Round selection between 9 candidates

This master thesis started and took place during **Round 2 of the NIST Additional Digital Signature Schemes competition**. However, some implementations evolved during the thesis, such as MQOM, for which version 2.1 is used in this work.

1.2 Content

This master thesis aims to introduce, analyze, and compare the runtime performance of three post-quantum signature schemes selected from the second round of the NIST Post-Quantum Cryptography standardization process. The selected candidates are:

- **CROSS**, chosen for its similarities with concepts studied in the courses LELEC2770 - Privacy Enhancing Technologies and LELEC2348 - Information Theory and Coding.
- **MAYO**, chosen for its good time performance on PQ-SORT, a visual comparison platform from the Technology Innovation Institute [7].
- **MQOM**, chosen for its originality and its alternative approach to the construction of post-quantum signature schemes.

To limit the scope of this thesis, the study focuses only on runtime performance on general-purpose computers and server platforms. The comparison mainly targets x86 and ARM-A architectures using AVX2 and NEON optimized implementations.

The different schemes are compared for the same NIST security categories. **Category I, III and V** correspond to increasing security levels. In practice, this is reflected by different parameter sizes. This allows us to compare implementations providing a similar level of security.

The objective of this work is to answer the following questions:

- How do CROSS, MAYO, and MQOM perform on the current x86 and ARM-A architectures?
- How could the performance of the different signature schemes evolve with future hardware and software optimizations for AVX2 and NEON versions?

To answer these questions, I first benchmarked the existing reference, AVX2 and NEON implementations of CROSS and MAYO.

I also perform an internal analysis of the implementations. The goal is not only to identify which parts take the most execution time, but also to compare how the proportion of the execution time changes between AVX2 and NEON. This allows us to check if the observed differences are consistent with the architectural differences presented previously, and to identify unexpected behaviours.

For MQOM, an optimized AVX2 implementation was available but no ARM-A NEON implementation was available for the comparison. A main contribution of this thesis is therefore the implementation of a NEON optimized version of MQOM v2.1. This implementation allows MQOM to be included in the AVX2 and NEON comparison.

Contribution	Description
Benchmark	Benchmark of CROSS, MAYO and MQOM for key generation, signing and verification on AVX2 and NEON.
Internal analysis	Division of the implementations into different parts to compare the proportion of execution time between AVX2 and NEON, verify if the differences are consistent with the architectures, and identify unexpected behaviours.
MQOM NEON implementation	Implementation of an ARM-A NEON optimized version of MQOM v2.1.
SHA-3	Integration and adaptation of a NEON implementation of Keccak/SHA-3 for MQOM.
Rijndael	Implementation of AES-128, AES-256 and Rijndael-256-256 using ARM AES and NEON instructions.
Finite fields	Port of the AVX2 optimized finite-field operations of MQOM from 256-bit AVX2 vectors to 128-bit NEON vectors.
Correctness	Verification of the MQOM NEON implementation using the Known Answer Tests.
CROSS	Verification of the CrossNEON statement to be a better implementation. With comparison of reduction time with other signatures schemes.

Table 1.1: Main contributions of this master thesis.

My main contributions are summarized in Table 1.1.

Finally, the overall and internal results of CROSS, MAYO and MQOM are compared. The overall benchmarks show which implementations are the fastest, while the internal analysis helps explain the differences observed between AVX2 and NEON and identify possible future optimizations.

Chapter 2

Supports and Setup

2.1 Introduction

In this chapter, we will introduce the two most used general purpose architecture set and the optimizations coming with them used for our signature schemes on x86 and arm.

2.2 x86

The Call for proposal from the NIST required an Intel x64 processor optimized version, this type of processor is an x86-64, widely used in PCs and servers. This type of family is defined as CISC which means that the processor uses a large set of instructions that can be complex or can perform multiple operations. This processor also comes with an instruction set extension enabling processor specific optimizations.

The optimized versions required rely mainly on two extensions to optimize the reference version to the AVX2 optimized version. The first one is the vectorization called SIMD (single instruction multiple data) that give the possibility to execute an instruction with multiple data in parallel. The second is AES-NI (Advanced Encryption Standard Instructions), an instruction set for x86. That optimization led to the biggest improvement in performances, to quote Intel "*AES-NI can provide a 10x improvement over software solutions*".

2.2.1 AVX2 Extension

AVX2 or Advanced Vector Extensions 2 is the 256 bit evolution of the previous 128 bit SSE extension. This extension allows operation on vectors instead of simply registers.

2.2.2 AES-NI Extension

AES-NI or Advanced Encryption Standard – New Instructions is a set allowing us to perform AES. In the different schemes only AES-128 and AES-256 are used. Note that AES-NI posses an instruction named keygen assist, This instruction assists the AES key expansion by performing the SubWord transformation, the RotWord operation, and the incorporation of the round constant (Rcon). See Appendix A.1 for the instructions available. With those instructions, we are able to create rijndael-256-256 which is similar to AES-256 with a part mixing the 32 bytes. But, this will be explained in greater detail in the subsection 5.3.3.

2.2.3 SHA

SHA is divided into 3 different categories: SHA-1, SHA-2 and SHA-3. SHA-1 is considered now as insecure, SHA-2 is the most widely used and SHA-3[8] arrived in 2015 is relatively new. See the Appendix A.2 for the instructions available.

Parameter	AES-128	AES-256	Rijndael-256-256
Key size	128 bits	256 bits	256 bits
Bloc size	128 bits	128 bits	256 bits
Number of rounds	10	14	14

Table 2.1: AES-128 and AES-256 Comparison

2.2.4 GFNI

The Galois Field New Instructions (GFNI) are a set of instructions that perform arithmetic operations over the finite field $GF(2^8)$. Among the signature schemes, This extension is only used by MQOM. it presents a large number of instructions which won't be discussed in this Master's thesis.

2.3 ARM

ARM is a family of energy-efficient processor architectures widely used in mobile devices, embedded systems, and increasingly in servers. It is an important architecture class, being the most widely used after x86. For example, ARM-based processors are used by Apple and the majority of Android devices, two major players in the technology market. ARM is further divided into different categories, each designed for specific use cases.

2.3.1 ARM types

The different types of ARM Architectures. Arm is divided into 3 big families: Arm-A, Arm-M and Arm-R¹.

	Arm-A	Arm-M	Arm-R
Meaning	Application	Microcontroller	Real-Time
OS	Linux, Android	Bare-metal, RTOS	RTOS
64 bits	Yes	No	No
Performance	High	Low	Medium
Power consumption	High	Very Low	Medium
example	server, mobile	IoT	Car Braking Systems

Table 2.2: ARM types

In this master thesis, we focus on the Arm-A family since we focus on general purpose computers and servers. It is important to note that it includes multiple versions, each introducing its own set of upgrades, as illustrated in Figure 2.2. One aspect not shown in this figure is the Scalable Vector Extension (SVE), which extends NEON Adv SIMD and was introduced in ARMv8.2-A and SVE2 introduced in ArmV9-A.

The Arm-A equivalent extensions to x86 are NEON, SVE and SVE2 for AVX2 and AES for AES-NI. Since ArmV8.2-A instruction to accelerate SHA-3 has been created.

¹Note: Arm-A is a type of architecture and Cortex-A is a type of processor

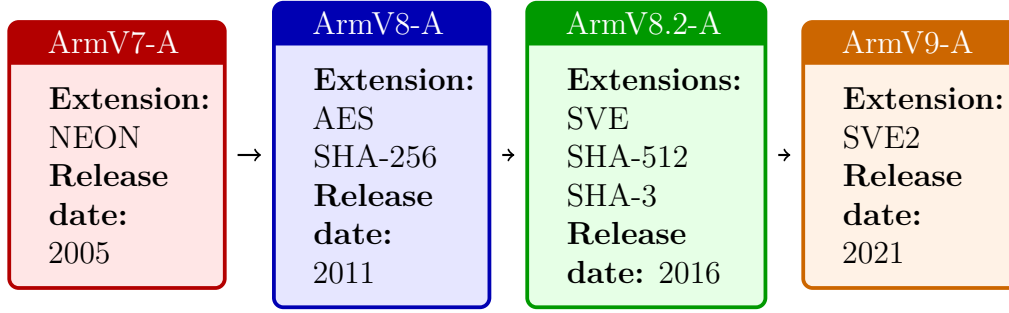


Figure 2.1: ARM-A versions

Note that ARMv7-A was built on 32 bits architecture and since ARMv8-A a 64 bits architecture is available. In this master thesis, we focused only 64-bits architecture.

2.3.2 NEON, SVE and SVE2 Extensions

NEON and SVE are two SIMD extensions. NEON is an extension integrated in the cortex A design since the version 7, it is 128-bit based. SVE (Scalable Vector Extension) is an extension mostly used for HPC (High Performance Computing) and can operate on 128, 256, 512, 1024, and 2048 bits. SVE2 is the last extension added on Armv9-A to upgrade and expand SVE beyond HPC workloads, enabling its adoption in a wider range of general-purpose processors, including mobile and embedded systems.

Currently, NEON is the current standard when implementing an optimized version.

2.3.3 AES Extension

AES is an instruction set allowing ARM to perform AES. As we saw in the subsection 2.2.2 and table 2.1 all the instructions are usable for AES 128 and 256. But, we can note that AES-NI and AES for ARM are built differently as shown on table 2.2 and that it doesn't possess instruction for roundkey. See the Appendix A.3 for the instructions available.

	x86 AES-NI	ARM AES
AddRoundKey		AESE
SubBytes	AESENC	
ShiftRows		
MixColumns		AESMC
AddRoundKey		

Figure 2.2: AES single round encryption AVX2 and NEON

2.3.4 SHA

ARM unlike x86 possesses instructions for more versions. It has instructions for SHA-2 version SHA-256 and SHA-512 and for SHA-3. See the Appendix A.4 for the instructions available.

2.4 Comparison of x86 and ARM

2.4.1 SIMD

Here is a comparison of SIMD for x86 and ARM solutions:

	x86 AVX2	NEON	ARM SVE	SVE2
Architecture	x86-64	ARMv7-A+	ARMv8.2-A+	ARMv9-A+
Included	Yes	Yes ²	Optional	Optional
Vector width	256-bit	128-bit	128–2048 bit	128–2048 bit
Integer types	8/16/32/64-bit	8/16/32/64-bit	8/16/32/64-bit	8/16/32/64-bit

Table 2.3: SIMD Comparison

Working hypothesis following those observations : NEON optimizations for the 3 signatures schemes could require more cycles for the vectorized parts than AVX2. Leading to a worst reduction time for NEON implementations from the reference implementation.

2.4.2 AES

In Table 2.4, I reported the results from the paper: *Advanced Encryption Standard New Instructions (AES-NI) Analysis: Security, Performance, and Power Consumption*[9]. The paper doesn't tell the frequency used for the ARM processor. But for this type, there only exist a version at 1.5 GHz and another one at 2 GHz. The AES-NI processors used were at 1.5 GHz for their experiment. Since execution time is directly related to the number of cycles and the processor frequency, we can say that AES-NI requires fewer cycles than the AES ARM implementation.

We can note that only MAYO and MQOM use AES which can give an advantage in term % of acceleration.

	x86 AES-NI	ARM AES
Implementation type	Dedicated instructions	Dedicated instructions
Architecture	x86-64	ARMv8-A+
Uses SIMD	Yes	Yes
Throughput [MB/s]	≈ 920	≈ 355
Constant-time	Yes	Yes

Table 2.4: AES Comparison

Working hypothesis following those observations : Since the performances for ARM AES are much lower than the one for AES-NI, MAYO and MQOM should show worst reduction time for an ARM optimized version than for an AVX2 version from the Reference version.

2.4.3 SHA

As we saw, for SHA-2, AVX2 and NEON have equivalent instructions. For SHA-3, AVX2 doesn't have specialized instructions. So, NEON has an advantage over AVX2 for SHA-3. But a

²NEON is optional in ArmV7-A but mandatory in ArmV8-A

SHA-3 accelerated implementation falls outside the scope of this Master’s thesis. The analysis will be limited to the ARMv8 implementation.

2.4.4 Final note

Although the measurements suggest that x86 provides better acceleration, it is difficult to determine which architecture is better because the measurements, in cycles here, are not directly comparable.

As we will observe further down, the numbers of cycles differ from the ones given in the specification papers for the same architecture because although two processors may execute exactly the same signature scheme, the number of cycles required to complete the execution can still differ. These differences are mainly explained by variations in processor microarchitecture. One important factor is the maximum number of instructions that a processor can execute per cycle (IPC). Processors with a higher IPC are able to exploit more instruction-level parallelism, allowing several instructions to be executed simultaneously.

Another important factor is the structure of the instruction pipeline and the latency of execution units. Even if two processors implement the same instruction set, the internal organization of their pipelines and execution units may differ, resulting in different execution latencies for certain instructions.

Memory hierarchy also plays a significant role. Differences in cache size, cache latency, and cache policies may lead to different numbers of cache misses, which significantly increase the number of cycles required due to the higher latency of main memory accesses.

Finally, mechanisms such as branch prediction and out-of-order execution can affect performance. More advanced branch predictors reduce the number of pipeline flushes caused by incorrect predictions, while more efficient instruction scheduling can hide certain latencies. As a result, two processors running the same code may still exhibit different execution times and cycle counts due to these microarchitectural differences.

2.5 Compiling the signature scheme

The implementations are compiled using the compiler’s `-O3` optimization level. This optimization level enables a set of aggressive optimizations designed to improve runtime performance, such as function inlining and loop transformations.

Although `-O3` does not always guarantee the best performance for all programs, it is commonly used in experimental evaluations because it provides an aggressively optimized version of the code.

However, the use of aggressive compiler optimizations requires some caution. One important risk is the presence of undefined behavior in the implementation. Compilers generally assume that programs follow the rules of the language and that undefined behavior does not occur. When this assumption is violated, the compiler may apply transformations that lead to incorrect or unexpected behavior, especially when high optimization levels such as `-O3` are used.

For that reason, when evaluating the performance of cryptographic algorithms, it is important to carefully verify the correctness of the implementation and ensure that the code does not rely on undefined behavior before measuring performance.

For all the signature schemes, I verified that the same optimization level was used before measuring their performance.

2.6 Benchmark

The benchmark is divided into two different sections: Overall and Internal. Overall is the category that measures the execution of the key generation, signing and verification algorithms. Internal provides a measurement of the individual parts of the different algorithms. The sections are

divided according to the pseudocode provided by the specification paper of the signature scheme being benchmarked. Minor changes in the divisions are made when the code differs or combines multiple lines of the pseudocode. The size of the message being signed is always 32-bit long.

2.6.1 Signature-scheme benchmark

Overall

```
SCHEME/
├─ + build_bench.sh
├─ + bench.sh
├─ + log_bench_ref.txt
├─ + log_bench_avx2.txt
├─ + log_bench_armref.txt
├─ + log_bench_neon.txt
├─ + beautify_data.py
└─ ...
```

Figure 2.3: Files added or modified for the Overall benchmark

As shown in Figure 2.3, I created files to ease the benchmarking process. `build_bench.sh` takes the implementation we want to compile as an argument. `bench.sh` runs the compiled implementation and saves the raw data to the corresponding log file. Finally, `beautify_data.py` transforms the raw data into a formatted CSV file in another directory dedicated to the processed data.

The benchmark measures key generation, signing, and verification algorithms 10 000 times each. Then the mean result, in kilocycles, is stored in a CSV. We can measure them across 3 implementations: **reference**, **avx2** and **neon**. **reference** and **armref** compile the same implementation, the original implementation without explicit vectorization, but the results are stored in different files. This way, we can save the results of the reference implementation of both architectures without rewriting the same file.

To measure the algorithms, I used the benchmark provided with each signature scheme.

How to benchmark:

- Compile the desired implementation using `bash build_bench.sh <arg>`, with `<arg> = reference, avx2, armref or neon`.
- Run the benchmark using `bash bench.sh <arg>`, with `<arg> = reference, avx2, armref or neon`.

Internal

```
SCHEME/
├─ + build_bench_inline.sh
├─ + bench_inline.sh
├─ + log_bench_avx2_inline.txt
├─ + log_bench_neon_inline.txt
├─ + beautify_data_inline.py
└─ ...
```

Figure 2.4: Internal bench files modification

As shown in Figure 2.4, I created files to ease the benchmarking process. `build_bench_inline.sh` takes the implementation we want to compile as an argument. `bench_inline.sh` runs the compiled implementation and saves the raw data to the corresponding log file. Finally, `beautify_data_inline.py` transforms the raw data into a formatted CSV file in another directory dedicated to the processed data.

The benchmark measures the internal parts of key generation, signing and verification algorithms 10 000 times each. Then the mean result in kilocycles is stored in a CSV. We can measure them over 2 implementations: `avx2` and `neon`.

To measure the algorithms, I added tags to get the cycle count and saved the difference between the cycle counts recorded at the two tags into the raw data file. The tags are placed before and after a loop. Each entire loop is treated as a single benchmarked section.

How to benchmark:

- Compile the desired implementation using `bash build_bench_inline.sh <arg>`, with `<arg> = avx2` or `neon`.
- Run the benchmark using `bash bench_inline.sh <arg>`, with `<arg> = avx2` or `neon`.

For CROSS, I started making measure inside loops and condition. However, due to time constraints, I was unable to perform the same measurements for the other signature schemes, and therefore they are not analysed in this thesis.

2.6.2 Implementation benchmark

To bench a code in C, the principle is the same on all architecture. Modern processors provide a hardware cycle counter that can be read during program execution. The value of this counter is recorded before and after the section of code to be measured. The difference between these two values gives an estimate of the number of CPU cycles required to execute that section.

For the Overall benchmark, each signature scheme has its own way to perform this measurement. For the Inline benchmark, I needed to create tags. I will explain them in the next sections.

AVX2

For the AVX2 tag, I used the file `timing_and_stat.h` from CROSS. This is pretty straight forward to use. It is a a function call that invokes an assembly instruction as shown on Figure 2.5.

AVX2 tag

```
#if defined(CYCLE_MAIN)
    a_time = x86_64_rtdsc();
#endif

-----

uint64_t x86_64_rtdsc(void) {
    unsigned long long result;
    __asm__ __volatile__(
        "rdtscp;"
        "shl $32, %%rdx;"
        "or %%rdx, %%rax"
        : "=a"(result)
        :
        : "%rcx", "%rdx");
    return result;
}
```

Figure 2.5: AVX2 tag

NEON

For the NEON tag, I directly used an assembly instruction to record the cycle count. This can be seen in Figure 2.6.

NEON tag

```
#if defined(CYCLE_MAIN)
asm volatile("mrs %0, cntvct_el0" : "=r"(a_time));
#endif
```

Figure 2.6: NEON tag

2.7 Benchmarking Platforms

The platforms used for benchmarking were my laptop and a Raspberry Pi 5 accessed through SSH. The specifications are shown on Table 2.5

Specification	x86	ARM
CPU	Intel Core i7-1065G7 @ 1.30 GHz	Cortex-A76
Architecture	x86_64	aarch64
Logical CPUs	8	4
RAM	23 GB	7.9 GB
Operating System	Parrot Security 6.2 (lorikeet)	Debian GNU/Linux 12 (bookworm)
Kernel	6.10.11-amd64	6.12.47+rpt-rpi-2712
SIMD	AVX2	NEON

Table 2.5: Specifications of the benchmarking platforms

Chapter 3

CROSS

3.1 Description of the CROSS signature scheme

CROSS[10] is a digital signature scheme whose security relies on the hardness of the Restricted Syndrome Decoding Problem (R-SDP) and the Restricted Syndrome Decoding Problem with subgroup (R-SDP(G)) [11]. CROSS is designed to achieve EUF-CMA security under these hardness assumptions. At its core, the scheme executes, for each round t , a Schnorr-like zero-knowledge proof called CROSS-ID [11] (see Figure 3.2). This protocol serves as the building block of the signature.

To combine multiple rounds into a single signature, CROSS uses a Merkle tree structure to commit efficiently to all round commitments, while a Seed tree is employed to derive all round randomness from a compact initial Seed.

According to the 2024 NIST report [12], “*While there are no known attacks against either of the two CROSS variants, the problems are relatively new,*” highlighting that the underlying assumptions, although promising, have not yet been studied as extensively as more established hardness assumptions.

CROSS is available in several variants. First, there are different parameter sets corresponding to distinct security levels, ensuring consistent security across signatures. Second, the scheme can be instantiated either using R-SDP or R-SDP(G). Finally, each instantiation can be optimized for one of three profiles: balanced, fast, or small. These trade-offs arise because the underlying proof is asymmetric, with one of the responses being larger than the others.

As noted in the 2024 NIST report [12], “*The security of R-SDP is considered more conservative than that of R-SDP(G). However, schemes based on the R-SDP(G) problem offer better performance in both signature size and algorithm runtime.*”

The next sections present how CROSS works. **I purposely omitted the Seed Tree and Merkle Tree from the abstract representation.**

3.1.1 Parameters

Symbol	Meaning
p	Prime number defining the finite field $\mathbb{F}_p$
$\mathbb{F}_p$	Finite field with p elements
$\mathbb{F}_p^*$	Multiplicative group $\mathbb{F}_p \setminus \{0\}$
$\star$	Component-wise multiplication
G	Subgroup used to define the restricted vectors
n	Code length and length of the restricted vectors
k	Code dimension, with $k < n$
m	Parameter defining the size of the subgroup $\mathcal{G}$
H	Parity-check matrix, $H \in \mathbb{F}_p^{(n-k) \times n}$
e	Secret restricted vector, $e \in \mathcal{G}$
s	Public syndrome, $s = eH^\top$

λ	Security parameter
t	Number of CROSS-ID repetitions
w	Weight of the second challenge
Msg	Message to be signed
sk	Secret key
pk	Public key
σ	Signature
$Seed[i]$	Seed used to generate the randomness of round i
$e'[i], u'[i]$	Random vectors generated from $Seed[i]$ during round i
$v[i]$	Vector computed as $v[i] = e \star (e'[i])^{-1}$
$cmt_0[i], cmt_1[i]$	Commitments generated during round i
$chall_1[i]$	First challenge of round i , with $chall_1[i] \in \mathbb{F}_p^*$
$y[i]$	First response of round i
$digest_y[i]$	Hash digest of $y[i]$
$chall_2$	Second challenge vector, $chall_2 \in \mathcal{B}(t, w)$
$resp[i]$	Response to the second challenge for round i
$Salt$	Random salt used during signature generation
$digest_{cmt_0}$	Root digest committing to the $cmt_0[i]$ values through the Merkle Tree
$digest_{chall_2}$	Digest used in the generation of the second challenge
$Path$	Minimal set of SeedTree nodes required to reconstruct the necessary $Seed[i]$ values
$Proof$	Minimal set of Merkle Tree nodes required to reconstruct $digest_{cmt_0}$
Hash	Cryptographic hash function with codomain $\{0, 1\}^{2\lambda}$
$\text{CSPRNG}_{\mathcal{S}}(\cdot)$	Cryptographically secure pseudorandom number generator with output in the set $\mathcal{S}$

3.1.2 CROSS-ID Protocol

Private Key : $e \in G$

Public Key : $G \subseteq \mathbb{E}^n$, $H \in \mathbb{F}_p^{(n-k) \times n}$, $s = eH^T \in \mathbb{F}_p^{n-k}$

Prover	Verifier
// Sample Seed to compute e', u'	
Seed $\xleftarrow{\$} \{0, 1\}^\lambda$	
$(e', u') \leftarrow \text{CSPRNG}(\text{Seed})$	
// Compute v, u, s'	
$v = e \cdot (e')^{-1}$	
$u = v \cdot u'$	
$s' = uH^T$	
// Commitments	
$cmt_0 = \text{Hash}(s' v)$	
$cmt_1 = \text{Hash}(u' e')$	
	$\xrightarrow{cmt_0, cmt_1}$
	$\xleftarrow{chall_1}$
	// Sampling first challenge
	$chall_1 \xleftarrow{\$} \mathbb{F}_p^*$
// First response	
$y = u' + chall_1 e'$	
$digest_y = \text{Hash}(y)$	
	$\xrightarrow{digest_y}$
	$\xleftarrow{chall_2}$
	// Sampling second challenge
	$chall_2 \xleftarrow{\$} \{0, 1\}$
// Second response	
$resp = \begin{cases} (y, v) & \text{if } chall_2 = 0 \\ \text{Seed} & \text{if } chall_2 = 1 \end{cases}$	
	$\xrightarrow{resp}$
	// Verification

If $\text{chall}_2 = 0$:
 $y' = v \cdot y$
 $s' = y' H^T - \text{chall}_1 s$
 Accept if:
 1) $\text{Hash}(y) = \text{digest}_y$
 2) $\text{Hash}(s' \| v) = \text{cmt}_0$
 3) $v \in G$
If $\text{chall}_2 = 1$:
 $(e', u') \leftarrow \text{CSPRNG}(\text{Seed})$
 $y = u' + \text{chall}_1 e'$
 Accept if:
 1) $\text{Hash}(y) = \text{digest}_y$
 2) $\text{Hash}(u' \| e') = \text{cmt}_1$

Table 3.2: CROSS-ID

3.1.3 Key Generation

Algorithm 1 Key Generation (Abstract Form)

Public parameters: H

Key Generation:

1: $e \xleftarrow{\$} \mathbb{E}^n$

2: $s = eH^\top$

Output: $(sk, pk) \leftarrow (e, s, H)$

3.1.4 Signing Algorithm

Algorithm 2 Sign (Abstract Form)

Input: sk, Msg

Public parameters: H

Signing:

1: **for** i **from** 1 **to** t **do**

2: $\text{cmt}_0[i], \text{cmt}_1[i] \leftarrow \text{CROSS-ID}_{\text{Commitment}}$

3: $\text{chall}_1[1], \dots, \text{chall}_1[t] \leftarrow \text{CSPRNG}(\text{Hash}(Msg \| \text{cmt}_1[1] \| \dots \| \text{cmt}_1[t] \| \text{cmt}_0[1] \| \dots \| \text{cmt}_0[t]))$

4: **for** i **from** 1 **to** t **do**

5: $y[i] \leftarrow \text{CROSS-ID}_{\text{First response}}$

6: $\text{chall}_2[1], \dots, \text{chall}_2[t] \leftarrow \text{CSPRNG}(\text{Hash}(y[1] \| \dots \| y[t] \| \text{chall}_1[1] \| \dots \| \text{chall}_1[t]))$

7: **for** i **from** 1 **to** t **do**

8: $\text{resp}[i]_{0/1} \leftarrow \text{CROSS-ID}_{\text{Second response}}$

Output: $\sigma = (\text{digest}_{\text{cmt}}, \text{digest}_{\text{chall}_2}, \text{resp}[1], \dots, \text{resp}[t])$

3.1.5 Verification Algorithm

Algorithm 3 Verify (Abstract Form)

Input: pk, Msg, σ

Public parameters: H

Verifying:

- 1: **for** i **from** 1 **to** t **do**
- 2: $verif[i] \leftarrow \text{CROSS-ID}_{Verification}$

Output: $Verif = (verif[0] \text{ and } \dots \text{ and } verif[t])$

3.1.6 SeedTree and Merkle-Tree

The CROSS protocol requires the construction of two types of trees: a SeedTree and a Merkle Tree. They are both employed in the protocol for completely different purposes.

The SeedTree enables generation of many Seeds using a CSPRNG starting from a single root Seed. This single root Seed represents the commitment to all other Seeds. However, the signature doesn't contain all the Seeds. It is only composed of a limited amount of intermediate nodes, enough to reconstruct the needed Seeds.

The Merkle Tree allows the commitment to several values (for example, commitment or response values). This way the verifier only needs to recalculate the root value to ensure the correctness of the signature.

CROSS divide itself into three different versions: small, fast, and balanced, each providing a certain trade-off between the two properties discussed above. The small version aims to minimize the amount of transferred data. The fast version aims to reduce the computational overhead by having a tree with only 3 levels. The balanced set attempts to provide a compromise between the two extremes.

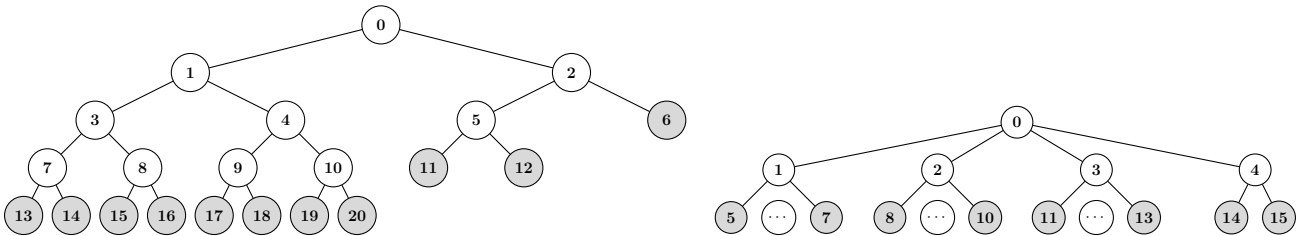


Figure 3.1: Illustrative tree structures for $t = 11$: balanced and small versions (left) vs. fast version (right). Gray nodes represent leaves.

• SeedTree Functions

- **SeedLeaves** is a function that takes a root Seed and a Salt as input and derives it consecutively to get t $Seed[i]$ by computing a tree. In the balanced and small versions it creates a binary tree top-to-bottom, expanding each parent node into two children using a CSPRNG. In the fast version, it uses a 3-level tree, expanding the root into four intermediate Seeds which are then each expanded into their respective round Seeds.
- **SeedPath** is a function that takes the root Seed, Salt, and the second challenge $chall_2$ as input and returns the minimal subset of SeedTree nodes (called Path) that allows the verifier to reconstruct all $Seed[i]$ where $chall_2[i] = 1$. In the fast version, it simply returns exactly those w leaf $Seed[i]$ indicated where $chall_2[i] = 1$.
- **RebuildLeaves** is a function that takes the Path, Salt, and $chall_2$ as input and recomputes each $Seed[i]$ for all rounds where $chall_2[i] = 1$, from the SeedTree using the nodes provided in Path and expanding them downward. In the fast version, it directly returns the t $Seed[i]$ from the Path data.

- **SeedTree Functions**

- **TreeRoot** is a function that takes the t commitments $cmt_0[0], \dots, cmt_0[t-1]$ as leaves and computes the root of the commitment tree through iterative hashing pairwise each leaf or node in the balanced and small versions, or for the fast version, by hashing groups into four intermediate nodes then a single root.
- **TreeProof** is a function that takes the commitment leaves $cmt_0[i]$ and the second challenge $chall_2$ as input and returns, for the balanced and small versions, the minimal subset of Merkle tree nodes (called Proof) that a verifier needs to recompute the Merkle root, given the $cmt_0[i]$ where $chall_2[i] = 0$. In the fast version it simply selects the w leaves $cmt_0[i]$ where $chall_2[i] = 1$.
- **RecomputeRoot** is a function that takes as input the recomputed commitments $cmt_0[i]$ (for $chall_2[i] = 0$), the Proof nodes from the signature, and $chall_2$, and outputs the reconstructed Merkle root $digest_{cmt_0}$. In the balanced and small versions, the verifier places its recomputed $cmt_0[i]$ values onto the tree leaves, then traverses the tree from bottom to top, filling in any missing nodes from the Proof and hashing pairs together bottom-to-top. In the fast version, the Proof is composed only of the leaf $cmt_0[i]$.

3.2 Current Implementations

Category	Implementation
General	Reference implementation in C
Optimized	• AVX2-optimized implementation in C • NeonCROSS: Vectorized Implementation of Post-Quantum Signature CROSS on Cortex-A72 and Apple M3[13] • Cortex-M4 implementation in C

Table 3.3: Available implementations of CROSS

3.3 CrossNEON

3.3.1 Overview

CrossNEON[13] is a January 2026 constant-time implementation providing a fully vectorized version of the CROSS signature scheme on ARMv8-A architectures, with a focus on exploiting SIMD parallelism through the Neon extension. This implementation is not just a ported version but an upgrade of the previous AVX2 version. All optimizations presented afterward are NEON optimized.

Modular Arithmetic

The paper tries to find the best reduction operation for CROSS. In the signature scheme, we have $\mathbb{F}_p$ groups moduli primes 127 for R-SDP and 509 for R-SDP(G). 127 is a Mersenne prime since $127 = 2^7 - 1$ and 509 is a pseudo-Mersenne prime (PMP) since $509 = 2^9 - 3$. The current reduction method in the reference version is Mersenne prime reduction for 127 and Barrett-like reduction for 509. CrossNEON compared four different methods: Barrett, Montgomery, pure PMP and PMP hybrid,

PMP hybrid reduction being a combination of PMP with single-width Barrett. After comparing these methods, the authors decided on different reductions for each prime p . When $p = 127$, CrossNEON uses pure PMP reduction. When $p = 509$, CrossNEON uses pure PMP reduction and Barrett-like reduction at the end to bound it into $\mathbb{F}_p$. This technique is called PMP hybrid. See Appendix B for more details.

Vectorized Matrix-Vector Multiplication

For the matrix-vector multiplication product goes from an inner product version to a linearized version to lower the number of cache misses. They also add a register-blocked approach, multiplying, accumulating, and then reducing directly in the result register and thus eliminating unnecessary memory writes. All using is vectorized instructions to increase the throughput.

To optimize the vectorized matrix-vector multiplication, CrossNeon needs to find a trade-off between the smallest size of registers to maximize the throughput while minimizing the number of reductions. The paper answers this problem like this: *"For $p = 127$, the product of two field elements fits within 14 bits, and after accumulation and lazy reductions, the intermediate result remains well within 16 bits. Thus, we select 16-bit lanes (i.e., .8h vectors) to enable efficient Neon processing without overflow. In contrast, for $p = 509$, the product of two field elements can reach up to 18 bits. To safely accommodate intermediate results and support modular reduction, we use 32-bit lanes (i.e., .4s vectors)."*

The choice of lane size and reduction method leads to distinct lazy reduction strategies for each prime p . For $p = 127$, the author proved that reducing every four accumulations ensures no overflow while maximizing performance. At the end, two reductions are followed by normalization of the double-zero representation (127 and 0). For $p = 509$, since we are on 32-bits we don't need to reduce until the end of the accumulation and so we only need PMP hybrid which will execute two PMP reductions that bound the value within 15 bits and then apply an unsigned Barrett reduction.

Vectorized Tree Structures

For the SeedTree (GGM tree) and the Merkle Tree, the parent-children dependency of the trees constrains the throughput. The SeedTree expands top-down where each node depends on its parent, while the MerkleTree aggregates bottom-up where each node depends on its children. These sequential dependencies limit parallelism, and the computational intensity of CSPRNG and hash operations further exacerbates the bottleneck.

CrossNEON addresses this by batching independent nodes at the same tree level together. Since Neon supports only two-way parallel Keccak (compared to four-way on AVX2), the strategy processes two pairs of nodes simultaneously using CSPRNGx2 and Hashx2 primitives. For the SeedTree, two parent nodes are expanded in parallel to produce four child nodes at once. For the Merkle Tree, two pairs of child nodes are hashed in parallel to compute two parent nodes simultaneously. When the number of nodes at a given level is not divisible by four, the remaining nodes fall back to the non-vectorized Hash.

The balanced and small versions use truncated binary trees and apply this two-way batching uniformly across levels, processing pairs of nodes at each level until the root, which is computed with a single sequential Hash.

The fast version uses a compact three-level tree and requires additional case handling based on $t \bmod 4$, since the middle-layer nodes may have unequal numbers of children depending on t . When $t \bmod 4 = 0$ or 2 , all middle-layer nodes have equal numbers of children, so two instances of CSPRNGx2 (for SeedLeaves) or Hashx2 (for TreeRoot) are sufficient and outputs are written directly to their target positions, avoiding any memory copies. When $t \bmod 4 = 1$ or 3 , some middle-layer nodes have unequal numbers of children. For SeedLeaves, one CSPRNGx2 handles the equal pair directly, while for the unequal pair the outputs are extended to the larger length and copied to their target positions. For TreeRoot, the two hashes of the unequal pair are computed separately without batching. This single check on $t \bmod 4$ efficiently exploits Neon's two-way parallelism while keeping

memory copies minimal.

3.3.2 Portability to AVX2

As we can see the *Modular Arithmetic* and the *Vectorized Matrix-Vector Multiplication* optimizations are not restricted to NEON. So this can be ported to the reference version and potentially benefit also the AVX2 implementation. The third optimization *Vectorized Tree Structures* is only an adaptation of something already existing in AVX2.

3.4 Analysis of internal performance

3.4.1 Key Generation

Overall

In Figure 3.2, the AVX2 implementation of the key generation algorithm achieves an average time reduction of 25.63%. In Figure 3.3, the CrossNEON implementation achieves an average time reduction of 40.38% for the key generation algorithm.

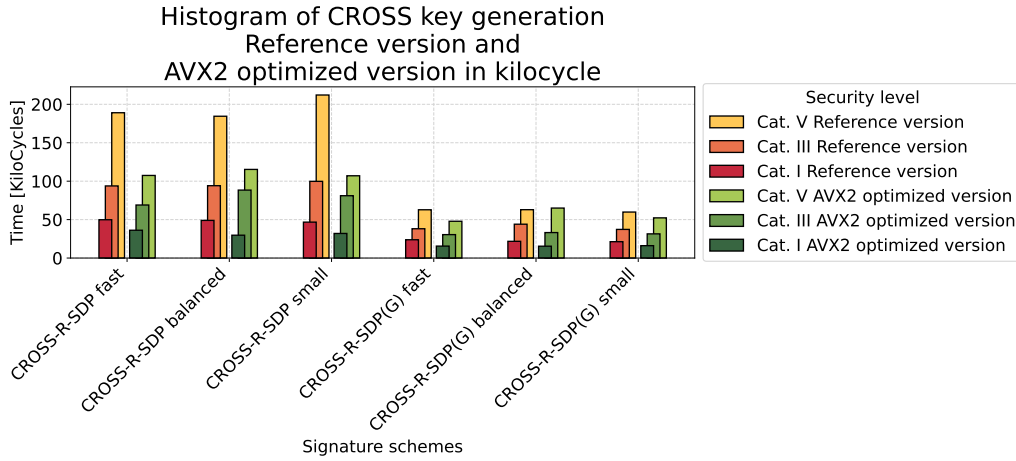


Figure 3.2: Comparison of CROSS Key generation, Reference and AVX2 versions (Table C.4)

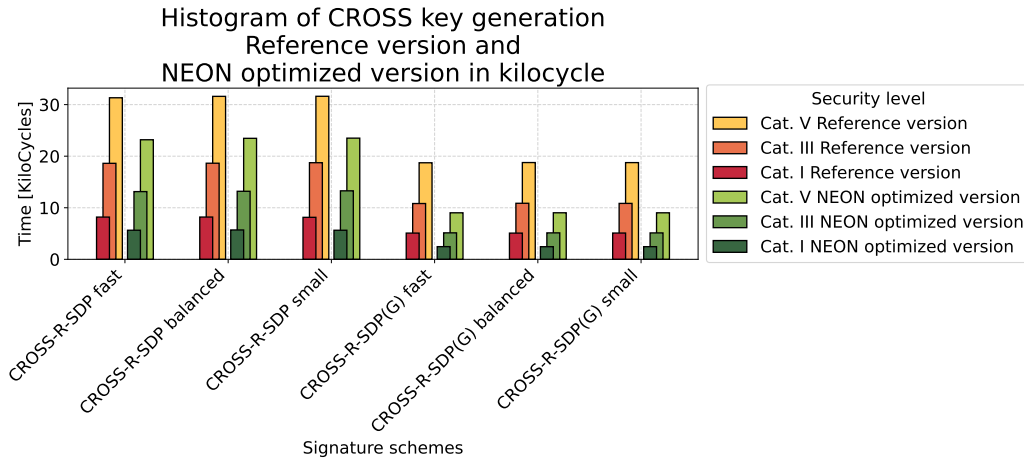


Figure 3.3: Comparison of CROSS Key generation, Reference and NEON versions (Table C.5)

Internal

The internal analysis is based on the pseudocode given by the reference paper[11]. I divided the code into these sections : "row 1 and 8", "declaration", "row 2", "row 3", "row 5", "row 7 (row 4 and 6 are implicit here)" and "row 9".

In Figure 3.4, we can observe that the three rows taking the most time are in order: row 3, row 7 and row 5. Row 3 and 5 are generate pseudorandom data from a given seed using a CSPRNG. and row 7 represents the syndrome computation: $s \leftarrow eH^T$.

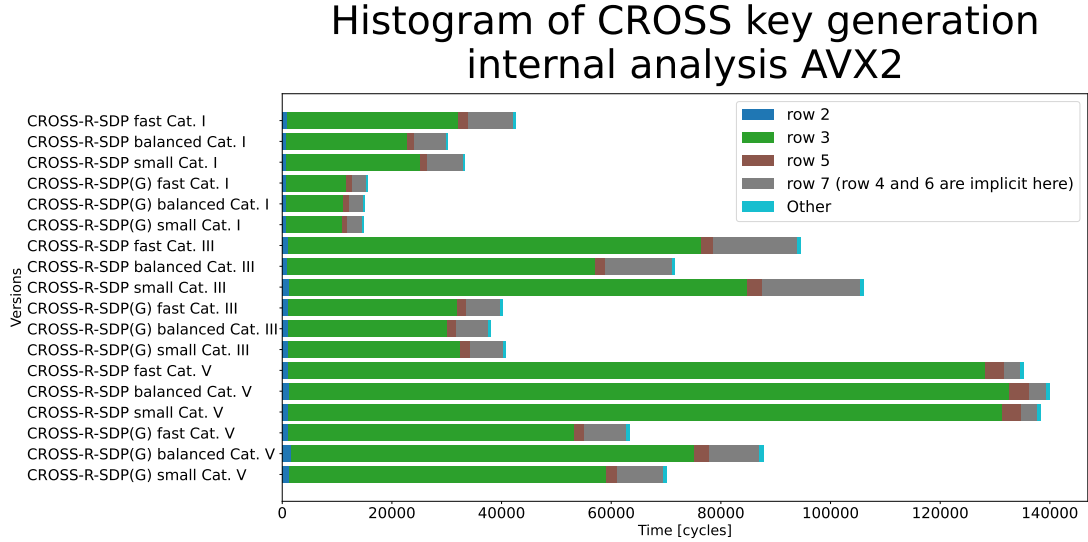


Figure 3.4: CROSS internal analysis of key generation for AVX2 implementation¹ (Table C.10)

In Figure 3.5, we can observe similar observations as for the Figure 3.4. When comparing those two figures, we can observe that we have more disparity between performances of the different versions for the AVX2 implementation than for the NEON implementation.

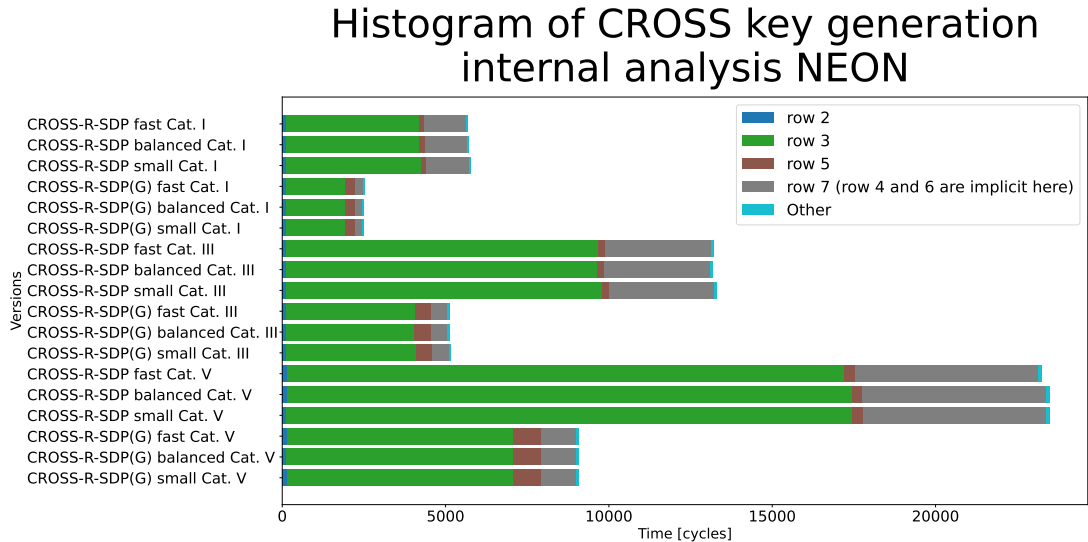


Figure 3.5: CROSS internal analysis of key generation for NEON implementation (Table C.11)

In Figure 3.6, we can observe the difference of proportion between AVX2 and NEON implementation. We can observe that the difference in proportion is more pronounced with an upper category. We can observe that the row 7 takes more cycles in NEON in proportion because even the negative numbers decrease, same result for row 5. But row 3 tend to show the opposite result as the others. The other Rows are stable.

¹The sections taking less than 1% are classified as other.

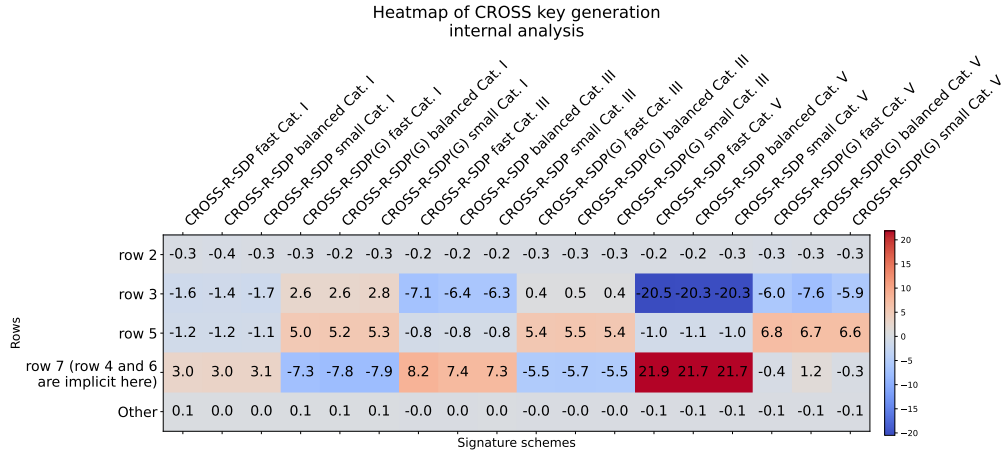


Figure 3.6: Difference in proportion (NEON - AVX2) for CROSS key generation internal analysis (Tables C.10, C.11)

3.4.2 Signing Algorithm

Overall

In Figure 3.7, the AVX2 implementation of the signing algorithm achieves an average time reduction of 37.92%. In Figure 3.8, the CrossNEON implementation achieves an average time reduction of 61.05% for the signing algorithm.

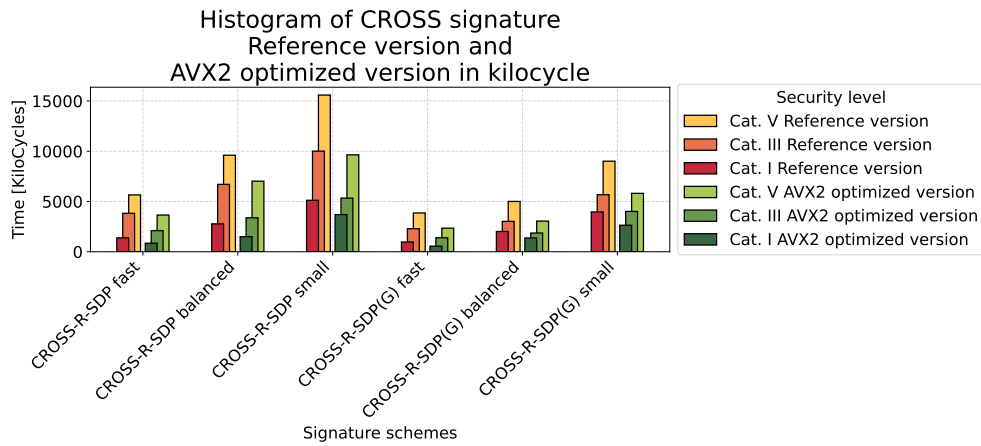


Figure 3.7: Comparison of CROSS signing algorithm, Reference and AVX2 versions (Table C.6)

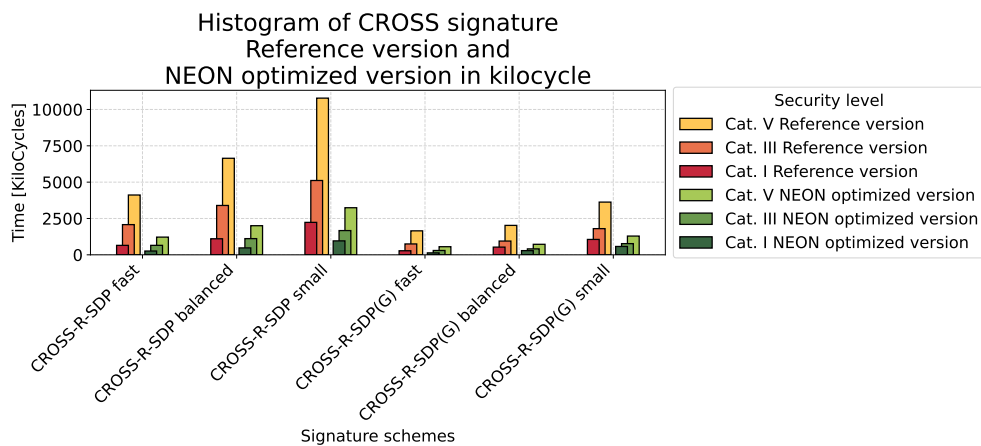


Figure 3.8: Comparison of CROSS signing algorithm, Reference and NEON versions (Table C.7)

Internal

The internal analysis is based on the pseudocode given by the reference paper[11]. I divided the code into these sections : "row 28 (preparation)", "row 1", "avx2 preparation", "row 2", "row 3", "declaration", "row 5 and 10 preparation", "declaration 2", "row 4 loop", "row 11", "row 12", "row 13", "row 14", "row 15", "row 16", "row 17 loop", "row 21", "row 22", "row 23", "row 24", "row 25 loop".

In Figure 3.9, we can observe that the rows taking the most time are in order: row 4 loop, row 21, row 11 and row 3. Row 4 loop compute v,u,s' and the first commitment. Row 21 Hash. Row 11 compute TreeRoot. Row 3 compute SeedLeaves.

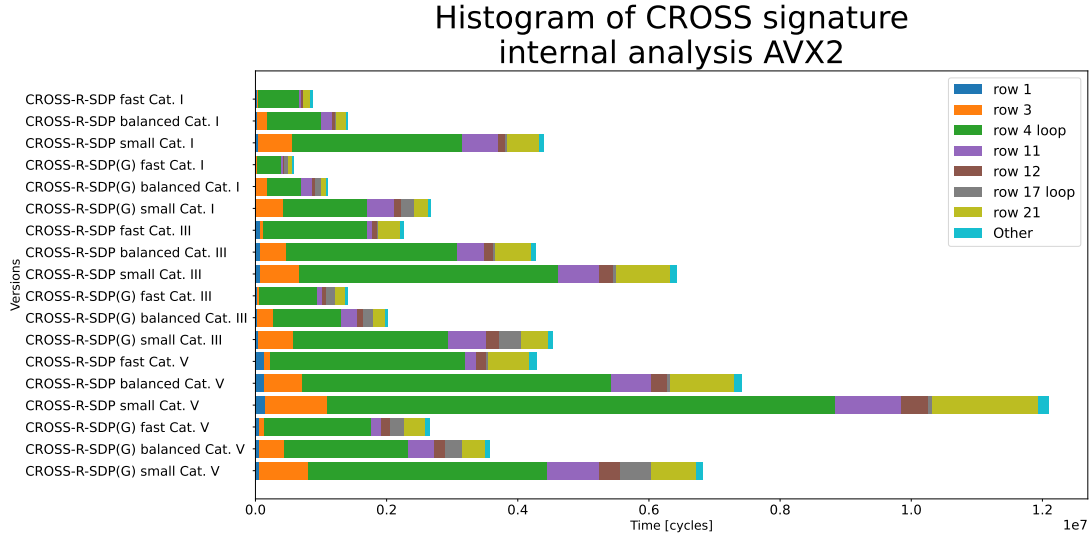


Figure 3.9: CROSS internal analysis of signing algorithm for AVX2 implementation (Tables C.12, C.13)

In Figure 3.9, we can directly see that row 4 loop takes more time overall than for the Figure 3.10. Since CrossNEON is an upgraded version from the AVX2 implementation, we could say that this result is the effects of the upgrades.

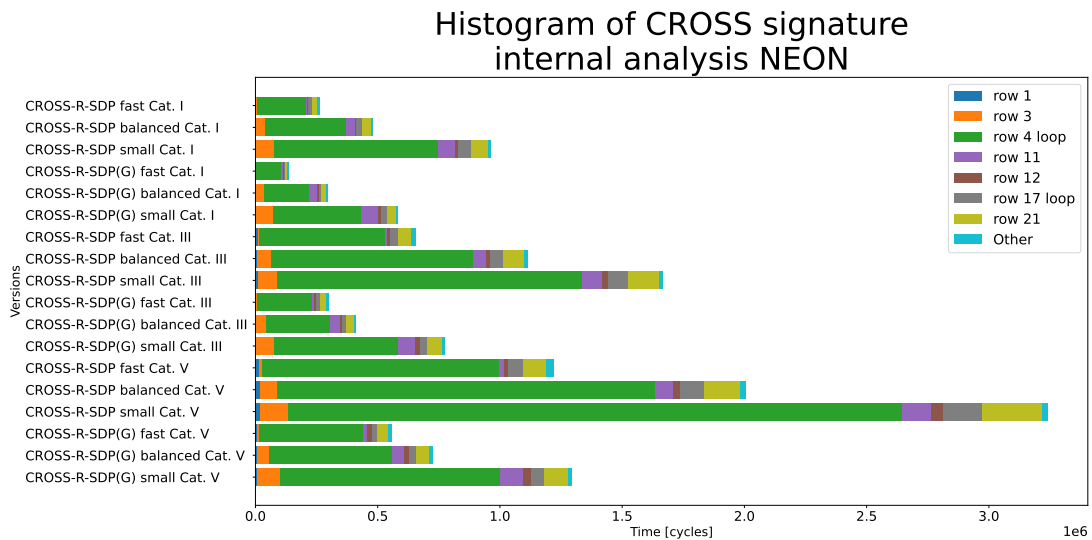


Figure 3.10: CROSS internal analysis of signing algorithm for NEON implementation (Tables C.14, C.15)

In Figure 3.11, we can clearly see what we were explaining just before. The row 4 loop takes a much part in the CrossNEON implementation. But, we also observe a different behavior for the row 17. Row 17 compute the first response.

Heatmap of CROSS signature internal analysis

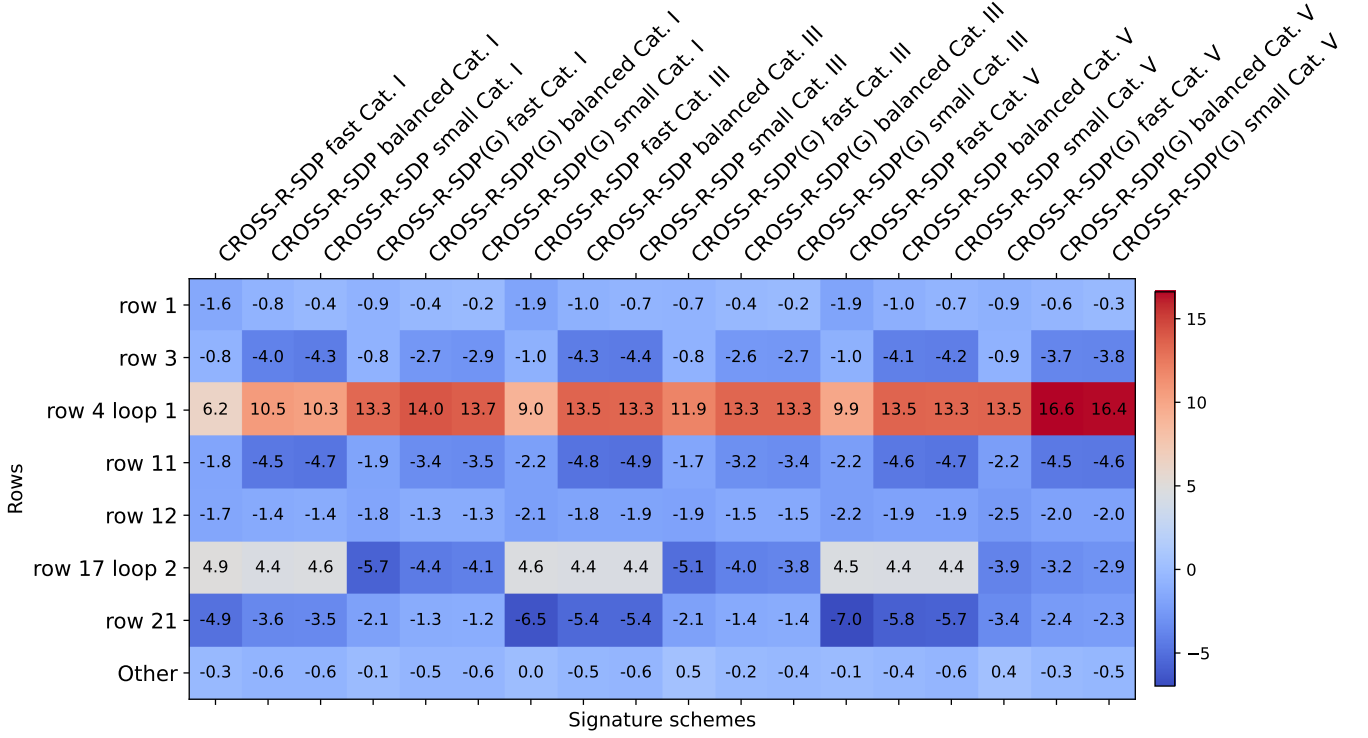


Figure 3.11: Difference in proportion (NEON - AVX2) for CROSS signing algorithm internal analysis (Tables C.12, C.13, C.14, C.15)

3.4.3 Verification Algorithm

Overall

In Figure 3.12, the AVX2 implementation of the verification algorithm achieves an average time reduction of 30.59%. In Figure 3.13, the CrossNEON implementation achieves an average time reduction of 52.79% for the verification algorithm.

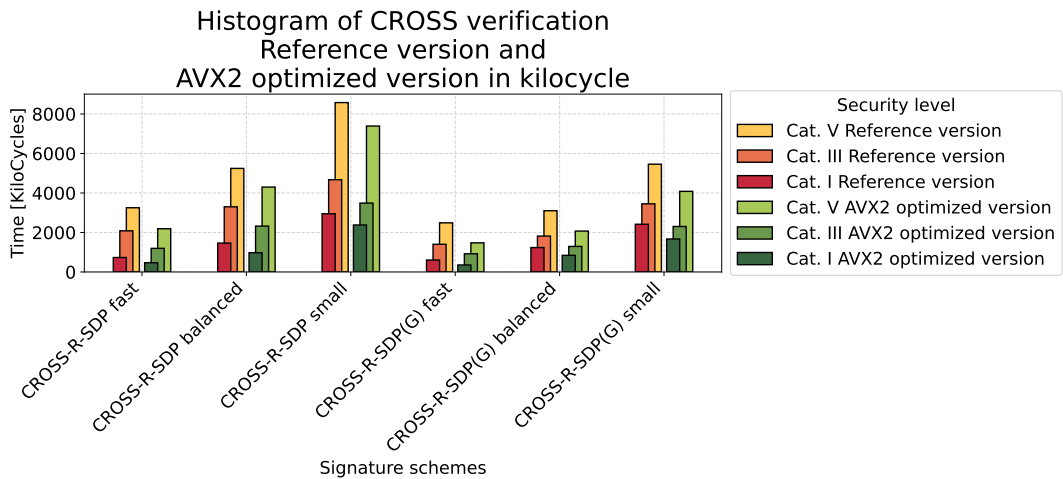


Figure 3.12: Comparison of CROSS verification algorithm, Reference and AVX2 versions (Table C.8)

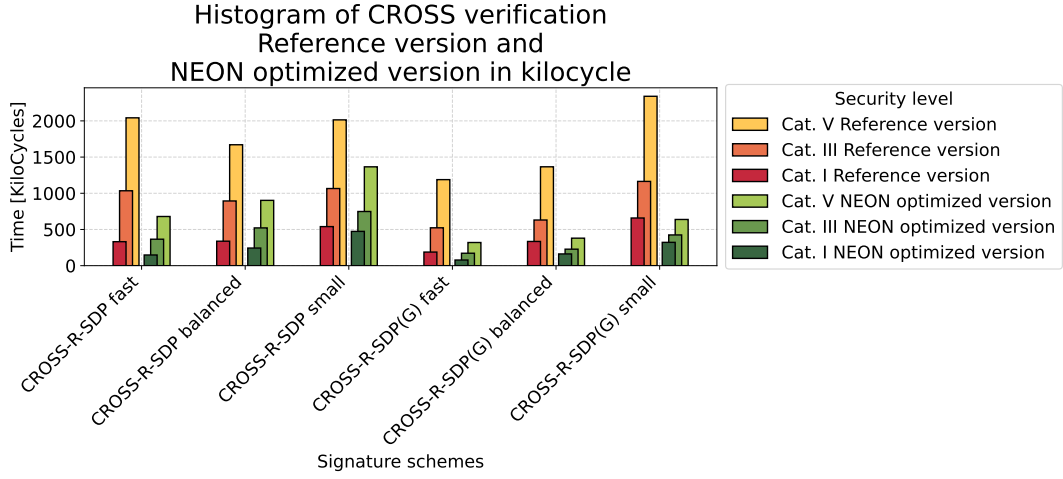


Figure 3.13: Comparison of CROSS verification algorithm, Reference and NEON versions (Table C.9)

Internal

The internal analysis is based on the pseudocode given by the reference paper[11]. I divided the code into these sections : "row 1", "preparation avx2", "unpack syndrome", "row 4", "row 5", "row 6", "row 7", "row 8", "declaration", "row 9 loop", "row 22", "row 23", "row 24", "row 25", "row 26"

In Figure 3.14, we can observe that the rows taking the most time are in order: row 9 loop and row 25. Row 9 loop compute the commitments. Row 25 Hash.

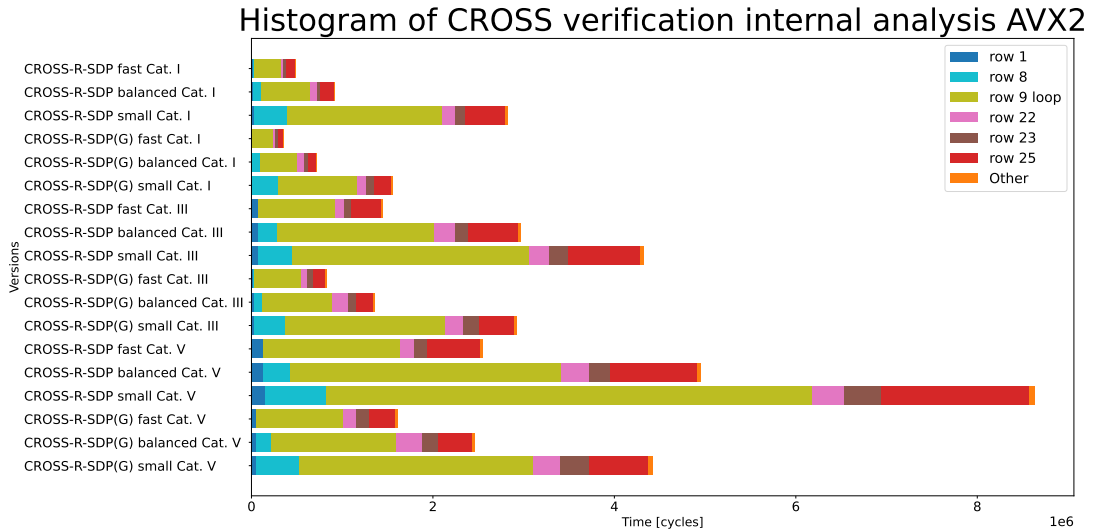


Figure 3.14: CROSS internal analysis of verification algorithm for AVX2 implementation (Tables C.16, C.17)

In Figure 3.15, We don't observe significant differences from the Figure 3.14.

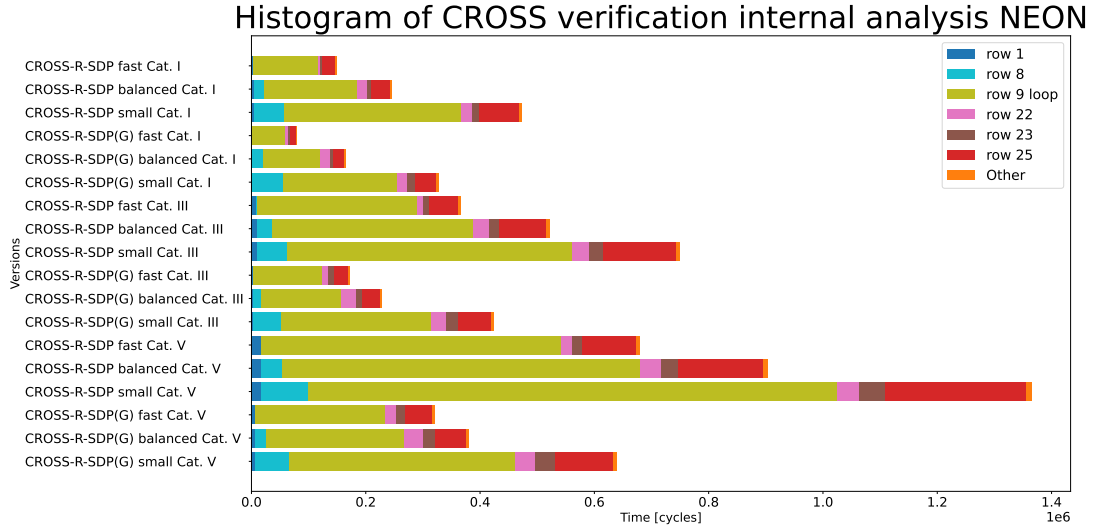


Figure 3.15: CROSS internal analysis of verification algorithm for NEON implementation (Table C.18, C.19)

In Figure 3.16, we observe that the row 9 loop take more space in in the NEON implementation than the AVX2 implementation. For the row 25, we observe an opposite effect. The other rows are stable.

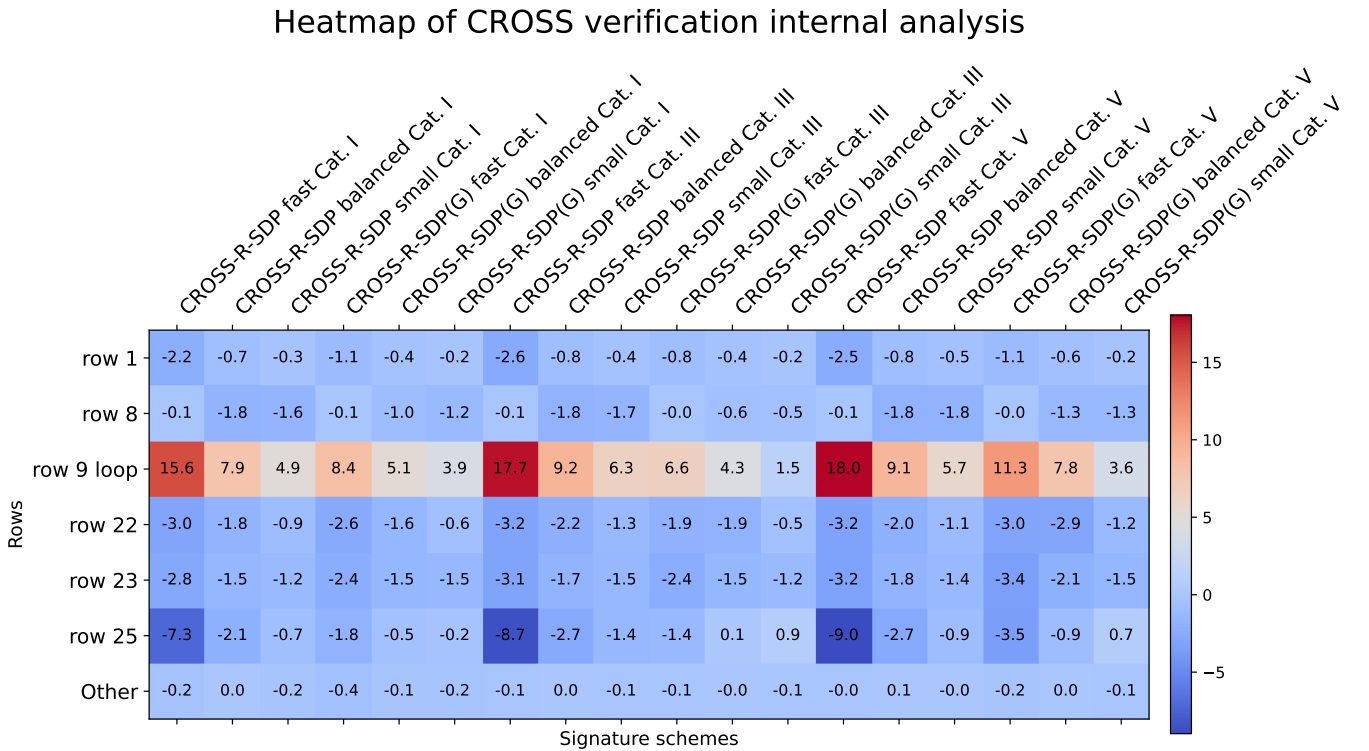


Figure 3.16: Difference in proportion (NEON - AVX2) for CROSS verification algorithm internal analysis (Tables C.16, C.17, C.18, C.19)

3.5 Conclusion and Future work

3.5.1 Conclusion

	AVX2	CrossNEON
Key generation	25.63%	40.38%
Signing algorithm	37.92%	61.05%
Verification algorithm	30.59%	52.79%

Table 3.4: CROSS: Performance comparison between AVX2 and NEON

To conclude, we saw that CrossNEON included additional optimizations, such as new modular reductions and vectorized tree structures, which are not implemented in the AVX2 version.

We can show on the Table 3.4, that the results obtained for CROSS show that both optimized implementations significantly improve the performance compared to the reference implementation. A big part could come from SHA-3 vectorization.

If my working hypothesis are exact and that an ARM implementation should have a smaller reduction time than an AVX2 implementation. This could verify that CrossNEON is a better implementation. But without comparison numbers, we can't decide so we'll need the MAYO and MQOM time reduction to decide. So currently, this does not mean that NEON is more efficient than AVX2 for CROSS.

The internal analysis also showed different proportions between both implementations. This observed differences therefore could not only from the instruction sets, but also from the additional optimizations introduced in CrossNEON.

3.5.2 Future work

For future work, we have different option:

- Port and test CrossNEON implementation on AVX2 implementation.
- Implement an ARMv8.2 implementation: on the XKCP GitHub repository [14], the Keccak Team[15] released a SHA-3 implementation using the new vectorized instructions we saw in 2.3.4.
- We can reduce the number of versions to only a small list of potential standards like, as we will see, MAYO did. Instead of studying all the different versions.
- Analyze the loop and condition internal analysis from the key generation, signing and verification algorithm. That I couldn't analyze here. And make the same analysis on other important internal function. (e.g. the Tree functions)
- When I looked into the code, I realized that the Tree functions are replaced by `memcpy` since depending on which macro is defined, the `#if` branch or the `#else` branch is compiled in the fast version. If we could directly go into the structure without copying we could gain some time.
- Some part may be more time consuming. I think it could be a good idea to port those into assembly. It could improve the performances.

Chapter 4

MAYO

4.1 Description of the MAYO signature scheme

MAYO[16] is derived from Oil and Vinegar. To understand it, I invite you to read the Introduction and design rationale page from the MAYO documentation [17], which is concise and easy to read. To quote NIST from the [12] "MAYO significantly improves the public key size relative to UOV while inheriting UOV's small signature size. While it is not as fast as UOV, MAYO is still very efficient.".

MAYO also comes with different versions: Mayo 1, Mayo 2, Mayo 3, and Mayo 5. The difference between Mayo 1 and 2 is that Mayo 1 has smaller public keys but larger signatures and Mayo 2 has smaller signatures but larger public keys, as we will see later in subsection 2.4 Comparison of signature sizes. Mayo 3 and 5 are only here to cover all security levels. To quote NIST from the 2024 report[12] "Recent results on solving underdetermined systems of multivariate equations[18] might slightly impact MAYO's security.".

The next sections present how MAYO works.

4.1.1 Parameters

Symbol	Meaning
q	Size of the finite field $\mathbb{F}_q$, with $q = 16$
n	Number of variables of the public quadratic map
m	Number of quadratic polynomials of the public map
o	Dimension of the secret oil space $\mathcal{O}$
k	Number of vectors combined during signing
$\mathcal{P}$	Public quadratic map $\mathcal{P} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$
$\mathcal{O}$	Secret oil space of dimension o
$\mathbf{O}$	Secret matrix defining the oil space
$\mathcal{P}^*$	Whipped quadratic map combining k inputs
$\mathbf{v}_i$	Random vinegar part of the i -th signature vector
$\mathbf{x}_i$	Oil variables obtained by solving the linear system
$\mathbf{t}$	Target vector obtained from the hash of the message and the salt
Msg	Message to be signed
sk	Secret key
pk	Public key
salt	Salt included in the signature

4.1.2 MAYO Principle

The public key is a quadratic map

$$\mathcal{P} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m.$$

The secret key defines an oil space $\mathcal{O}$ such that

$$\mathcal{P}(\mathbf{o}) = 0, \quad \forall \mathbf{o} \in \mathcal{O}.$$

For a quadratic map, define

$$\mathcal{P}'(\mathbf{x}, \mathbf{y}) = \mathcal{P}(\mathbf{x} + \mathbf{y}) - \mathcal{P}(\mathbf{x}) - \mathcal{P}(\mathbf{y}).$$

Therefore, for $\mathbf{o} \in \mathcal{O}$,

$$\mathcal{P}(\mathbf{v} + \mathbf{o}) = \mathcal{P}(\mathbf{v}) + \mathcal{P}'(\mathbf{v}, \mathbf{o}),$$

which is linear in $\mathbf{o}$ once $\mathbf{v}$ is fixed.

MAYO combines k vectors using the whipped map

$$\mathcal{P}^* : (\mathbb{F}_q^n)^k \rightarrow \mathbb{F}_q^m.$$

This provides ko oil variables and allows the signing problem to be reduced to a linear system

$$\mathbf{A}\mathbf{x} = \mathbf{y}.$$

4.1.3 Key Generation

Algorithm 4 MAYO Key Generation (Abstract Form)

Key Generation:

- 1: $\text{seed}_{sk} \xleftarrow{\$} \{0, 1\}^{\text{sk.seed.bits}}$
- 2: $(\text{seed}_{pk}, \mathbf{O}) \leftarrow \text{Expand}(\text{seed}_{sk})$
- 3: $\mathcal{P} \leftarrow \text{GeneratePublicMap}(\text{seed}_{pk}, \mathbf{O})$
- 4: $\text{sk} \leftarrow \text{seed}_{sk}$
- 5: $\text{pk} \leftarrow \mathcal{P}$

Output: (sk, pk)

4.1.4 Signing Algorithm

Algorithm 5 MAYO Sign (Abstract Form)

Input: sk, Msg

Signing:

- 1: $\mathbf{O} \leftarrow \text{Expand}(sk)$
- 2: $\text{salt} \leftarrow \text{GenerateSalt}(\text{Msg}, sk)$
- 3: $\mathbf{t} \leftarrow \text{Hash}(\text{Msg} \parallel \text{salt})$
- 4: **for** ctr **from** 0 **to** 255 **do**
- 5: $(\mathbf{v}_0, \dots, \mathbf{v}_{k-1}) \leftarrow \text{CSPRNG}(\text{Msg} \parallel \text{salt} \parallel sk \parallel ctr)$
- 6: $(\mathbf{A}, \mathbf{y}) \leftarrow \text{BuildLinearSystem}(\mathbf{v}_0, \dots, \mathbf{v}_{k-1}, \mathbf{t})$
- 7: $\mathbf{x} \leftarrow \text{SampleSolution}(\mathbf{A}, \mathbf{y})$
- 8: **if** $\mathbf{x} \neq \perp$ **then**
- 9: **break**
- 10: **for** i **from** 0 **to** $k-1$ **do**
- 11: $\mathbf{s}_i \leftarrow (\mathbf{v}_i + \mathbf{O}\mathbf{x}_i) \parallel \mathbf{x}_i$

Output: $\sigma = (\mathbf{s}_0, \dots, \mathbf{s}_{k-1}, \text{salt})$

4.1.5 Verification Algorithm

Algorithm 6 MAYO Verify (Abstract Form)

Input: pk, Msg, σ

Verifying:

1: Parse $(s_0, \dots, s_{k-1}, salt) \leftarrow \sigma$

2: $t \leftarrow \text{Hash}(Msg \parallel salt)$

3: $y \leftarrow \mathcal{P}^*(s_0, \dots, s_{k-1})$

Output: Verif = $(y == t)$

4.2 Current Implementations

Category	Implementation
General	Reference implementation in C
Optimized	• AVX2-optimized implementation in C • NEON-optimized implementation in C • Cortex-M4 implementation in C

Table 4.2: Available implementations of CROSS

4.3 Analysis of internal performance

4.3.1 Key Generation

Overall

In Figure 4.1, the AVX2 implementation of the key generation algorithm achieves an average time reduction of 96.44%. In Figure 4.2, the NEON implementation achieves an average time reduction of 95.08% for the key generation algorithm.

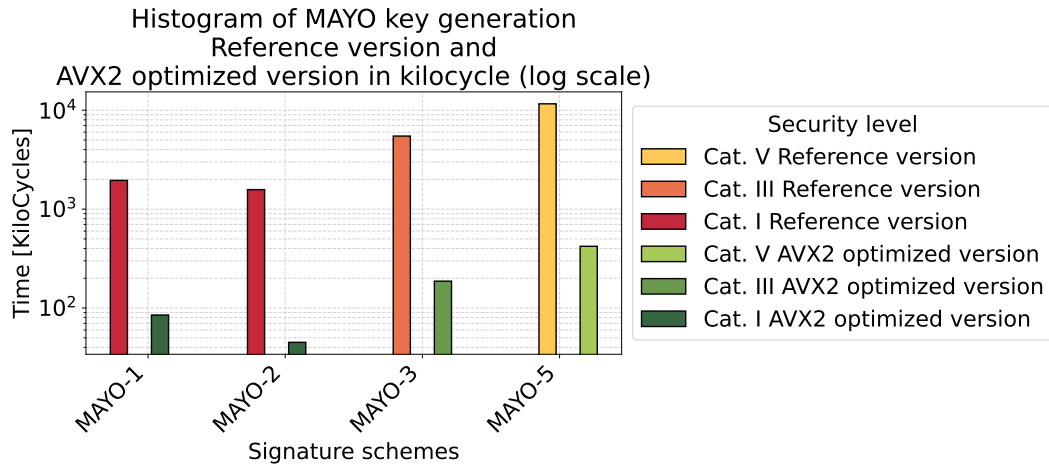


Figure 4.1: Comparison of MAYO Key generation, Reference and AVX2 versions (Table C.4)

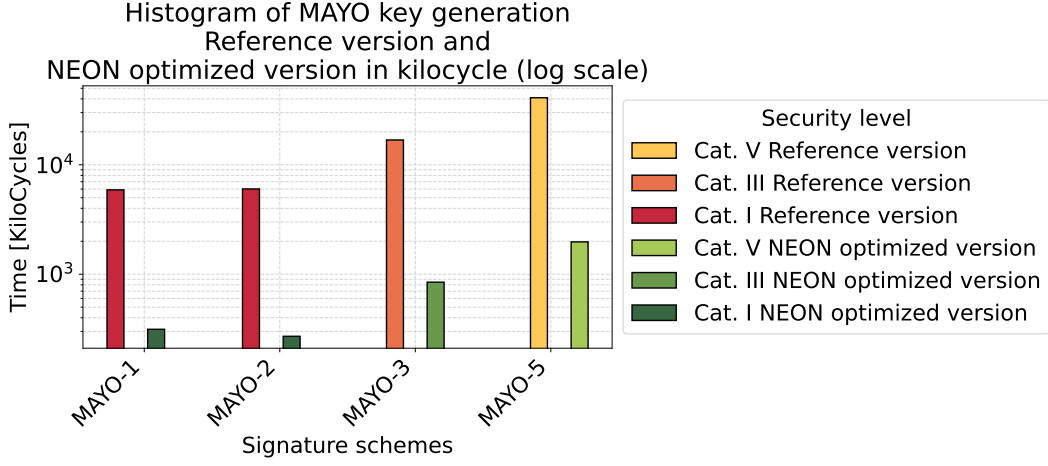


Figure 4.2: Comparison of MAYO Key generation, Reference and NEON versions (Table C.5)

Internal

The internal analysis is based on the pseudocode given by the reference paper[17]. I divided the code into these sections : "declaration", "row 2", "row 5", "row 6", "row 7", "row 9-12", "row 14-16", "row 18-20", "cleanup".

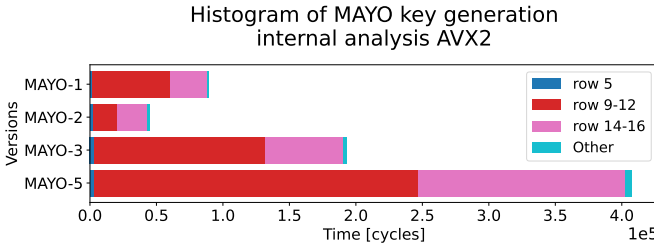


Figure 4.3: MAYO internal analysis of key generation for AVX2 implementation¹ (Table C.20)

In Figure 4.4, We can observe that the row 9-12 take less time in proportion than for the Figure 4.3. This opposite effect can be observed on row 14-16.

In Figure 4.3, we can observe that the rows taking the most time are row 9-12 and row 14-16. row 9-12 generate the first part of $\mathcal{P}$ from $seed_{pk}$. row 14-16 compute the last part of $\mathcal{P}$ from the first part.

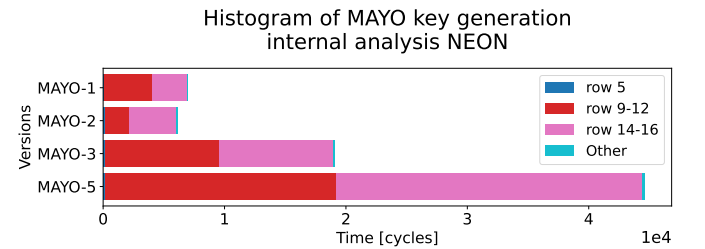
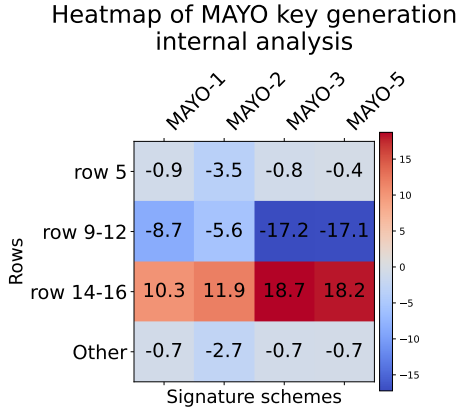


Figure 4.4: MAYO internal analysis of key generation for NEON implementation (Table C.21)

¹The sections taking less than 1% are classified as other.



In Figure 4.5, we can observe exactly what we just discussed.

Figure 4.5: Difference in proportion (NEON - AVX2) for MAYO key generation internal analysis (Tables C.20, C.21)

4.3.2 Signing Algorithm

Overall

In Figure 4.6, the AVX2 implementation of the signing algorithm achieves an average time reduction of 93.93%. In Figure 4.7, the NEON implementation achieves an average time reduction of 92.52% for the signing algorithm.

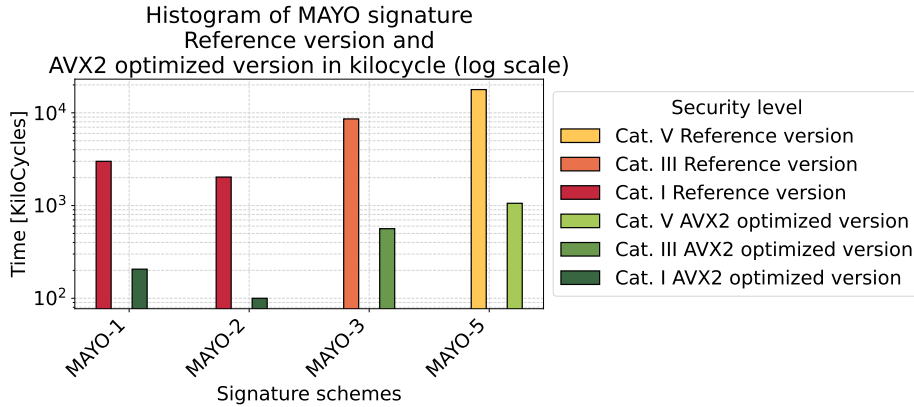


Figure 4.6: Comparison of MAYO signature algorithm, Reference and AVX2 versions (Table C.6)

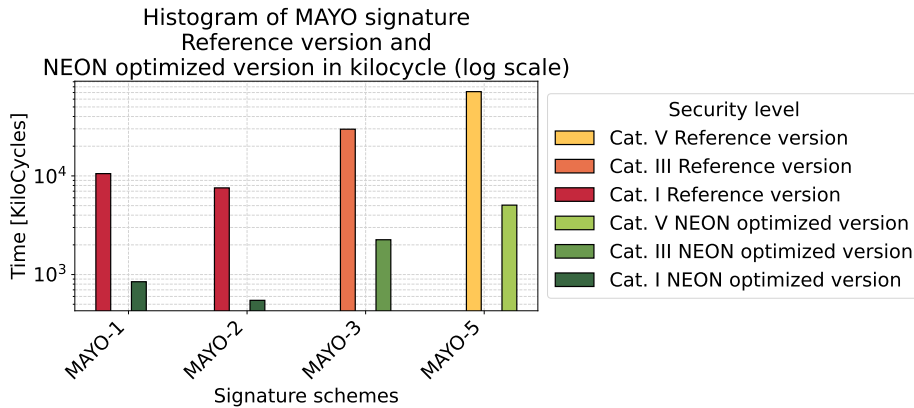


Figure 4.7: Comparison of MAYO signature algorithm, Reference and NEON versions (Table C.7)

Internal

The internal analysis is based on the pseudocode given by the reference paper[17]. I divided the code into these sections : "declaration", "expand sk", "row 8", "declaration 2", "row 9", "row 10", "row 11", "row 14 loop", "row 43-45", "row 46", "cleanup"

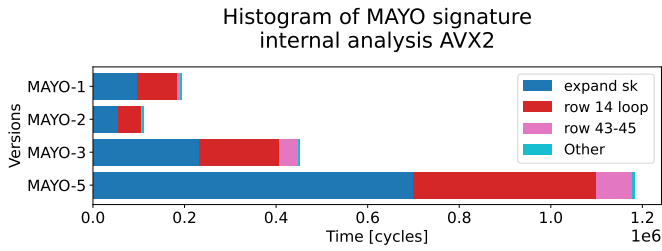


Figure 4.8: MAYO internal analysis of signing algorithm for AVX2 implementation (Table C.22)

In Figure 4.9, We can observe that the row 14 loop take more time in proportion than for the Figure 4.8.

In Figure 4.8, we can observe that the rows taking the most time are expand sk and row 43-45. Expand sk expand the compacted secret key. Row 43-45 compute the signature.

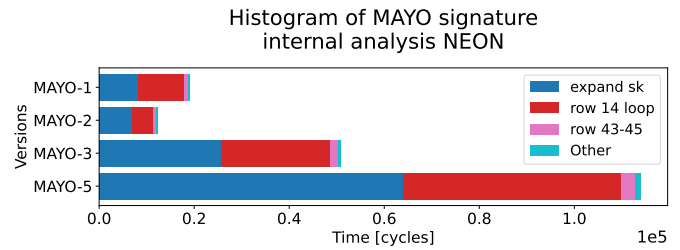


Figure 4.9: MAYO internal analysis of signing algorithm for NEON implementation (Table C.23)

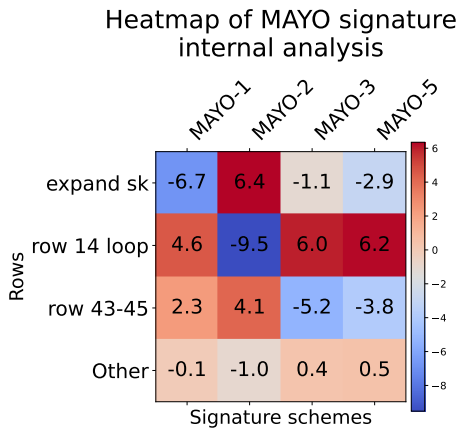


Figure 4.10: Difference in proportion (NEON - AVX2) for MAYO signing algorithm internal analysis (Tables C.22, C.23)

In Figure 4.10, we can observe that as we saw row 14 loop takes a bigger proportion except for MAYO-2. MAYO-3 and MAYO-5 look alike. But, MAYO-1 have different results especially for expand sk.

4.3.3 Verification Algorithm

Overall

In Figure 4.11, the AVX2 implementation of the verification algorithm achieves an average time reduction of 95.99%. In Figure 4.12, the NEON implementation achieves an average time reduction of 92.28% for the verification algorithm.

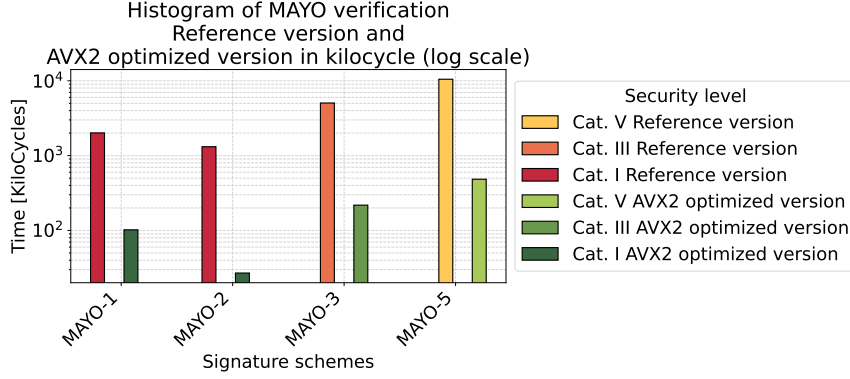


Figure 4.11: Comparison of MAYO verification algorithm, Reference and AVX2 versions (Table C.8)

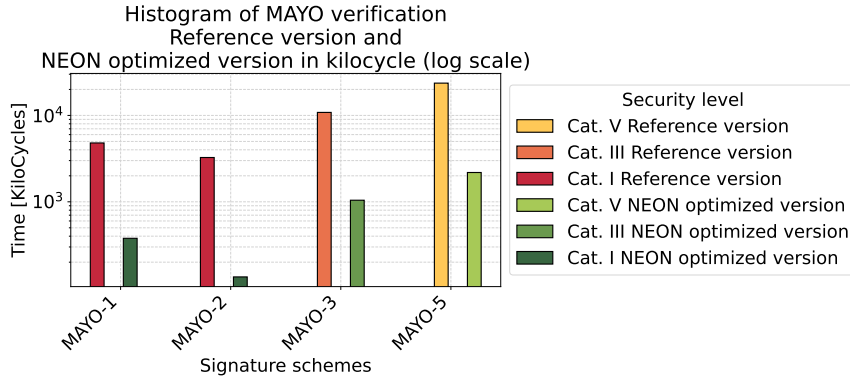


Figure 4.12: Comparison of MAYO verification algorithm, Reference and NEON versions (Table C.9)

Internal

The internal analysis is based on the pseudocode given by the reference paper[17]. I divided the code into these sections : "declarations and decode", "expand pk", "row 1-7", "row 16", "row 17", "row 11-13", "row 20-26", "row 29-31"

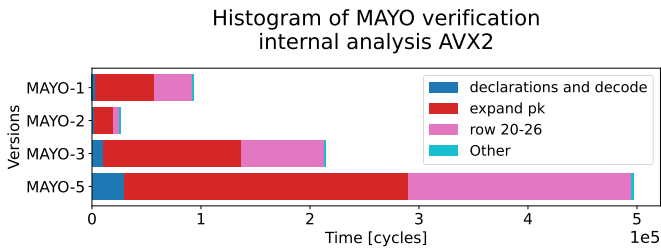


Figure 4.13: MAYO internal analysis of verification algorithm for AVX2 implementation (Table C.24)

In Figure 4.13, we can observe that the rows taking the most time are expand pk and row 20-26. Expand pk expand the compacted public key. Row 20-26 compute $y \leftarrow \mathcal{P}^*(s)$.

In Figure 4.14, We can observe that the expand pk take less time in proportion than for the Figure 4.13. This opposite effect can be observed on row 20-26.

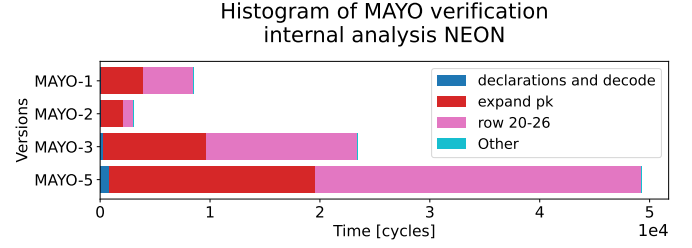


Figure 4.14: MAYO internal analysis of verification algorithm for NEON implementation (Table C.25)

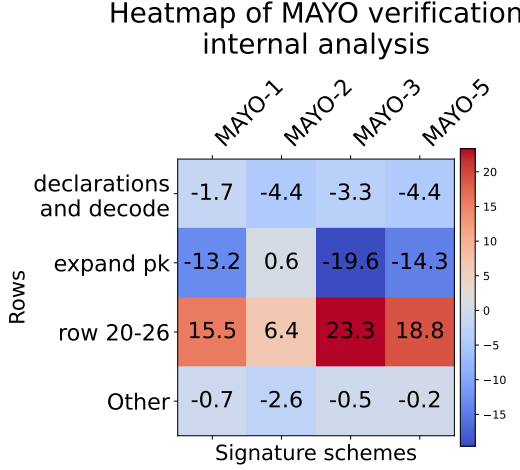


Figure 4.15: Difference in proportion (NEON - AVX2) for MAYO verification algorithm internal analysis (Tables C.24, C.25)

In Figure 4.15, We can observe that the observation made just above is observable here. But, we can see that declarations and decode, which declare the needed variables and decode the signature, take less proportions in the NEON implementation than in the AVX2 implementation.

4.4 Conclusion and Future work

4.4.1 Conclusion

	AVX2	NEON
Key generation	96.44%	95.08%
Signing algorithm	93.93%	92.52%
Verification algorithm	95.99%	92.28%

Table 4.3: MAYO: Performance comparison between AVX2 and NEON

To conclude, we can show on the Table 4.3, the results obtained for MAYO show that both optimized implementations significantly improve the performance compared to the reference implementation. A big part could come from SHA-3 vectorization and AES vectorization.

Here we can see that the ARM implementation have a smaller reduction time than the AVX2 implementation this is exactly what I predicted with my working hypothesis . But without other comparison numbers, we can't decide now of it means that my working hypothesis is correct or not. We will need the MQOM time reduction to verify.

The internal analysis also showed different proportions between both implementations. This observed differences could be from the different instruction sets provided by x86 and ARM-A.

4.4.2 Future work

For future work, we have different option:

- Implement an ARMv8.2 implementation: on the XKCP GitHub repository [14], the Keccak Team[15] released a SHA-3 implementation using the new vectorized instructions we saw in 2.3.4.
- Bench and analyze the loop and condition internal analysis from the key generation, signing and verification algorithm. And make the same analysis on other important internal function. (e.g. ExpandSK, ExpandPK, ...)
- Some part may be more time consuming. I think it could be a good idea to port those into assembly. It could improve the performances.

Chapter 5

MQOM

5.1 Description of the MQOM signature scheme

The last signature scheme is MQOM[19][20], which is based on the hardness of the Multivariate Quadratic Problem and Polynomial IOP as proof. In MQOM, the expansion relies on a GGM seed tree. A GGM seed tree consists of pseudorandomly expanding a root seed into N leaf seeds using a binary tree. For more information watch the presentation by Thibault Feneuil[21] who contributed to MQOM[22].

MQOM presents different versions. First, we need to distinguish between 3r and 5r, respectively 3-round and 5-round. The difference is that 5r has smaller signatures; however, 3r is simpler and easier to implement. For each version a fast and short (signature) was implemented with a changing parameter. To quote NIST from the 2024 report[12] "While the problem of solving multivariate quadratic systems has been well-studied, the recent result of [23] may have a small effect on the MQOM parameters needed to meet the target security levels." and "MQOM is expected to have highly competitive performance within the fast-moving area of MPC/VOLE/Threshold in the Head signatures."

For the future of MQOM, there is a new optimization coming for GGM tree that she was not added in the last round version. And To quote the NIST from the 2024 report [12] "MQOM offers very small public keys with signatures of intermediate size between those of ML-DSA and SLH-DSA. It is expected that significant improvements in the performance and signature size of MQOM will be obtained using new techniques (i.e., techniques from Threshold in the Head)[24] that will not change the underlying security assumptions."

Since the start of the Round 2 of the competition, the team updated the MQOM2 to MQOMv2.1. This new version introduces several improvements compared to MQOMv2, mainly concerning the finite field operations and the folding step. These changes improve the efficiency of the scheme without affecting its security. A new variant over $\mathbb{F}_{16}$. In this Master's thesis, I'll use the latest available version: MQOMv2.1.[25]

The next sections present how MQOMv2.1 works.

5.1.1 Parameters

Symbol	Meaning
$\mathbb{F}$	Base finite field
$\mathbb{K}$	Extension field of $\mathbb{F}$ used by the proof system
n	Number of unknowns of the MQ system
m	Number of quadratic equations of the MQ system
λ	Security parameter
μ	Extension degree, $\mu = [\mathbb{K} : \mathbb{F}]$
$\hat{n}$	Number of packed MQ equations, $\hat{n} = m/\mu$

N	Number of possible evaluation points
τ	Number of parallel executions of the proof system
η	Number of equations after batching
w	Grinding parameter
$\mathbf{x}$	Secret solution of the MQ system
$\hat{\mathbf{A}}_i$	Quadratic matrix of the i -th packed MQ equation
$\hat{\mathbf{b}}_i$	Linear vector of the i -th packed MQ equation
$\hat{y}_i$	Output of the i -th packed MQ equation
com_1	Commitment to the secret-sharing structure
com_2	Commitment to the polynomial proof values
i^*	Challenge indexes determining which parts of the commitments are opened
salt	Random salt included in the signature
nonce	Counter used for the grinding procedure

5.1.2 MQOM Principle

MQOM is based on the Multivariate Quadratic (MQ) problem. The public key defines a system of quadratic equations

$$\hat{y}_i = \mathbf{x}^T \hat{\mathbf{A}}_i \mathbf{x} + \hat{\mathbf{b}}_i^T \mathbf{x}, \quad i = 1, \dots, \hat{m},$$

while the secret key contains the solution $\mathbf{x}$. Equivalently, define

$$\hat{f}_i(\mathbf{x}) = \mathbf{x}^T \hat{\mathbf{A}}_i \mathbf{x} + \hat{\mathbf{b}}_i^T \mathbf{x} - \hat{y}_i.$$

A valid secret key therefore satisfies

$$\hat{F}(\mathbf{x}) = 0.$$

During signing, MQOM does not reveal $\mathbf{x}$. Instead, the signer uses the Threshold-Computation-in-the-Head (TCitH) framework, a variant of MPC-in-the-Head, to prove knowledge of a valid solution of the public MQ system without revealing it.

The proof is repeated τ times. In the interactive version, the verifier would challenge the signer to open specific parts of the commitments.

In the signature scheme, the Fiat–Shamir transform replaces these interactive challenges with challenges derived from the message and the commitments using a hash function, making the proof non-interactive.

The proof internally represents the secret using a degree-one polynomial

$$P_x(X) = \mathbf{x}_0 + \mathbf{x}X,$$

whose leading coefficient is the secret solution $\mathbf{x}$.

The MQ equations are evaluated on this polynomial to obtain

$$P_z = \hat{F}(P_x).$$

A random polynomial P_u is then added to hide this computation, and the proof uses

$$P_\alpha = P_u + \Gamma P_z,$$

where Γ is a public value derived from the commitment when batching is enabled.

5.1.3 Key Generation

Algorithm 7 MQOM Key Generation (Abstract Form)

Key Generation:

- 1: $\text{seed}_{key} \xleftarrow{\$} \{0, 1\}^{2\lambda}$
 - 2: $(\mathbf{x}, \text{mseed}_{eq}) \leftarrow \text{Expand}(\text{seed}_{key})$
 - 3: $(\{\hat{\mathbf{A}}_i\}, \{\hat{\mathbf{b}}_i\}) \leftarrow \text{ExpandEquations}(\text{mseed}_{eq})$
 - 4: **for** i **from** 1 **to** $\hat{n}$ **do**
 - 5: $\hat{y}_i \leftarrow \mathbf{x}^T \hat{\mathbf{A}}_i \mathbf{x} + \hat{\mathbf{b}}_i^T \mathbf{x}$
 - 6: $\text{pk} \leftarrow (\text{mseed}_{eq}, \hat{\mathbf{y}})$
 - 7: $\text{sk} \leftarrow (\text{pk}, \mathbf{x})$
- Output:** (sk, pk)
-

5.1.4 Signing Algorithm

Algorithm 8 MQOM Sign (Abstract Form)

Input: sk, Msg

Signing:

- 1: $(pk, \mathbf{x}) \leftarrow \text{Parse}(sk)$
 - 2: $\text{mseed} \xleftarrow{\$} \{0, 1\}^\lambda$
 - 3: $\text{salt} \xleftarrow{\$} \{0, 1\}^\lambda$
 - 4: $(\text{com}_1, \text{key}, P_x, P_u) \leftarrow \text{BLC.Commit}(\text{mseed}, \text{salt}, \mathbf{x})$
 - 5: $(\alpha_0, \alpha_1) \leftarrow \text{ComputePAlpha}(P_x, P_u, pk)$
 - 6: $\text{com}_2 \leftarrow \text{Hash}(\alpha_0, \alpha_1)$
 - 7: $(i^*, \text{nonce}) \leftarrow \text{SampleChallenge}(\text{Hash}(pk \parallel \text{com}_1 \parallel \text{com}_2 \parallel Msg))$
 - 8: $\text{opening} \leftarrow \text{BLC.Open}(\text{key}, i^*)$
- Output:** $\sigma = (\text{salt}, \text{com}_1, \text{com}_2, \alpha_1, \text{opening}, \text{nonce})$
-

5.1.5 Verification Algorithm

Algorithm 9 MQOM Verify (Abstract Form)

Input: pk, Msg, σ

Verifying:

- 1: $\text{Parse}(\text{salt}, \text{com}_1, \text{com}_2, \alpha_1, \text{opening}, \text{nonce}) \leftarrow \sigma$
 - 2: $(i^*, val) \leftarrow \text{ComputeChallenge}(pk, \text{com}_1, \text{com}_2, Msg, \text{nonce})$
 - 3: **if** $val \neq 0$ **then**
 - 4: **return** False
 - 5: $(\text{verif}, P_x(r), P_u(r)) \leftarrow \text{BLC.Eval}(\text{salt}, \text{com}_1, \text{opening}, i^*)$
 - 6: **if** $\text{verif} = \text{False}$ **then**
 - 7: **return** False
 - 8: $\alpha_0 \leftarrow \text{RecomputePAlpha}(P_x(r), P_u(r), \alpha_1, pk)$
 - 9: $\text{com}'_2 \leftarrow \text{Hash}(\alpha_0, \alpha_1)$
- Output:** $\text{Verif} = (\text{com}'_2 == \text{com}_2)$
-

5.2 Current Implementations

Category	Available Implementations
General	• Reference implementation in C • Reference implementation in Python
Optimized	• AVX2-optimized implementation • The NEON implementation developed during this Master's thesis • GFNI-optimized implementation using AVX-512 and GFNI (Galois Field New Instructions) • Cortex-M4 implementation

Table 5.2: Available implementations

5.3 Implementation of MQOMv2.1 NEON

To port MQOM to a Neon implementation, I started by copying the `Optimized_Implementation` directory and named it `NEON_Implementation` as shown in Figure 5.1. All directories contain all the implementations (reference, AVX2, avx512, ...) but I chose to work on each directory as if it were the implementation defined by the directory.

The inside of each Implementation directory is divided into different directories, each containing a different version. The reality is that only the `mqom2_parameters.h` is different. So all other files are repeated in each version directory. This is the reason why I created `sync_file.sh` to propagate the changes I made in files. Figure 5.2 shows a visual representation of what I just explained.

```
MQOMv2.1/
├── KAT/
├── + NEON_Implementation/a
├── Optimized_Implementation/
├── Reference_Implementation/
└── ...
```

Figure 5.1: Overall MQOM directory

^aGreen is for new file and Orange for modified file

```
MQOMv2.1/
├── NEON_Implementation/
│   ├── mqom2_cat1_gf2_fast_r3/
│   │   ├── mqom2_parameters.h
│   │   └── ...
│   ├── mqom2_cat1_gf2_fast_r5/
│   │   ├── mqom2_parameters.h
│   │   └── ...
│   └── ...
└── + sync_file.sh
```

Figure 5.2: Overall NEON implementation directory

The files I need to modify to port the version are divided into 4 sections: Makefile, SHA-3, Rijndael and Fields. I also added a section KAT to explain how I debugged and proved the correctness of my NEON version.

For the translation of the AVX2 intrinsics into NEON intrinsics, I used SSE2NEON [26]. This GitHub repository provides a translation layer from SSE/SSE2 intrinsics to NEON intrinsics. However, it can also facilitate the porting of AVX2 code, as AVX2 is an extension of the SSE SIMD instruction set. Since SSE2NEON already provided an efficient solution for most of the required operations, I initially chose this approach without investigating alternative AVX2-specific translation tools. And verified the correctness of the SSE2NEON translation with the official Intel intrinsic documentation[27] and ARM Intrinsic documentation[28].

However, SSE2NEON could not be directly applied to the entire codebase, as some parts, such as the fields files, rely on 256-bit vectors that are not supported by NEON. Furthermore, using a ready-to-use AVX2-to-NEON translation solution for the whole codebase would still have introduced performance overhead, as some parts of the code heavily rely on 256-bit AVX2 vectors, which cannot be directly mapped to NEON's 128-bit vectors.

5.3.1 Makefile



Figure 5.3: MQOM Makefile NEON implementation

The "/" represented in Figure 5.3 represents the root of a version (e.g. mqom2_cat1_gf2_fast_r3/ or mqom2_cat1_gf2_fast_r5/ as seen in Figure 5.2).

I started by modifying the Makefile in Figure 5.3 to add a way to support an ARM NEON optimized implementation within the MQOM implementations.

The changes in this file are:

- **line 42:** Addition of the `rijndael_neon.c` source file to the list of compiled Rijndael implementations.
- **line 111 to 114:** Introduction of the `RIJNDAEL_NEON` compilation flag to enable the NEON-optimized Rijndael implementation.
- **line 129 to 132:** Introduction of the `FIELDS_NEON` flag to select NEON-accelerated fields computation.
- **line 245 to 251:** Addition of the `FORCE_PLATFORM_NEON` option, forcing the build of the NEON implementation.
- **line 309 to 313:** Extension of the automatic platform detector to recognize ARM architectures by detecting the predefined macros: `__ARM_NEON__`, `__ARM_NEON` or `__aarch64__`.

5.3.2 SHA-3

To add SHA-3 NEON optimization, we need two things. First, I need to adapt the **Makefile** to accept a NEON SHA-3 optimization. Second, we need to add SHA-3 NEON files with the same API as AVX2 to fit.

SHA-3 Makefile

The changes in this file are:

- **line 62 to 67:** Addition of the source files from the `/sha3/neon/` directory to compile the SHA-3 NEON implementation.
- **line 92:** Modification of the error message.

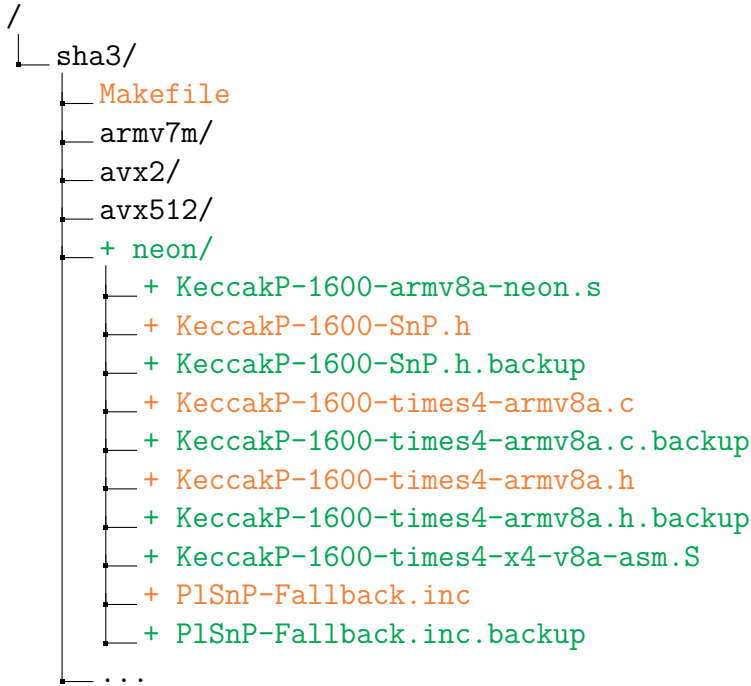


Figure 5.4: MQOM SHA-3 NEON implementation

SHA-3 NEON

1) Import SHA-3 NEON

For SHA-3 NEON, I chose the official implementation from the XKCP GitHub repository [14] from the Keccak Team[15]. This repository has an ARMv8A version with and without SHA-3 acceleration. For this version, I removed the SHA-3 acceleration part to get a pure ARMv8A version, since SHA-3 acceleration appears in ARMv8.2A. This way, we stay fair by using the same acceleration as the CROSS and MAYO ARM versions. With my version, modifying to add SHA-3 NEON acceleration is a small task.

The added files are:

- KeccakP-1600-SnP.h
- KeccakP-1600-armv8a-neon.S
- KeccakP-1600-times4-SnP.h
- KeccakP-1600-times4-armv8a.c
- KeccakP-1600-times4-x4-v8a-asm.S
- PlSnP-Fallback.inc

As seen in the figure 5.4. The two first files are used for XOF without optimization and all the files are used for XOFx4.

There is a difference between the .s files and the .S files. The first is directly compiled as is, and the second is preprocessed by the C preprocessor before being assembled. For this reason, I needed to change the original .s into .S.

Based on my investigation, the current AVX2 implementation comes from the rbc-lib GitLab [29] by the Rank-Based Cryptography Library[30]. This leads us to the second section, where I explain how I adapted the API to have the same API as the AVX2 version.

2) Adapt API

The original files are renamed .backup, and a copy is used to adapt the files.

2.1) void not KeccakP1600_align512plain64_state

The only difference between the AVX2 API and the original NEON API is the type of the state argument. In the AVX2 implementation, the type of the argument is void. And in the NEON implementation, the type of the argument is KeccakP1600_align512plain64_state. For KeccakP-1600-SnP.h, simply replacing by void poses no problem. For KeccakP-1600-times4-SnP.h, we need to modify some of the underlying files :

- **KeccakP-1600-times4-armv8a.c** : the line `KeccakP1600times4_states *s = states;` is added at the start of the function `KeccakP1600times4_PermuteAll_24rounds` and is renamed `s` in the function.
- **PlSnP-Fallback.inc** : the line `PlSnP_states *s = (PlSnP_states *)states;` is added at the start of each function and state is renamed `s` in each function.

2.2) Struct Size

200	200	200	200	
256	256	256	256	
200	56	200	56	200
				56

Figure 5.5: SHA-3 x4 structure [bits]

The second problem was that the structure used to store the state was not adapted. The requested structure was 4x200, but the file was doing 4x256 with 56 bits of padding. When merging the AVX2 and original NEON implementation, I observed that AVX2 defined its structure in the file `KeccakSponge.h` with the arguments `KeccakP1600_stateSizeInBytes` and `KeccakP1600_stateAlignment` given in the `KeccakP-1600-SnP.h` file and the arguments `KeccakP1600times4_statesSizeInBytes` and `KeccakP1600times4_statesAlignment` given in the file `KeccakP-1600-times4-SnP.h`. So I copied them into the existing structure of the original NEON implementation. Of course I verified that it wouldn't pose a problem in the file, since only the first 200 bits of `state[i]` are loaded and stored, as shown in Figure 5.6.

Assembly part

```

1  .macro LoadState
2      ld4      { v19.2d, v20.2d, v21.2d, v22.2d }, [x0], #64
3      ld4      { v23.2d, v24.2d, v25.2d, v26.2d }, [x0], #64
4      ld4      { v27.2d, v28.2d, v29.2d, v30.2d }, [x0], #64
5      ld1      { v31.d }[0], [x0], #8
6      sub      x0, x0, #200
7      movi     v16.2d, #0
8  .endm
9
10 .macro StoreState
11     st4      { v19.2d, v20.2d, v21.2d, v22.2d }, [x0], #64
12     st4      { v23.2d, v24.2d, v25.2d, v26.2d }, [x0], #64
13     st4      { v27.2d, v28.2d, v29.2d, v30.2d }, [x0], #64
14     st1      { v31.d }[0], [x0], #8

```

Figure 5.6: Assembly from KeccakP-1600-armv8a-neon.S

5.3.3 Rijndael

```

/
└─ rijndael
    └─ rijndael.h
    └─ rijndael_platform.h
    └─ + rijndael_neon.h
    └─ + rijndael_neon.c
    └─ rijndael_aes_ni.h
    └─ rijndael_aes_ni.c
    └─ rijndael_common.h
    └─ ...

```

For this section, I modified the file `rijndael.h` and `rijndael_platform.h`. I also create dthe file `rijndael_neon.h` and `rijndael_neon.c` which are translations of the `rijndael_aes_ni.h` and `rijndael_aes_ni.c`. Note that these files are not literal translations because, as shown in Figure 2.2, AES-NI and ARM NEON AES instructions differ. As we will see, this difference had a direct impact on Rijndael-256-256.

`rijndael.h`

This file converts the specific API into the general API. The changes in this file are:

- **line 6 to 30** : Modification of the error message and addition of the NEON error message.
- **line 128 to 148** : Addition of the part that converts the NEON API to the general API.

`rijndael_platform.h`

This file detects the current platform. The changes in this file are:

- **line 13 to 24** : Addition of the NEON platform detector.

`rijndael_neon.h` and `.c`

The files `rijndael_neon.h` and `rijndael_neon.c` are the core of the Rijndael part of the port. They contain the implementation behind the API. As mentioned previously, these files are adaptations of the AVX2 implementation. Therefore, I will compare the two implementations to highlight their main differences.

1) `rijndael_neon.h`

This file serves four purposes: verifying that the platform supports the Arm Cryptographic Extension (AES), importing the NEON headers, defining the required data structures, and declaring the required functions. The main changes are shown in Figure 5.7.

AVX2 version

```
#ifndef __RIJNDAEL_AES_NI_H__
#define __RIJNDAEL_AES_NI_H__

/* First of all check for the support of
AES-NI */
#ifdef __AES__
#error "Error: asking for RIJNDAEL_AES_NI while AES-NI support is not detected... Please check your platform and/or compilation options!"
#endif /* __AES__ */

#include "rijndael_common.h"

/* Header for AES-NI intrinsics */
#include <wmmintrin.h>
#include <emmintrin.h>
#include <smmintrin.h>

/* The general Rijndael core context
structure */
typedef struct
{
    rijndael_type rtype; /* Type of
Rijndael */
    /* AES-NI context using aligned
__m128i */
    /* NOTE: __m128i type should align
on 16 bytes and avoid
unalignment issues with xmm
memory accesses */
    __m128i rk[30]; /*Round keys*/
} rijndael_aes_ni_ctx;

...
```

NEON version

```
#ifndef __RIJNDAEL_NEON_H__
#define __RIJNDAEL_NEON_H__

/* First of all check for the support of
AES */
#ifdef __ARM_FEATURE_CRYPT0
#error "Error: asking for RIJNDAEL_NEON while ARM_FEATURE_CRYPT0 support is not detected... Please check your platform and/or compilation options!"
#endif /* __ARM_FEATURE_CRYPT0 */

#include "rijndael_common.h"

/* Header for AES ARM intrinsics */
#include <arm_neon.h>
#include <arm_acle.h>

/* The general Rijndael core context
structure */
typedef struct
{
    rijndael_type rtype; /* Type of
Rijndael */
    /* AES ARM context using aligned
uint8x16_t */
    /* NOTE: uint8x16_t type should
align on 16 bytes and avoid
unalignment issues with xmm
memory accesses */
    uint8x16_t rk[30]; /*Round keys*/
} rijndael_aes_neon_ctx;

...
```

Figure 5.7: rijndael_aes_ni.h and rijndael_neon.h comparison

One of the main differences to highlight between AVX2 and NEON is the vector definition. For AVX2, you only need to define the length of your vector (e.g. `__m128i` for 128-bits or `__m256i` for 256-bits)¹. But for NEON, on the side, it requires the length, the type and its partitioning (e.g. `uint8x16_t`, `uint16x8_t`, `uint32x4_t` or `uint64x2_t`).

2) rijndael_neon.c

This file implements the functions for AES-128, AES-256, and Rijndael-256-256, as well as x2, x4 and x8 interleaved versions of these algorithms.

2.1) AES-128 and AES-256

AES-128 and AES-256 are very similar so I'll explain them together. AES-128 functions are `aes128_aes_neon_setkey_enc` and `aes128_aes_neon_enc` with x2, x4 and x8 versions of the function. AES-256 functions are `aes256_aes_neon_setkey_enc` and `aes256_aes_neon_enc` with x2, x4 and x8 versions of the function.

2.1.1) Key Scheduling

¹AVX2 supports up to 256-bit vectors, whereas NEON is limited to 128-bit vectors, but this comparison is intended to highlight the different approaches.

aes128_aes_ni_setkey_enc and aes256_aes_ni_setkey_enc functions are similar, so I'll only present the first function here.

In figure 5.8, we can observe the function aes128_aes_ni_setkey_enc, the ... represents a repeating pattern. In the function, we can see that the only change is for the intrinsic _mm_loadu_si128 to be replaced by vld1q_u8. This is a substitution from an intrinsic 128-bit to an equivalent 128-bit. This process will continue for lots of intrinsic since the whole file was written in vectors of 128-bit.

AVX2 version	NEON version
<pre> WEAK int aes128_aes_ni_setkey_enc(rijndael_aes_ni_ctx *ctx, const uint8_t enc_key[16]) { __m128i *Key_Schedule = (__m128i*)ctx->rk ; /**/ ctx->rtype = AES128; /**/ Key_Schedule[0] = _mm_loadu_si128((const __m128i*) enc_key); Key_Schedule[1] = AES_128_key_exp(Key_Schedule[0], 0x01); ... Key_Schedule[10] = AES_128_key_exp(Key_Schedule[9], 0x36); return 0; } </pre>	<pre> WEAK int aes128_aes_ni_setkey_enc(rijndael_aes_ni_ctx *ctx, const uint8_t enc_key[16]) { __m128i *Key_Schedule = (__m128i*)ctx->rk ; /**/ ctx->rtype = AES128; /**/ Key_Schedule[0] = vld1q_u8((const __m128i*) enc_key); Key_Schedule[1] = AES_128_key_exp(Key_Schedule[0], 0x01); ... Key_Schedule[10] = AES_128_key_exp(Key_Schedule[9], 0x36); return 0; } </pre>

Figure 5.8: rijndael_aes_ni.h and rijndael_neon.h comparison, AES Key scheduling

We can see the definition of the macro AES_128_key_exp, called by aes128_aes_ni_setkey_enc, in Figure 5.9. This macro calls two functions aes_128_key_expansion. It's a pure transcription for most of it. What's interesting is the transcription of _mm_shuffle_epi32. This function doesn't have a pure equivalent. I used SSE2NEON for this transcription. I also verified the intrinsics one by one to make sure it was correct.

AVX2 version	NEON version
<pre> #define AES_128_key_exp(k, rcon) aes_128_key_expansion(k, _mm_aeskeygenassist_si128(k, rcon)) static inline __m128i aes_128_key_expansion(__m128i key, __m128i keygened){ keygened = _mm_shuffle_epi32(keygened, _MM_SHUFFLE(3,3,3,3)); key = _mm_xor_si128(key, _mm_slli_si128(key, 4)); key = _mm_xor_si128(key, _mm_slli_si128(key, 4)); key = _mm_xor_si128(key, _mm_slli_si128(key, 4)); return _mm_xor_si128(key, keygened); } </pre>	<pre> #define AES_128_key_exp(k, rcon) aes_128_key_expansion(k, aeskeygenassist_neon(k, rcon)) static inline uint8x16_t aes_128_key_expansion(uint8x16_t key, uint8x16_t keygened){ uint32x4_t tmp = vreinterpretq_u32_u8(keygened); // change view tmp = vdupq_laneq_u32(tmp, 3); // broadcast the 4th word keygened = vreinterpretq_u8_u32(tmp); // change view key = veorq_u8(key, vextq_u8(vdupq_n_u8 (0), key, 12)); // (16-4) key = veorq_u8(key, vextq_u8(vdupq_n_u8 (0), key, 12)); key = veorq_u8(key, vextq_u8(vdupq_n_u8 (0), key, 12)); return veorq_u8(key, keygened); } </pre>

Figure 5.9: rijndael_aes_ni.h and rijndael_neon.h comparison, AES Key scheduling part 2

In Figure 5.9, if we look closely we see that the intrinsic `_mm_aeskeygenassist_si128` is transcribed into the function `aeskeygenassist_neon`. As we saw in the section 2.2.2, this is normal since it doesn't have an equivalent. The function is composed of the 3 steps done by the intrinsic on AVX2 : subbyte, ShiftRows, Rcon xor and finally `vtrn2q_u32` which keeps the bytes at odd positions of the two given vectors.

NEON version

```
#define SSE2NEON_AES_SBOX(w) \
{ \
...
}
#define SSE2NEON_AES_RSBOX(w) \
{ \
...
}

#define SSE2NEON_AES_H0(x) (x)
const uint8_t _sse2neon_sbox[256] = SSE2NEON_AES_SBOX(SSE2NEON_AES_H0);
const uint8_t _sse2neon_rsbox[256] = SSE2NEON_AES_RSBOX(SSE2NEON_AES_H0);
#undef SSE2NEON_AES_H0

static inline uint8x16_t _aes_subbytes(uint8x16_t x)
{
    uint8x16_t v = vqtbl4q_u8(vld1q_u8_x4(_sse2neon_sbox), x);
    v = vqtbx4q_u8(v, vld1q_u8_x4(_sse2neon_sbox + 0x40),
        vsubq_u8(x, vdupq_n_u8(0x40)));
    v = vqtbx4q_u8(v, vld1q_u8_x4(_sse2neon_sbox + 0x80),
        vsubq_u8(x, vdupq_n_u8(0x80)));
    v = vqtbx4q_u8(v, vld1q_u8_x4(_sse2neon_sbox + 0xc0),
        vsubq_u8(x, vdupq_n_u8(0xc0)));
    return v;
}

static inline uint8x16_t aeskeygenassist_neon(uint8x16_t a, const int rcon)
{
    // AES S-box
    uint8x16_t sub = _aes_subbytes(a);

    // view as u32x4
    uint32x4_t sub_u32 = vreinterpretq_u32_u8(sub);

    // rotate bytes inside each 32-bit lane
    uint32x4_t rot = vorrq_u32(vshrq_n_u32(sub_u32, 8), vshlq_n_u32(sub_u32, 24));

    // lane-wise xor
    uint32x4_t rcon_vec = vdupq_n_u32((uint32_t)rcon);

    // xor rcon
    uint32x4_t rot_xor = veorq_u32(rot, rcon_vec);

    // return as bytes
    return vreinterpretq_u8_u32(vtrn2q_u32(sub_u32, rot_xor));
}
```

Figure 5.10: rijndael_neon.h, AES Key scheduling part 3

2.1.2) Rounds

The philosophy of the entire file, and particularly here, is to use macros each time we have repeating code. This choice is guided by the fact that a macro is always inlined, where an inline function might not be inlined if the compiler chooses differently. In figure 5.11, we can see the rounds of AES-128. I didn't give the whole function call only this part was transcribed. I created a macro to transcribe the instruction since an AES ARM round is divided into two rounds. I also adapted the code to respect the order since as I explained on Section 2.3.3 the rounds between AVX2 and NEON are shift.

AVX2 version	NEON version
<pre> #define DO_ENC_BLOCK_128_128(m,k) \ do{\ // AddRoundKey m = _mm_xor_si128(m, k[0]); \ // ShiftRows -> SubBytes -> MixColumns -> AddRoundKey m = _mm_aesenc_si128(m, k[1]); \ ... m = _mm_aesenc_si128(m, k[9]); \ // SubBytes -> ShiftRows -> AddRoundKey m = _mm_aesenc_si128(m, k[10]);\ }while(0) </pre>	<pre> #define AESE_AESMC(c, rk) \ { \ // AddRoundKey -> SubBytes -> ShiftRows c = vaeseq_u8(c, rk); \ // MixColumns c = vaesmcq_u8(c); \ }; #define DO_ENC_BLOCK_128_128(m,k) \ do{\ // AddRoundKey -> SubBytes -> ShiftRows -> MixColumns AESE_AESMC(m, k[0]); \ ... AESE_AESMC(m, k[8]); \ // AddRoundKey -> SubBytes -> ShiftRows m = vaeseq_u8(m, k[9]); \ // AddRoundKey m = _mm_aesenc_si128(m, k[10]);\ }while(0) </pre>

Figure 5.11: rijndael_aes_ni.h and rijndael_neon.h comparison, AES Round

2.1) Rijndael-256-256

2.2.1) Key Scheduling

The key scheduling part of Rijndael-256-256 transcription is similar to the key generation. The function called by it got through a trivial transcription. One intrinsic worth mentioning is `_mm_slli_si128(a, 0x4)` which transcript to `vextq_u8( vdupq_n_u8(0), a, 12 )`. The original intrinsic shifts the contents of the 128-bit vector left by 4 bytes, filling the empty spaces with zeros. However, the NEON transcription performs this operation by concatenating a zero vector with the vector `a` and extracting a 16-byte window starting at offset 12 using `vextq_u8`, producing the same result in the end.

AVX2 version	NEON version
<pre> WEAK int rijndael256_aes_ni_setkey_enc(rijndael_aes_ni_ctx *ctx, const uint8_t enc_key[32]) { __m128i temp1, temp2, temp3; __m128i *Key_Schedule = (__m128i*)ctx->rk ; /**/ ctx->rtype = RIJNDAEL_256_256; /**/ temp1 = _mm_loadu_si128((__m128i*) enc_key); temp3 = _mm_loadu_si128((__m128i*) enc_key+16)); Key_Schedule[0] = temp1; Key_Schedule[1] = temp3; temp2 = _mm_aeskeygenassist_si128 (temp3 ,0x01); KEY_256_ASSIST_1(&temp1, &temp2); Key_Schedule[2]=temp1; KEY_256_ASSIST_2(&temp1, &temp3); Key_Schedule[3]=temp3; </pre>	<pre> WEAK int rijndael256_aes_neon_setkey_enc(rijndael_aes_neon_ctx *ctx, const uint8_t enc_key[32]) { uint8x16_t temp1, temp2, temp3; uint8x16_t *Key_Schedule = (uint8x16_t*) ctx->rk; /**/ ctx->rtype = RIJNDAEL_256_256; /**/ temp1 = vld1q_u8((uint8_t*)enc_key); temp3 = vld1q_u8((uint8_t*)(enc_key+16)); Key_Schedule[0] = temp1; Key_Schedule[1] = temp3; temp2 = aeskeygenassist_neon(temp3,0x01); KEY_256_ASSIST_1(&temp1, &temp2); Key_Schedule[2]=temp1; KEY_256_ASSIST_2(&temp1, &temp3); Key_Schedule[3]=temp3; ... temp2 = aeskeygenassist_neon(temp3,0x4d); </pre>

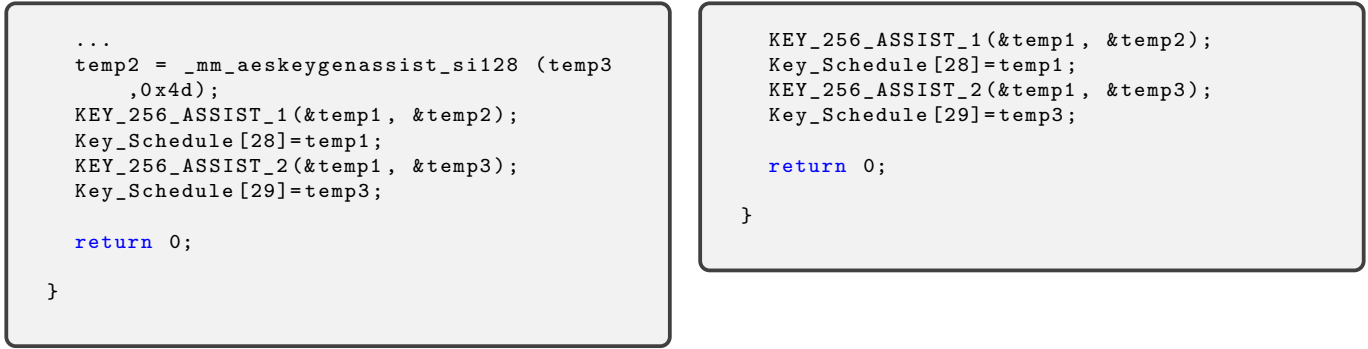


Figure 5.12: rijndael_aes_ni.h and rijndael_neon.h comparison, Rijndael Key scheduling

2.1.2) Rounds

Rijndael-256-256 has a width of 256-bit which is too much for AES NI or AES ARM. This is why, to compensate, we need to have a new step as shown in Figure 5.13. This step is in fact two step **Shuffle** and **Blend**. For AES NI, This step is trivial. AVX2 has the perfect instructions to do the job. We can see it in the code from the Figure 5.14. Those instructions take three arguments : the vector **ml**, the verctor **mr** and a vector mask, **mshuffle** for shuffle and **mblend** for blend.

While transcribing the round to NEON, I encountered a problem. The compensate block can't stay at the same place since it is placed in the middle of an intrinsic. I can't move it up because it would require to change the order of the key so I needed to move it down. I moved it just after the subbyte and shiftrow. This place is good since blend and shuffle just move bytes and subbyte just change the values of the bytes and shiftrow change the place of bytes. We don't have a problem of external value changing specific byte as we have with AddRoundkey. But, this change need an adaptation of **mblend** and **mshuffle** to keep correctness.

To find the values of **mblend** and **mshuffle**, I first looked after the documentation of the intrinsic **vaeseq_u8** (AESE). I first did the calculation on paper to find **mblend'** and **mshuffle'**. Then, I created a small test of one round to test my solution. For this Test, I set the key to a vector zero to remove the AddRoundkey and removed the MixColumn with the inverse Mixcolumn intrinsic. This way I only cared about subbyte and shiftrow. In the end, I obtained :

$$\text{mblend}' = \text{ShiftRow}(\text{mblend})$$

$$\text{mshuffle}' = \text{mshuffle}.$$

(5.1)

(5.2)

where ShiftRow is the byte permutation from the intrinsic **vaeseq_u8** applied on the vector NEON.

	x86 AES-NI	ARM AES	
Compensate	Blend Shuffle	AddRoundKey	AESE
AESENC	SubBytes	SubBytes	
	ShiftRows	ShiftRows	
	MixColumns	Blend Shuffle	
	AddRoundKey	MixColumns	Compensate AESMC

Figure 5.13: Rijndael-256-256 single round encryption

In Figure 5.14, We can see the actual code with `mblend` and `mshuffle`. For the rest of the code, as for AES-128, I didn't put the whole call but just the interesting part. As we can see, excluding the compensation par, the code transcription is similar to AES-128. For the compensate part, the intrinsics `_mm_blendv_epi8` and `_mm_shuffle_epi8` didn't have an equivalent so I used SSE2NEON to adapt the NEON part. We can see it in Figure 5.15.

AVX2 version

```

/* 256 bits blocks: ml is left block, mr is
   right block */
#define RINJDAEL_COMPENSATE_SR(ml, mr, tmp1,
    tmp2, mblend, mshuff) \
do{ \
    /* This compensates for the AES
       shiftrows to */ \
    /* conform to Rijdael one */ \
    tmp1 = _mm_blendv_epi8(ml, mr, mblend); \
    tmp2 = _mm_blendv_epi8(mr, ml, mblend); \
    tmp1 = _mm_shuffle_epi8(tmp1, mshuff); \
    tmp2 = _mm_shuffle_epi8(tmp2, mshuff); \
}while(0)

#define ENC_256_BLOCK(ml, mr, kl, kr, mb,
    ms) \
do{ \
    __m128i tmp1, tmp2; \
    /* Compensate for the shift rows */ \
    RINJDAEL_COMPENSATE_SR(ml, mr, tmp1,
        tmp2, mb, ms); \
    ml = _mm_aesenc_si128(tmp1, kl); \
    mr = _mm_aesenc_si128(tmp2, kr); \
}while(0)

#define ENC_256_BLOCK_LAST(ml, mr, kl, kr,
    mblend, mshuff) \
do{ \
    __m128i tmp1, tmp2; \
    /* Compensate for the shift rows */ \
    RINJDAEL_COMPENSATE_SR(ml, mr, tmp1,
        tmp2, mblend, mshuff); \
    ml = _mm_aesenc_si128(tmp1, kl); \
    mr = _mm_aesenc_si128(tmp2, kr); \
}while(0)

#define DO_ENC_BLOCK_256_256(ml, mr, k) \
do{ \
    /* ShiftRows compensation constants */ \
    \
    const __m128i mblend = _mm_set_epi32
    (0x80000000, 0x80800000, 0x80800000
    , 0x80808000); \
    \
    const __m128i mshuff = _mm_set_epi32
    (0x03020d0c, 0x0f0e0908, 0x0b0a0504
    , 0x07060100); \
    \
    /**/ \
    ml = _mm_xor_si128(ml, k[ 0]); \
    mr = _mm_xor_si128(mr, k[ 1]); \
    /**/ \

```

NEON version

```

/* 256 bits blocks: ml is left block, mr is
   right block */
#define RINJDAEL_COMPENSATE_SR(ml, mr, tmp1,
    tmp2, mblend, mshuff) \
do{ \
    /* This compensates for the AES
       shiftrows to */ \
    /* conform to Rijdael one */ \
    tmp1 = blendv_neon(ml, mr, mblend); \
    tmp2 = blendv_neon(mr, ml, mblend); \
    tmp1 = shuffle_neon(tmp1, mshuff); \
    tmp2 = shuffle_neon(tmp2, mshuff); \
}while(0)

#define ENC_256_BLOCK(ml, mr, kl, kr, mb,
    ms) \
do{ \
    ml = vaeseq_u8(ml, kl); \
    mr = vaeseq_u8(mr, kr); \
    \
    uint8x16_t tmp1, tmp2; \
    /* Compensate for the shift rows */ \
    RINJDAEL_COMPENSATE_SR(ml, mr, tmp1,
        tmp2, mb, ms); \
    \
    ml = vaesmcq_u8(tmp1); \
    mr = vaesmcq_u8(tmp2); \
}while(0)

#define ENC_256_BLOCK_LAST(ml, mr, kl, kr,
    mblend, mshuff) \
do{ \
    ml = vaeseq_u8(ml, kl); \
    mr = vaeseq_u8(mr, kr); \
    uint8x16_t tmp1, tmp2; \
    /* Compensate for the shift rows */ \
    RINJDAEL_COMPENSATE_SR(ml, mr, tmp1,
        tmp2, mblend, mshuff); \
    ml = tmp1; \
    mr = tmp2; \
}while(0)

#define DO_ENC_BLOCK_256_256(ml, mr, k) \
do{ \
    /* ShiftRows compensation constants */ \
    \
    const uint8x16_t mblend = { 0x00,0x00
    ,0x80,0x80, \
    0x00,0x00,0x00,0x80, \
    0x00,0x00,0x80,0x80, \
    0x00,0x80,0x80,0x80 }; \
    \
    const uint8x16_t mshuff = { 0x00,0x01
    ,0x06,0x07, \
    0x04,0x05,0x0a,0x0b, \
    0x08,0x09,0x0e,0x0f, \
    0x0c,0x0d,0x02,0x03 }; \
    /**/ \
    ENC_256_BLOCK(ml, mr, k[0], k[1] ,
        mblend, mshuff); \
    ...

```

```

ENC_256_BLOCK(ml, mr, k[2], k[3] ,
    mblend, mshuff); \
...
ENC_256_BLOCK(ml, mr, k[26], k[27],
    mblend, mshuff); \
/**/ \
ENC_256_BLOCK_LAST(ml, mr, k[28], k
    [29], mblend, mshuff); \
}while(0)

```

```

ENC_256_BLOCK(ml, mr, k[24], k[25],
    mblend, mshuff); \
/**/ \
ENC_256_BLOCK_LAST(ml, mr, k[26], k
    [27], mblend, mshuff); \
/**/ \
ml = veorq_u8(ml, k[ 28]); \
mr = veorq_u8(mr, k[ 29]); \
/**/ \
}while(0)

```

Figure 5.14: rijndael_aes_ni.h and rijndael_neon.h comparison, Rijndael Round

NEON version

```

static inline uint8x16_t blendv_neon(uint8x16_t a, uint8x16_t b, uint8x16_t mask)
{
    // Use a signed shift right to create a mask with the sign bit
    uint8x16_t new_mask = vreinterpretq_u8_s8(vshrq_n_s8(vreinterpretq_s8_u8(mask), 7));
    return vbslq_u8(new_mask, b, a);
}

static inline uint8x16_t shuffle_neon(uint8x16_t a, uint8x16_t idx)
{
    int8x16_t tbl = vreinterpretq_s8_u8(a); // input a
    uint8x16_t idx_masked = vandq_u8(idx, vdupq_n_u8(0x8F)); // avoid using meaningless bits
    return vreinterpretq_u8_s8(vqtbl1q_s8(tbl, idx_masked));
}

```

Figure 5.15: rijndael_neon.h , Rijndael Round part 2

5.3.4 Fields

```

/
├── fields.h
├── fields/
│   ├── fields_avx2.h
│   ├── fields_avx512.h
│   ├── fields_common.h
│   ├── fields_handling.h
│   ├── + fields_neon.h
│   ├── fields_ref.h
│   └── gf256_mult_table.h

```

This section explains how I ported fields. The purpose of this part of the the MQOM repository is to provide an API for the finite-field operations used throughout the implementation, such as operations on field elements, vector operations, and matrix multiplications. Those operations are highly repetitive and vectorizable. To port this fields component, I modified `fields.h` to create an public interface for my NEON version and created a `fields_neon.h` based on the `fields_avx2.h` file. Since AVX2 is a good basis for a NEON implementation as they both provide similar functionality with different vector size.

fields.h

This file make the transition between the general API and the API of the custom API (e.G. AVX2, NEON, ...). The changes in this file are:

- **line 87 to 90** : Addition of the part that convert the NEON API to other function of the NEON API.
- **line 94 to 108** : Addition of the NEON platform detector
- **line 133 to 136** : Addition the part to include the `fields_neon.h`

fields_neon.h

`fields_neon.h` was ported from `fields_avx2.h`. The biggest challenge here is that, unlike what we saw in Rijndael where we only had 128-bit vectors, this file uses 256-bit vectors. The approach used to solve this challenge will be explained later in this section. First I'll explain what the Table 5.3 represents. Those are all the functions that were ported from AVX2 to NEON. Since the functions follow the same porting principles, only the general approach used for the different types of operations will be explained.

Return Type (NEON)	AVX2 Functions	NEON Functions
uint64x2_t	load_incomplete_m256_aligned32	load_incomplete_128_aligned16
uint64x2_t	load_incomplete_m256_unaligned32	load_incomplete_128_unaligned16
uint64x2_t	load_incomplete_m256	load_incomplete_128
void	store_incomplete_m256_aligned32	store_incomplete_128_aligned16
void	store_incomplete_m256_unaligned32	store_incomplete_128_unaligned16
void	store_incomplete_m256	store_incomplete_128
uint8_t	parity_avx2	parity_neon
uint8_t	sum_uint8_avx2	sum_uint8_neon
uint16_t	sum_uint16_avx2	sum_uint16_neon
uint8_t	gf2_vect_mult_avx2	gf2_vect_mult_neon
uint64x2_t	gf256_mult_vectorized_avx2	gf256_mult_vectorized_neon*
void	gf256_constant_vect_mult_avx2	gf256_constant_vect_mult_neon
uint8_t	gf256_vect_mult_avx2	gf256_vect_mult_neon
uint8_t	gf2_gf256_vect_mult_avx2	gf2_gf256_vect_mult_neon
void	gf256_gf2_constant_vect_mult_avx2	gf256_gf2_constant_vect_mult_neon
void	gf256_vect_lift_from_gf16_avx2	gf256_vect_lift_from_gf16_neon
uint8_t	gf16_gf256_vect_mult_avx2	gf16_gf256_vect_mult_neon
uint64x2_t	gf256to2_mult_vectorized_avx2	gf256to2_mult_vectorized_neon*
void	gf256to2_constant_vect_mult_avx2	gf256to2_constant_vect_mult_neon
uint16_t	gf256to2_vect_mult_avx2	gf256to2_vect_mult_neon
void	gf256to2_gf2_constant_vect_mult_avx2	gf256to2_gf2_constant_vect_mult_neon
void	gf256to2_gf256_constant_vect_mult_avx2	gf256to2_gf256_constant_vect_mult_neon
uint16_t	gf2_gf256to2_vect_mult_avx2	gf2_gf256to2_vect_mult_neon
uint16_t	gf256_gf256to2_vect_mult_avx2	gf256_gf256to2_vect_mult_neon
uint16_t	gf16_gf256to2_vect_mult_avx2	gf16_gf256to2_vect_mult_neon

Table 5.3: Correspondence between AVX2 and NEON functions

Let's start with the adaptation of 256-bit intrinsics into 128-bit intrinsics. All the functions whose names begin with `gf` and return `void` work the same way : the data is loaded, processed then stored. For all functions whose names begin with `gf` and return a scalar value : the data is loaded,

processed then the result is xored with an accumulator. For all of these functions that I described, the data is loaded through a loop which reads 256 bits at a time. I changed the amount of data processed per iteration to read 128 bits at a time to be able to create 128-bit vectors to use the NEON intrinsics.

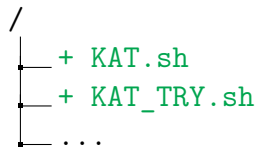
Now that the amount of data processed per iteration is reduced, I replaced all the AVX2 intrinsics with their NEON equivalent. Some AVX2 intrinsics do not have direct NEON equivalents, so I relied on SSE2NEON for those. Constant vectors are also created in those functions. By taking only the values of the first 128-bits, those functions stay correct since the constants used in these functions are applied independently to each lane.

The rest of the functions we didn't talk about, including `gf256_mult_vectorized_neon` and `gf256to2_mult_vectorized_neon`, are all adapted to take 128-bit vectors as input. As for the return type, it is set to 128-bit vector type for the functions returning vectors and unchanged for the others.

Another important aspect concerning the porting is that the AVX2 implementation also relied on GFNI intrinsics which weren't implemented in SSE2NEON. So, I ported them myself using the Intel documentation. Also, each time a vector was not processed directly in the function, I used `uint64x2_t` as a vector type to keep consistency throughout the file. This is purely a design choice.

5.3.5 Known answer test results

MQOM has already KAT test for me to use to verify the correctness of my implementation. "During development, five KAT request/response were used to reduce the testing time. Then, The final implementation was then validated with the complete KAT set.



I created the `KAT.sh` and `KAT_TRY.sh` files to verify the correctness of thre different versions. `KAT.sh` compile or verifie the overall correctness while `KAT_TRY.sh` allowed me to verify only one version.

Figure 5.16: MQOM KAT NEON implementation

Before starting the debugging , I changed the -o3 compilation flag for -o0 to disable compiler optimizations and simplify debugging. When I saw an incorrect version, I debugged using gdb and the correct AVX2 version next to it to understand where was the divergence coming from.

How to verify correctness with KAT :

- `bash KAT.sh build 1 1` (build_avx2 if you whant to compile for AVX2)
- `bash KAT.sh test` (test_avx2 if you whant to test for AVX2)

In the end all my version where successful and passed all the KAT.

XOFx4	ENCx8	-o0	-o3
1	1	✓	✓
1	0	✓	✓
0	1	✓	✓
0	0	✓	✓

Table 5.4: MQOM NEON version verified with or without XOFx4 or ENCx8

Small note on benchmarking: There were no active modifications to the benchmarking files, as they already supported benchmarking on ARM-A. The only change I made was in `‘/benchmark/utlis.h’`, where I modified the lines 46 to 48 to adjust the printed information.

5.4 Analysis of internal performance

In this section, the reference version are benchmarked over 50 tests. This is because they were the slowest but as we will see this won't have any impact in the analysis.

5.4.1 Key Generation

Overall

In Figure 5.17, the AVX2 implementation of the key generation algorithm achieves an average time reduction of 89.85%. In Figure 5.18, my NEON implementation achieves an average time reduction of 83.89% for the key generation algorithm.

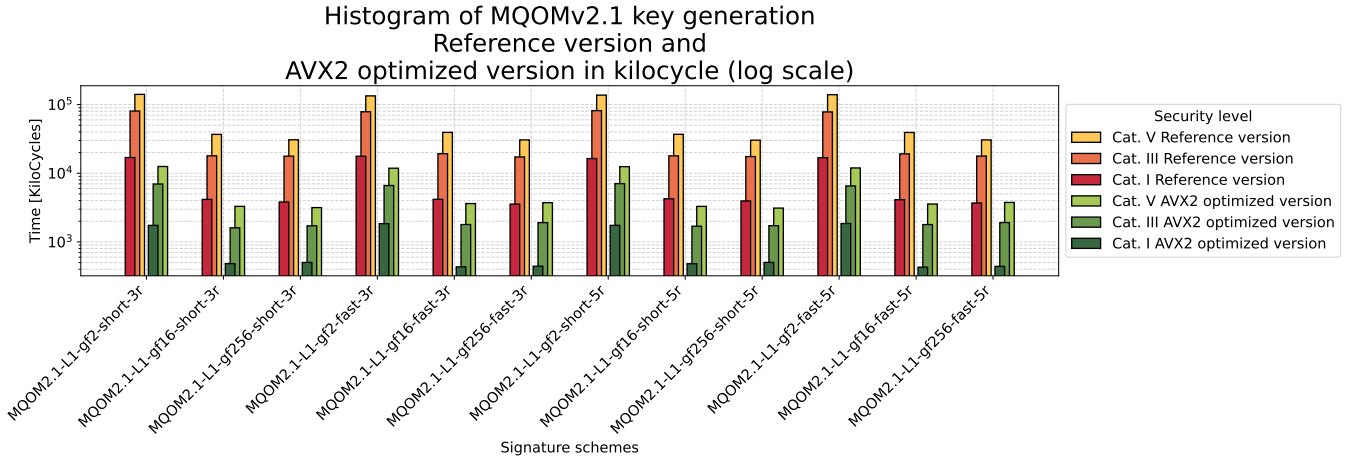


Figure 5.17: Comparison of MQOMv2.1 Key generation, Reference and AVX2 versions (Table C.4)

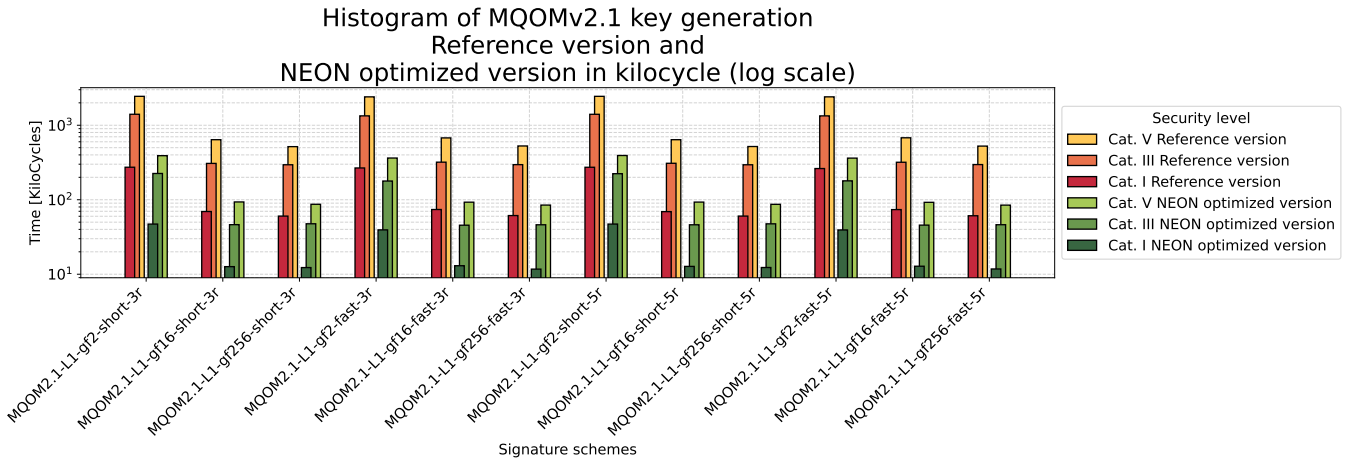


Figure 5.18: Comparison of MQOMv2.1 Key generation, Reference and NEON versions (Table C.5)

Internal

For the internal analysis of MQOMv2.1, I'll only present the first category to keep the analysis concise. I verified the other categories and they follow the same pattern.

The internal analysis is based on the pseudocode given by the reference paper[25]. I divided the code into these sections: "declarations", "allocation", "sanity checks", "row 1-2", "row 3", "row 4 (loop)", "row 7", "row 8", "cleanup".

In Figure 5.19, we can observe that the rows taking the most time are row 3 by a big margin, followed by row 4 loop and cleanup. Row 3 call the ExpandEquation function. Row 4 loop compute the matrix equation. Cleanup free the memory.

In Figure 5.20, we can observe that the row 4 loop takes a significant mare time portion than on the Figure 5.19. While row 3 reducce its time portion.

In Figure 5.21, we can observe the same conclusion as we saw before with a small variation in cleanup.

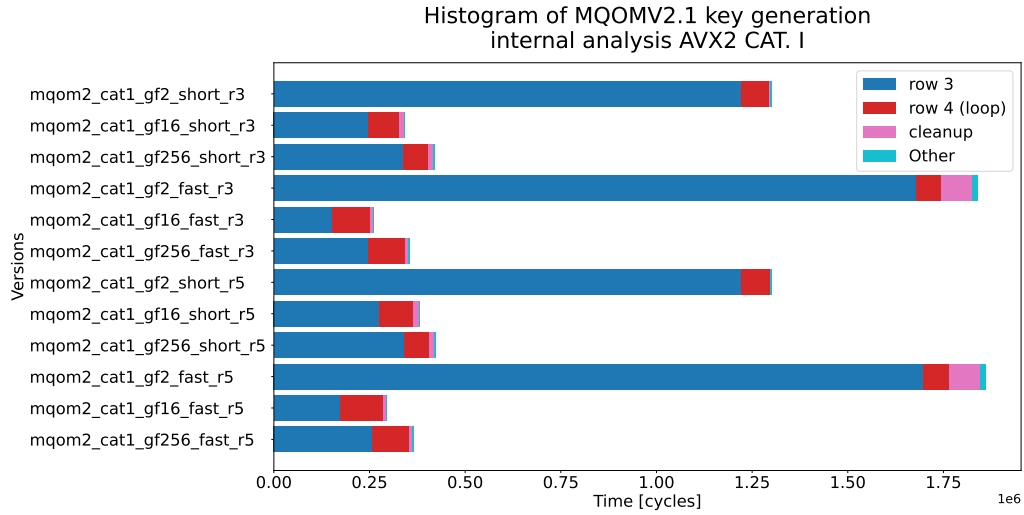


Figure 5.19: MQOMv2.1 internal analysis of Key generation for AVX2 implementation² (Table C.26)

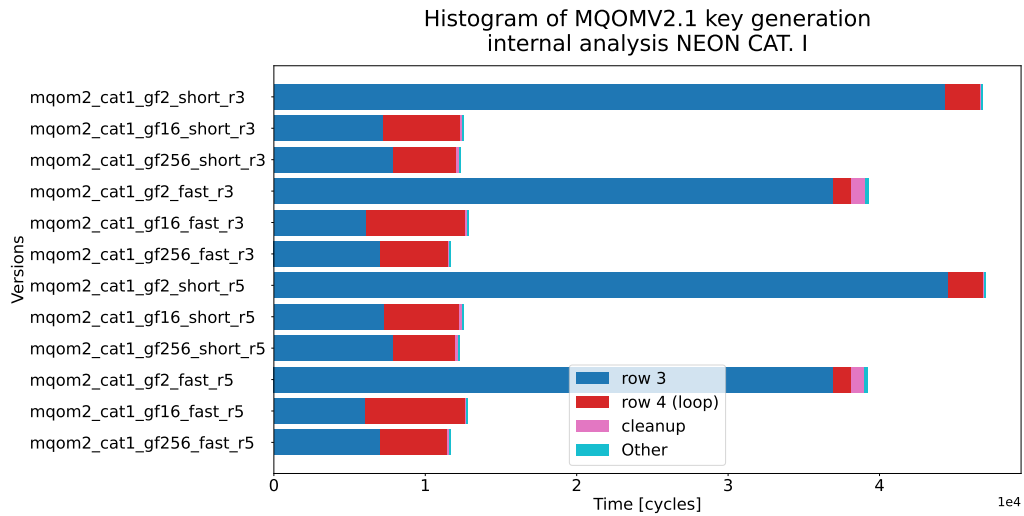


Figure 5.20: MQOMv2.1 internal analysis of Key generation for NEON implementation (Table C.27)

²The sections taking less than 1% are classified as other.

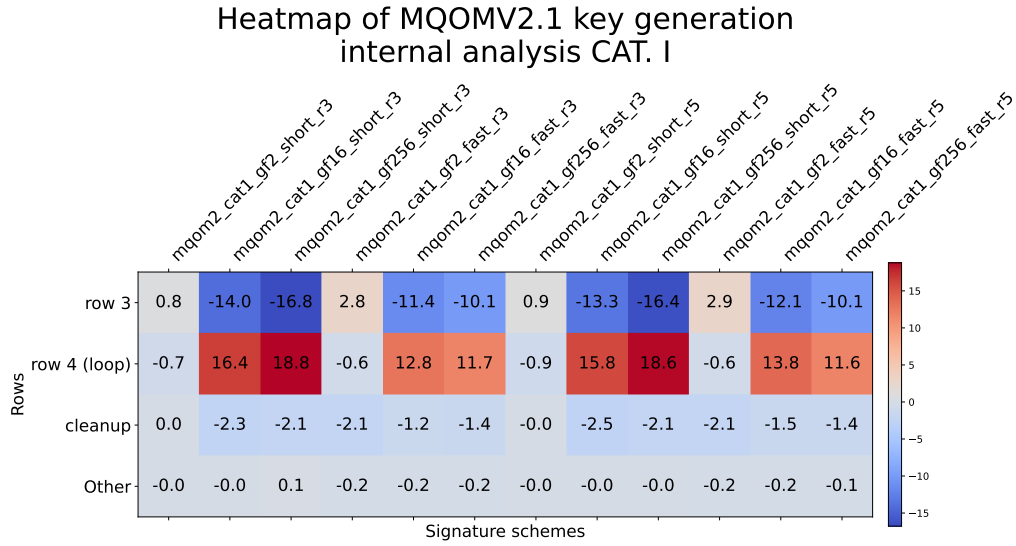


Figure 5.21: Difference in proportion (NEON - AVX2) for MQOMv2.1 key generation internal analysis (Tables C.26, C.27)

5.4.2 Signing Algorithm

Overall

In Figure 5.22, the AVX2 implementation of the signing algorithm achieves an average time reduction of 87.43%. In Figure 5.23, my NEON implementation achieves an average time reduction of 74.6% for the signing algorithm.

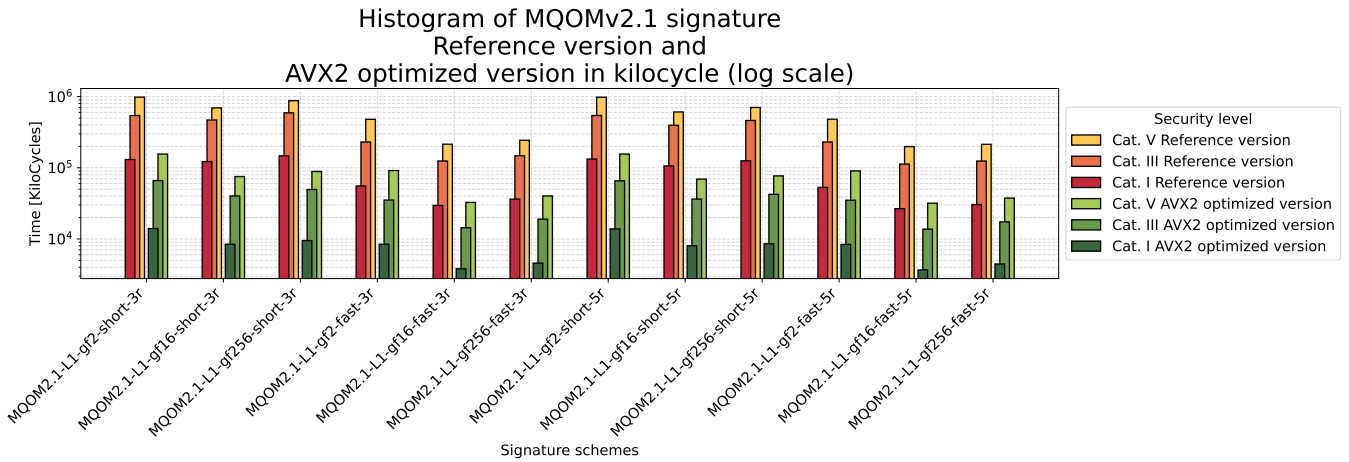


Figure 5.22: Comparison of MQOMv2.1 signing algorithm, Reference and AVX2 versions (Table C.6)

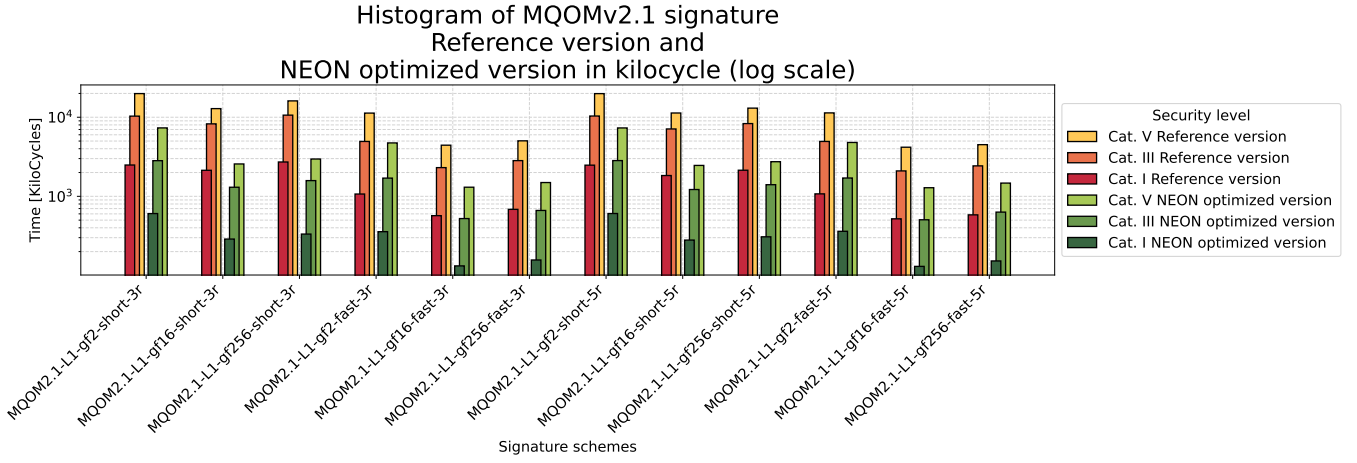


Figure 5.23: Comparison of MQOMv2.1 signing algorithm, Reference and NEON versions (Table C.7)

Internal

The internal analysis is based on the pseudocode given by the reference paper[25]. I divided the code into these sections : "declarations", "row 1-2", "row 5", "signature preparation", "row 6", "row 7", "row 8", "row 8 continued", "row 9", "row 10", "row 11", "return success".

In Figure 5.24, we can observe that the rows taking the most time are row 6, row 7 followed far away by the row 10. Row 6 is the commitment. Row 7 is the Compute Alpha. Row 10 is the sample Challenge

In Figure 5.25, we can observe that overall row 7 tend to take more space in the NEON implementation than in the AVX2 implementation.

In Figure 5.21, we can observe exactly what I said before with a small variation on row 101;

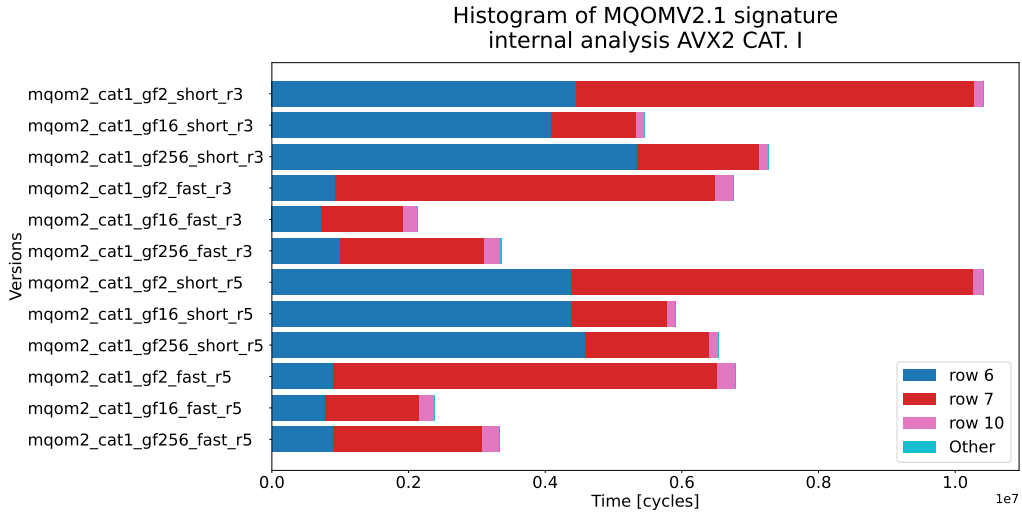


Figure 5.24: MQOMv2.1 internal analysis of signing algorithm for AVX2 implementation (Table C.28)

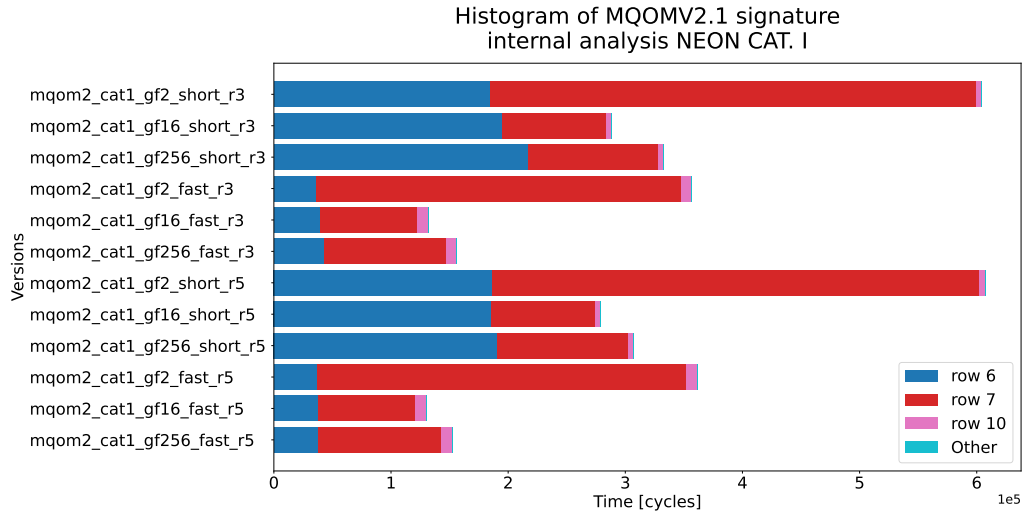


Figure 5.25: MQOMv2.1 internal analysis of signing algorithm for NEON implementation (Table C.29)

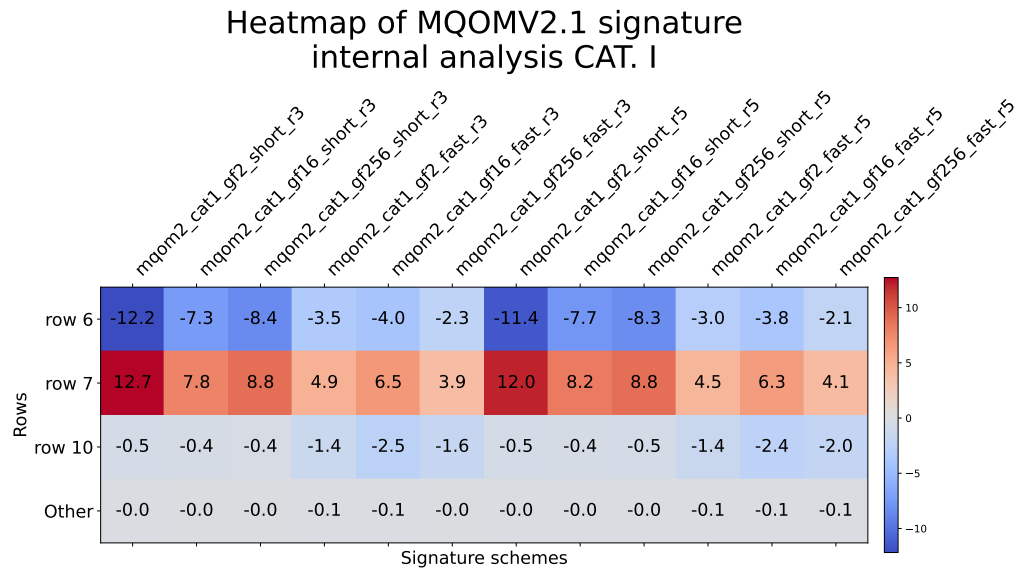


Figure 5.26: Difference in proportion (NEON - AVX2) for MQOMv2.1 signing algorithm internal analysis (Tables C.28, C.29)

5.4.3 Verification Algorithm

Overall

In Figure 5.27, the AVX2 implementation of the verification algorithm achieves an average time reduction of 87.89%. In Figure 5.28, my NEON implementation achieves an average time reduction of 75.44% for the verification algorithm.

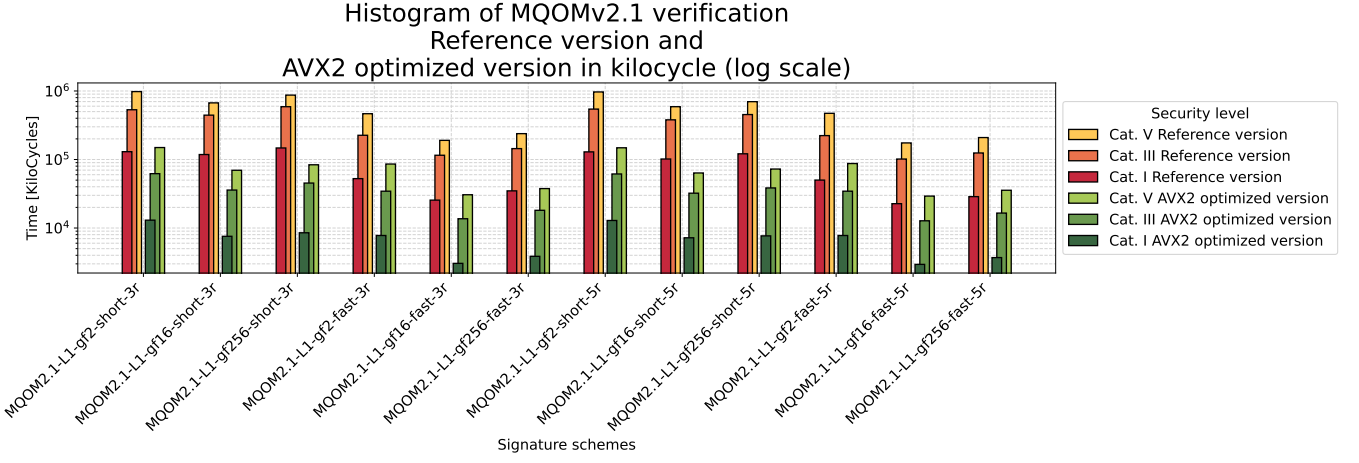


Figure 5.27: Comparison of MQOM verification algorithm, Reference and AVX2 versions (Table C.8)

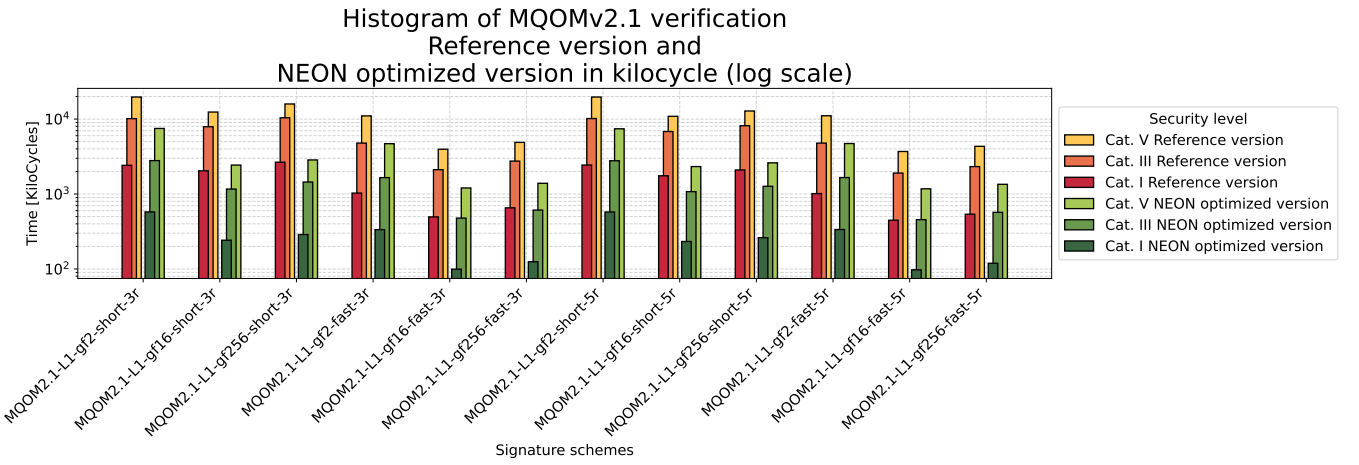


Figure 5.28: Comparison of MQOM verification algorithm, Reference and NEON versions (Table C.9)

Internal

The internal analysis is based on the pseudocode given by the reference paper[25]. I divided the code into these sections : "declaration", "row 1", "row 2", "row 3", "row 4", "row 5", "row 6", "row 7", "row 8 and 9", "row 10", "row 11", "row 12".

In Figure 5.29, we can observe that the rows taking the most time are row 7 and "row 8 and 9". Row 7 is the evaluation. Row 8 and 9 are the recompute alpha.

In Figure 5.30, we can observe that Row 8 and 9 tend to take more space than in the Figure 5.29

In Figure 5.31, we have the same observation as just above. the Row 8 and 9 take as a bigger time proportion in the NEON implementation. While in the AVX2 implementation, Row 7 takes a bigger time proportion.

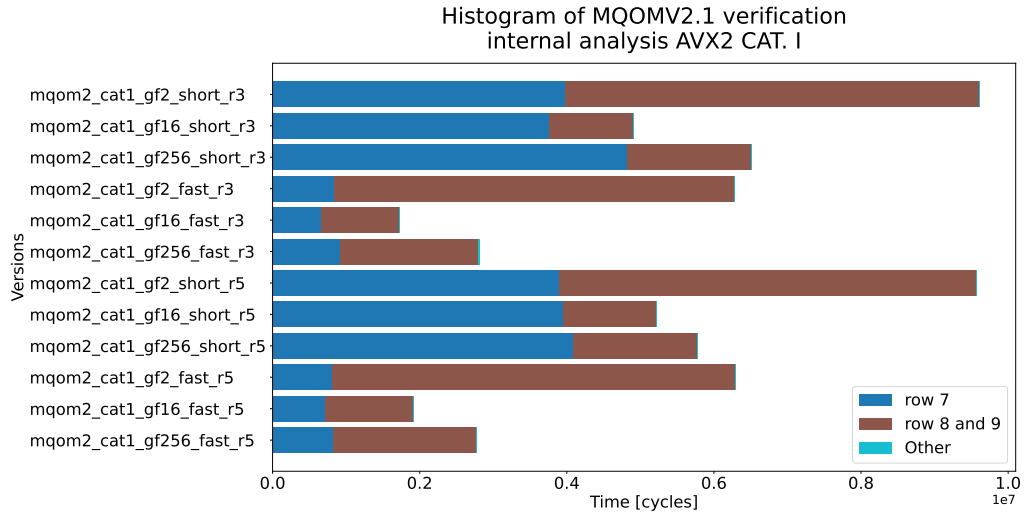


Figure 5.29: MQOMv2.1 internal analysis of verification algorithm for AVX2 implementation (Table C.30)

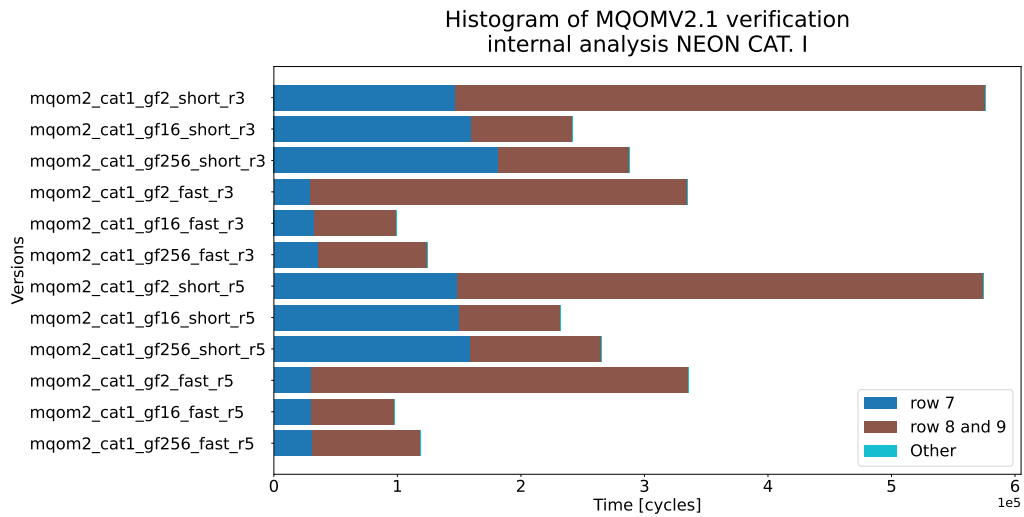


Figure 5.30: MQOMv2.1 internal analysis of verification algorithm for NEON implementation (Table C.31)

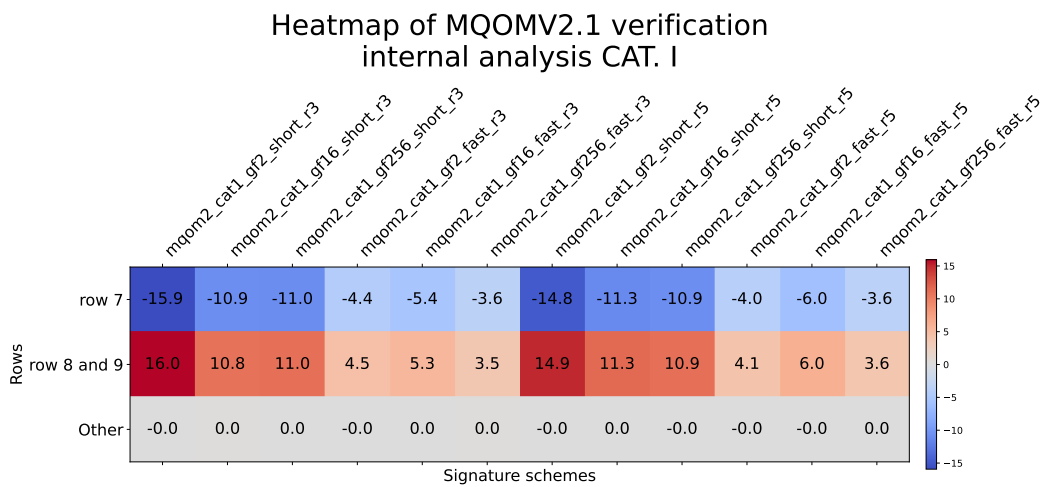


Figure 5.31: Difference in proportion (NEON - AVX2) for MQOMv2.1 verification algorithm internal analysis (Tables C.30, C.31)

5.5 Conclusion and Future work

5.5.1 Conclusion

	AVX2	NEON
Key generation	89.85%	83.89%
Signing algorithm	89.85%	74.60%
Verification algorithm	87.89%	75.44%

Table 5.5: MQOM: Performance comparison between AVX2 and NEON

In this chapter, I developed a NEON implementation from the AVX2 implementation and verified that this new implementation was correct.

To conclude, we can show on the Table 5.5, the results obtained for MQOM show that both optimized implementations significantly improve the performance compared to the reference implementation. A big part could come from SHA-3 vectorization and AES vectorization.

Here we can see that the ARM implementation have a smaller reduction time than the AVX2 implementation. This goes in the same direction as what my working hypothesis predicts. But without other comparison numbers, we can't decide now if it means that my working hypothesis is correct or not. We will need the MAYO time reduction to verify.

The internal analysis also showed different proportions between both implementations. This observed differences could be from the different instruction sets provided by x86 and ARM-A.

5.5.2 Future work

For future work, we have different option:

- Implement an ARMv8.2 implementation: on the XKCP GitHub repository [14], the Keccak Team[15] released a SHA-3 implementation using the new vectorized instructions we saw in 2.3.4.
- We can reduce the number of versions to only a small list of potential standards like MAYO did. Instead of studying all the different versions.
- Bench and analyze the loop and condition internal analysis from the key generation, signing and verification algorithm. And make the same analysis on other important internal function. (e.g. ExpandEquations, ComputeAlpha, ...)
- Some part may be more time consuming. I think it could be a good idea to port those into assembly. It could improve the performances.

Chapter 6

Comparison

In this Chapter, we will compare the different signature schemes on the different aspects analyzed before. The first part will be a benchmark comparison and the second a table comparison.

6.1 Size Comparison

In Figure 6.1, we can observe that MQOM has the worst private key size. In Figure 6.2, we can see that MAYO has the worst public key. In Figure 6.3, we can observe that MAYO has the best signature size.

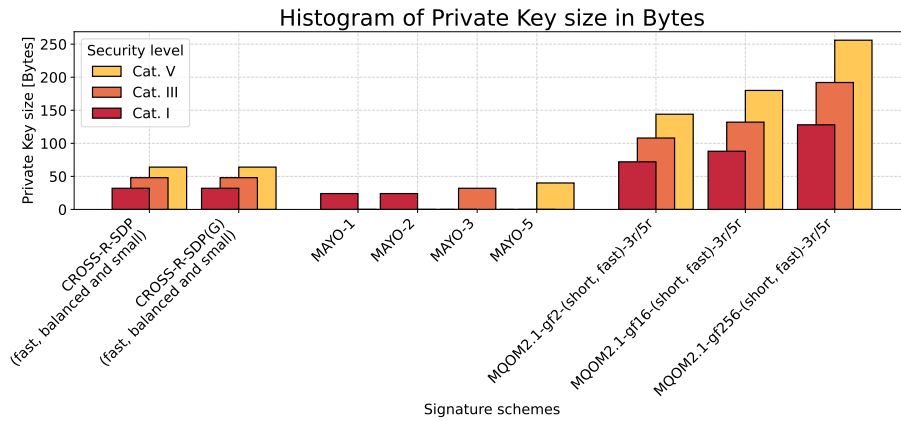


Figure 6.1: Comparison of the private key size of CROSS, MAYO and MQOM (Table C.1)

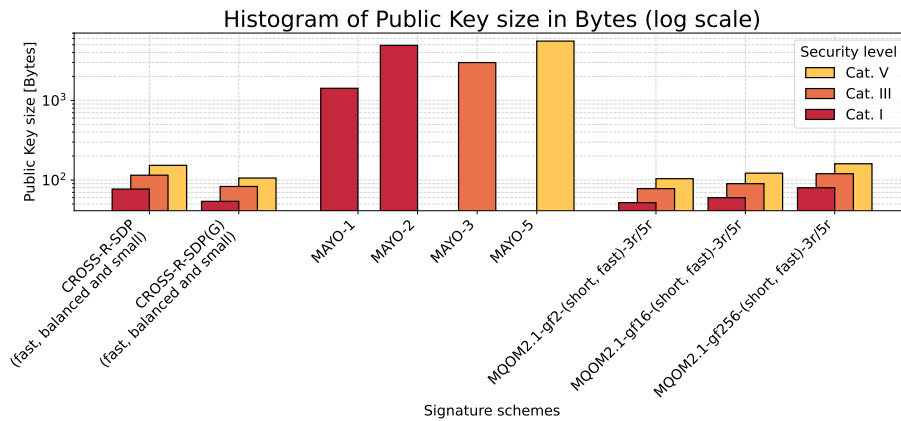


Figure 6.2: Comparison of the public key size of CROSS, MAYO and MQOM (Table C.2)

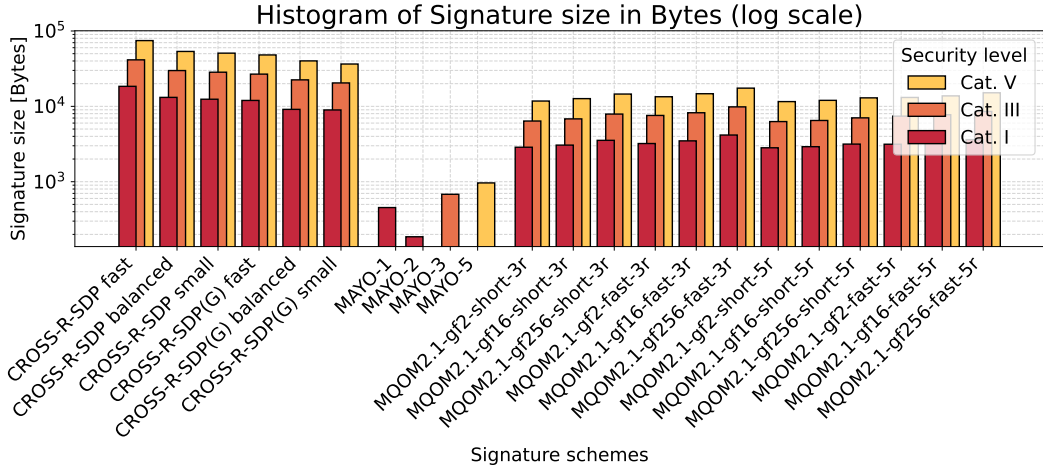


Figure 6.3: Comparison of the signature size of CROSS, MAYO and MQOM (Table C.3)

6.2 Performance Comparison

In this section, I will compare the different signature schemes and their AVX2 and NEON implementation. Let's remember that the AVX2 and NEON implementations of CROSS are not exactly the same. The CrossNEON implementation has upgrades that could be ported on AVX2 and are currently not.

Key Generation

In Figure 6.4, we can see that, for the key generation of the AVX2 implementations, CROSS achieves the best performance, followed by MAYO, with MQOMv2.1 being the slowest of the three schemes. In Figure 6.5, we can see that, for the key generation of the NEON implementations, CROSS achieves the best performance, followed by MQOMv2.1, with MAYO being the slowest of the three schemes.

When comparing Figure 6.4 and Figure 6.5, we can see something surprising, the order is not the same. We can deduce that the optimization had an influence on the implementations.

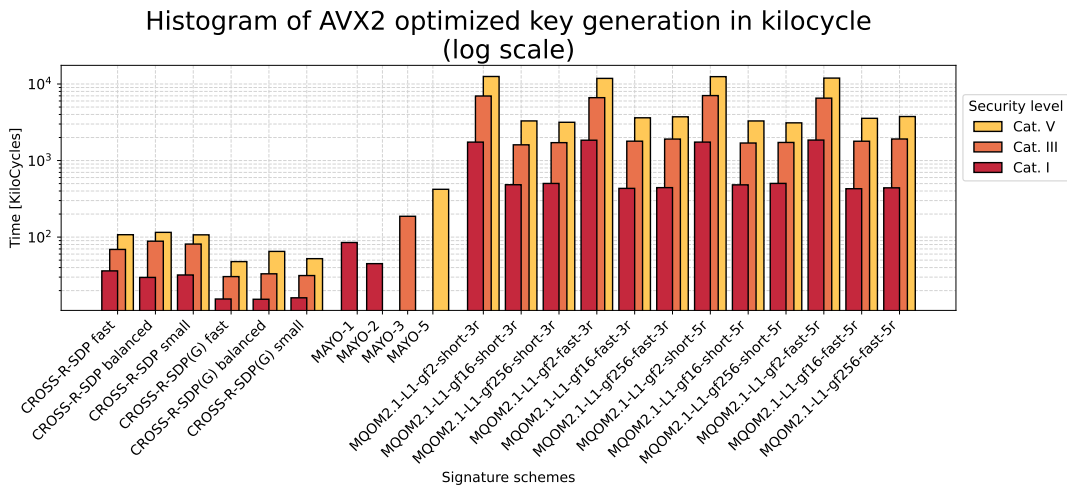


Figure 6.4: Comparison of signature schemes key generation by time for the AVX2 optimized implementations of each version run on Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (Table C.4)

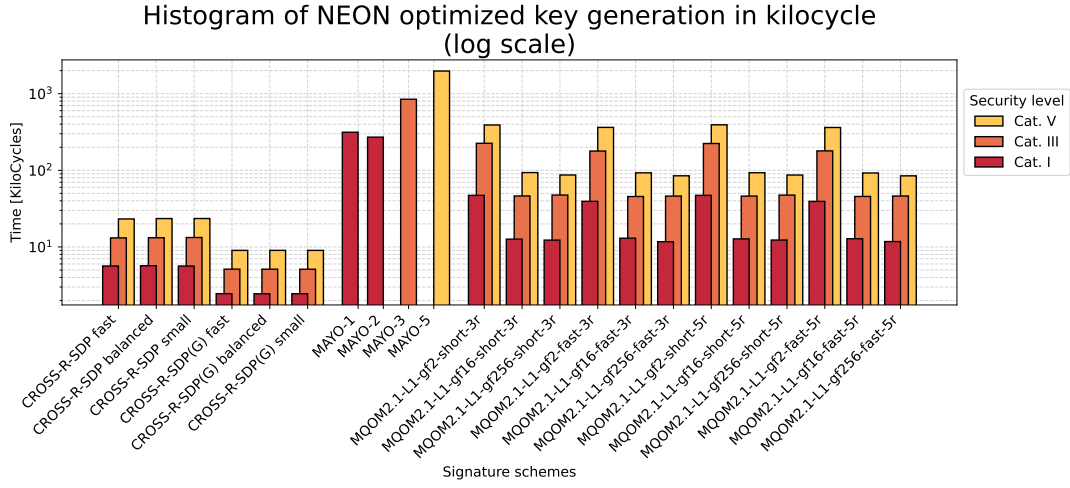


Figure 6.5: Comparison of signature schemes key generation by time for the NEON optimized implementations of each version run on Cortex-A76 (Table C.5)

Signature

In Figure 6.6, we can see that, for the signing algorithm of the AVX2 implementations, MAYO achieves the best performance, followed by CROSS and MQOMv2.1, with almost equivalent performance. In Figure 6.7, we can see that, for the signing algorithm of the NEON implementation, we don't have a clear ranking. CROSS has the best overall performances. While MAYO can be classed as the overall slowest. But MQOMv2.1 versions have the best and the worst performances. The versions are on a large spectrum.

When comparing Figure 6.6 and Figure 6.7, here again we see a difference in ranking. This is particularly concerning for MQOM. For me a part of the difference can be from the implementations used for SHA-3. We can also see a performance loss for MAYO from AVX2 to NEON.

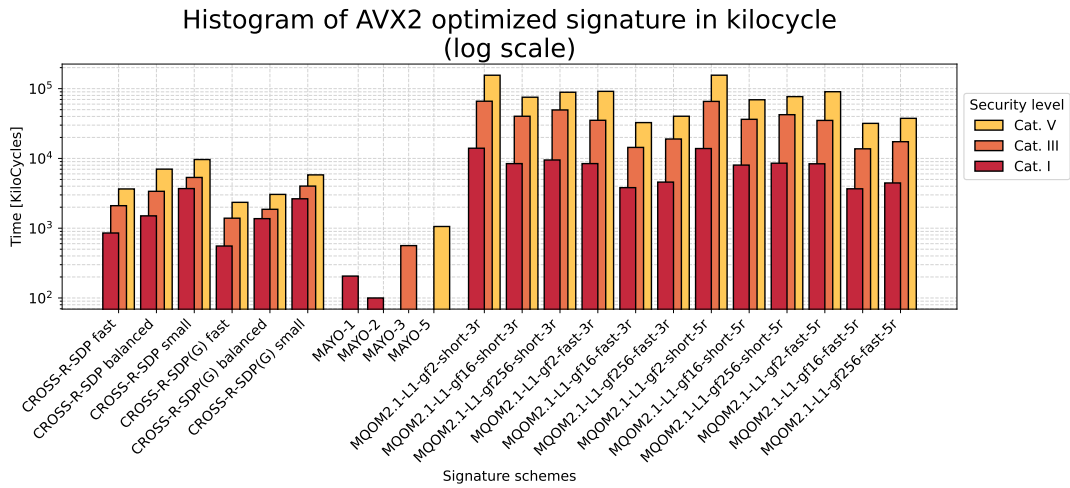


Figure 6.6: Comparison of signature schemes signing algorithm by time for the AVX2 optimized implementations of each version run on Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (Table C.6)

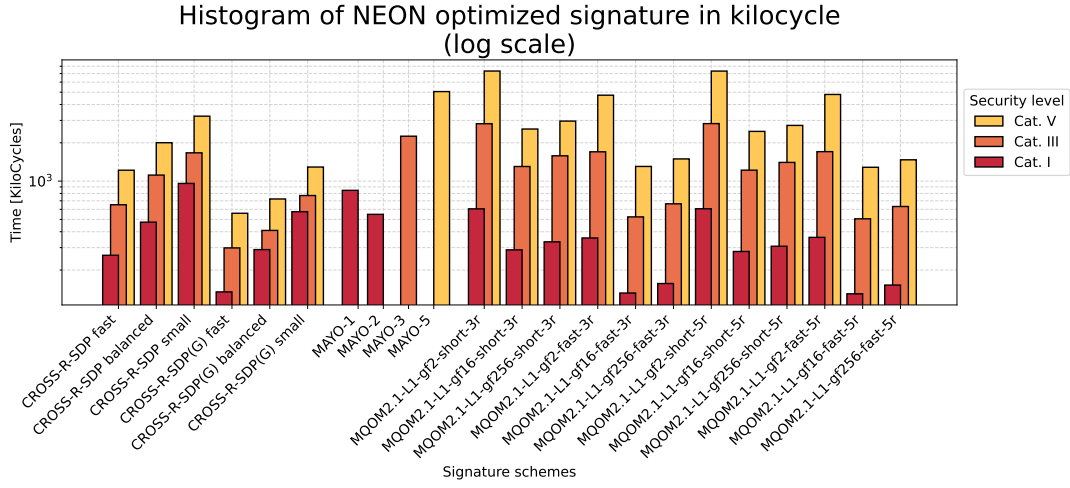


Figure 6.7: Comparison of signature schemes signing algorithm by time for the NEON optimized implementations of each version run on Cortex-A76 (Table C.7)

Verification

In Figure 6.8, we can see that, for the verification algorithm of the AVX2 implementation, MAYO achieves the best performance, followed by CROSS and MQOMv2.1, with almost equivalent performance. In Figure 6.9, we can see that, for the verification algorithm of the NEON implementation, here again, we can see that the ranking is not clear. CROSS achieve overall the best performances. MAYO perform better than for the signing algorithm. And MQOMv2.1 again perform on a large scale.

When comparing Figure 6.8 and Figure 6.9, almost the same trend can be observed here as in the signature section. Although, we denote less performances from MQOMv2.1.

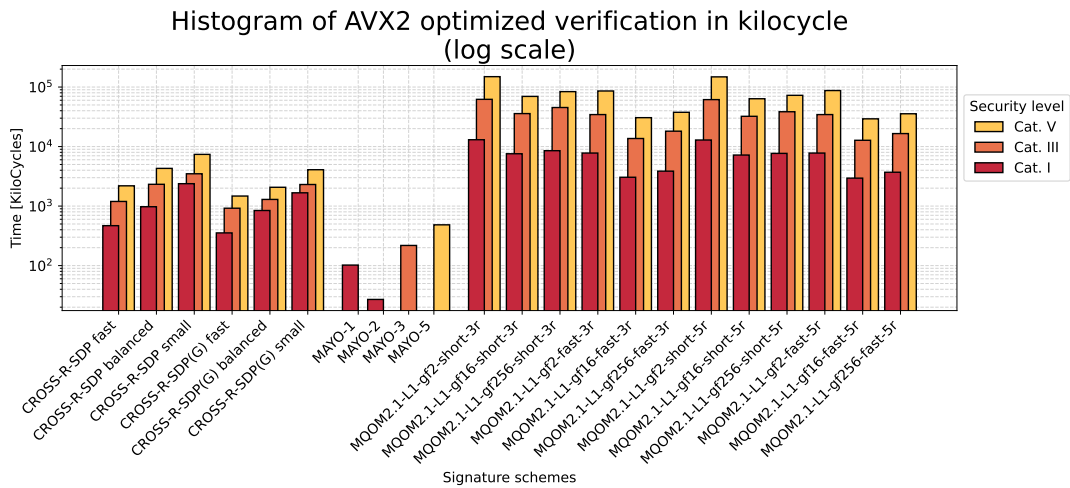


Figure 6.8: Comparison of signature schemes verification algorithm by time for the AVX2 optimized implementations of each version run on Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (Table C.8)

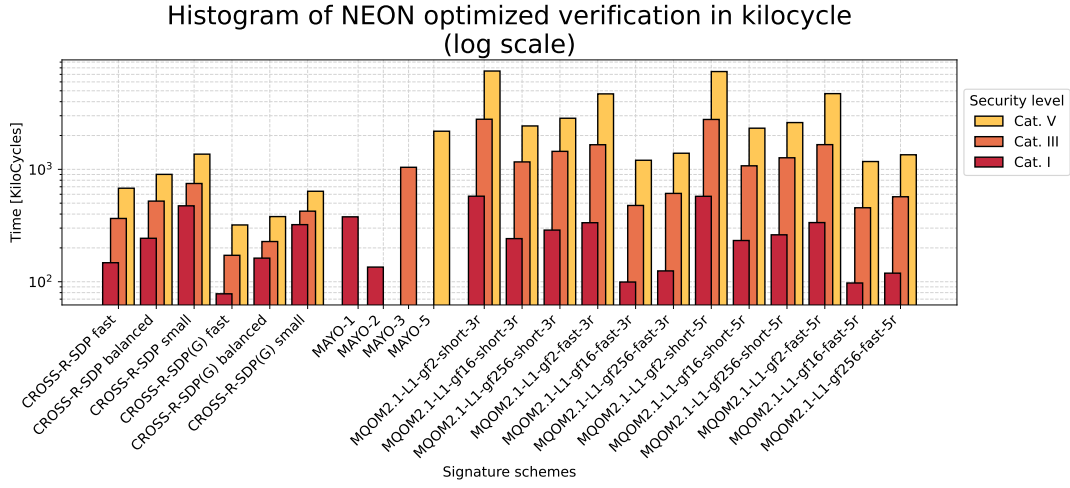


Figure 6.9: Comparison of signature schemes verification algorithm by time for the NEON optimized implementations of each version run on Cortex-A76 (Table C.9)

6.3 Signature Comparison Recap

The Table 6.1 compile a summary of the different results from this Master's thesis plus some interesting observations.

	CROSS	MAYO	MQOMv2.1
description	R-SDP and R-SDP(G) and CROSS-ID	Oil & Vinegar derived	Multivariate Quadratic Problem and polynomial (PIOP)
use AES	no	yes	yes
why I chose it	easy to understand, implement and based on theory seen in course	one of the best ones on the comparison site	test a new discovery (PIOP)
positive	fast reference version	small signature	small public key
negative	large signature	large public key	slow overall
Authors of signatures comment	high flexibility but large signature	small key, small signature size and flexible but limitations for scalability to higher security levels	Small public key, highly parallelizable and relatively small signatures but relatively slow and bad scalability for higher security levels
Mean size [Byte]			
private key	48	30	145
public key	98	3 718	866
signature	30 390	572	8 130
Reference version	yes	yes	yes
AVX2 optimized version	yes	yes	yes
NEON optimized version	no	yes	no
Cortex M4 optimized version	yes (unofficial)	yes	no
Average reduction time with AVX2 optimization version			
key generation	25.63%	96.44%	89.85%
signing	37.92%	93.93%	89.85%
verification	30.59%	95.99%	87.89%
Average reduction time with NEON optimization version			

key generation	40.38%	95.08%	83.89%
signing	61.05%	92.52%	74.60%
verification	52.79%	92.28%	75.44%
Mean execution time with reference version on x86 [kilocycles]			
key generation	77	5 154	38 578
signing	5 360	7 858	323 456
verification	3 014	4 716	315 705
Mean execution time with AVX2 optimized version [kilocycles]			
key generation	54	184	3 570
signing	3 346	482	41 010
verification	2 207	208	38 474
Mean execution time with reference version on ARM [kilocycles]			
key generation	16	17 427	665
signing	2 710	29 830	6 298
verification	1 018	10 638	6 116
Mean execution time with NEON optimized version [kilocycles]			
key generation	10	851	102
signing	925	2 176	1 696
verification	454	935	1 636

Table 6.1: Public Key size in Bytes (Table C.2)

One interesting result we can analyze, from the table 6.1, is the average reduction time. Firstly, we can see that the average reduction time for MAYO and MQOM is bigger than for CROSS. Seeing the similarity in MAYO and MQOM, we can conclude that this is due to AES vectorization since this is the only difference between CROSS and the two other schemes that could explain it. Another observation that we can make is the validation of the working hypothesis. Since MAYO and MQOM have for a similar version an AVX2 implementation reduction time greater than for the NEON implementation. With the hypothesis validated. We can conclude that a ported version of CrossNEON would be faster than the current AVX2 implementation.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

The objective of this master thesis was to study the performance of three post-quantum signature schemes, CROSS, MAYO and MQOM, and more particularly to compare their optimized implementations on x86 and ARM architectures to answer the questions:

- How do CROSS, MAYO, and MQOM perform on the current x86 and ARM-A architectures?
- How could the performance of the different signature schemes evolve with future hardware and software optimizations for AVX2 and NEON versions?

I first studied the main differences between both architectures and their instruction sets. This comparison showed an important difference for vectorization, with AVX2 using 256-bit vectors while NEON uses 128-bit vectors. I also observed differences between AES-NI and the ARM AES instructions. From these observations, two working hypotheses were made. The first one was that the smaller vector size of NEON could lead to a lower performance improvement than AVX2. The second one was that MAYO and MQOM, which use AES, could also have a lower performance improvement on ARM because of the difference between AES-NI and ARM AES.

To study these hypotheses, the work was then divided between the three signature schemes. For each scheme, I first studied how the algorithm works and how the optimized implementations differ from the reference implementation. I then benchmarked key generation, signing and verification. In addition to the overall benchmarks, an internal benchmark was developed to divide the algorithms into different parts. This allowed us not only to see if an optimized implementation was faster, but also to understand where the execution time was spent and how the proportions changed between AVX2 and NEON.

For CROSS, both optimized implementations improve the performance compared to the reference implementation. The AVX2 implementation reduces the execution time by 25.63%, 37.92% and 30.59% for key generation, signing and verification, while CrossNEON reduces it by 40.38%, 61.05% and 52.79%. At first sight, these results are opposite to our first hypothesis because the NEON version obtains a better improvement. However, the comparison is not direct. CrossNEON contains additional optimizations, in particular new modular reductions and vectorized tree structures, which are not present in the AVX2 implementation. The CROSS results therefore cannot be used alone to validate or reject the hypothesis.

For MAYO, the results give a different situation. The optimized implementations are much faster than the reference implementation. AVX2 reduces the execution time by 96.44%, 93.93% and 95.99% for key generation, signing and verification. For NEON, these reductions are 95.08%, 92.52% and 92.28%. In the three cases, the AVX2 reduction is therefore slightly better than the NEON reduction. A large part of the improvement compared to the reference implementation comes from the use of hardware AES instructions instead of the C implementation used in the reference version. These results are backing the working hypotheses made from the comparison of AVX2/NEON and AES-NI/ARM AES.

Finally, for MQOM, a NEON implementation was developed during this master thesis starting from the AVX2 optimized implementation. Its correctness was verified before benchmarking it. The results again show a large improvement compared to the reference implementation. AVX2 reduces the execution time by 89.85%, 89.85% and 87.89% for key generation, signing and verification, while the NEON implementation reduces it by 83.89%, 74.60% and 75.44%. Contrary to CROSS, the AVX2 implementation therefore obtains a clearly larger reduction than NEON. The internal analysis also showed differences in the proportions of the different parts of the algorithms, which confirms that the architecture and the available instruction sets have an important influence on the final performance.

With the results of the three schemes, I can finally come back to the working hypotheses. MAYO and MQOM follow the expected behavior: their AVX2 implementations obtain a better reduction from the reference implementation than their NEON implementations. This is consistent with both the difference in vector width between AVX2 and NEON and, for these two schemes, the difference observed between AES-NI and ARM AES. CROSS does not follow this tendency, but its NEON and AVX2 implementations do not contain the same optimizations, so it cannot be considered as a counterexample to the hypothesis. Overall, the experimental my results support my working hypotheses, even if they do not allow us to isolate exactly how much of the difference comes from SIMD, AES or other implementation choices.

The final comparison also showed that there is no single signature scheme which is the best for every aspects. CROSS gives very good execution times, while MAYO provides particularly small signatures, and the different schemes also have important differences in public and private key sizes. The comparison between AVX2 and NEON also showed that the performance of a post-quantum signature scheme cannot be separated from its implementation and from the target architecture. An algorithm can therefore have very different practical performance depending on the optimizations and instruction sets available on the processor. Furthermore, the implementations were only benchmarked on one device for each architecture. Therefore, those results are specific to the tested devices and should not be generalized to all x86 and ARM-based systems. This limitation should be taken into account when interpreting the performance differences observed between AVX2 and NEON.

In the end, this work also showed that the performance of these three signature schemes should not be considered as fixed. New improvements could arise from new algorithmic discoveries, new hardware capabilities, lower programming languages. The results obtained in this thesis represent only a snapshot of the current state of these implementations. As both post-quantum cryptography and processor architectures will continue to progress, those schemes will continue to constantly evolve.

7.2 Future work

In this section, I'll first summarize

CROSS specific:

- Port and test CrossNEON implementation on AVX2 implementation.
- We can reduce the number of versions to only a small list of potential standards like, as we will see, MAYO did. Instead of studying all the different versions.
- Analyze the loop and condition internal analysis from the key generation, signing and verification algorithm. That I couldn't analyze here. And make the same analysis on other important internal function. (e.g. the Tree functions)
- When I looked into the code, I realized that the Tree functions are replaced by `memcpy` since depending on which macro is defined, the `#if` branch or the `#else` branch is compiled in the fast version. If we could directly go into the structure without copying we could gain some time.

MAYO specific:

- Bench and analyze the loop and condition internal analysis from the key generation, signing and verification algorithm. And make the same analysis on other important internal function. (e.g. ExpandSK, ExpandPK, ...)

MQOM specific:

- We can reduce the number of versions to only a small list of potential standards like, as we will see, MAYO did. Instead of studying all the different versions.
- Bench and analyze the loop and condition internal analysis from the key generation, signing and verification algorithm. And make the same analysis on other important internal function. (e.g. ExpandEquations, ComputeAlpha, ...)

Common to the signature schemes:

- Implement an ARMv8.2 implementation: on the XKCP GitHub repository [14], the Keccak Team[15] released a SHA-3 implementation using the new vectorized instructions we saw in 2.3.4.
- Some part may be more time consuming. I think it could be a good idea to port those into assembly. It could improve the performances.

Other possible options:

- Create an SVE/SVE2 implementation for ARM-A.
- Study the possibility of SHA-3 specific instructions for Intel.
- Benchmark the signature schemes on multiple additional platforms to assess the stability of the observed performance and the hypotheses.

7.3 References

A lot of my references does not come from academic papers but on "work in progress" from the NIST competition and varoious implementations developed by the research centers.

7.4 AI usage

Artificial intelligence tools were used to understand principles, proofreading and to reformulate some parts of the text to make them clearer and easier to understand.

Bibliography

- [1] National Institute of Standards and Technology (NIST). *Post-Quantum Cryptography | CSRC | CSRC*. Jan. 2017. URL: <https://csrc.nist.gov/projects/post-quantum-cryptography>.
- [2] National Institute of Standards and Technology (NIST). *Post-Quantum Cryptography: Selected Algorithms*. Jan. 2017. URL: <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/selected-algorithms>.
- [3] National Institute of Standards and Technology (NIST). *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Tech. rep. Federal Information Processing Standard (FIPS) 203. U.S. Department of Commerce, Aug. 2024. DOI: 10.6028/NIST.FIPS.203. URL: <https://csrc.nist.gov/pubs/fips/203/final>.
- [4] National Institute of Standards and Technology (NIST). *Module-Lattice-Based Digital Signature Standard*. Tech. rep. Federal Information Processing Standard (FIPS) 204. U.S. Department of Commerce, Aug. 2024. DOI: 10.6028/NIST.FIPS.204. URL: <https://csrc.nist.gov/pubs/fips/204/final>.
- [5] National Institute of Standards and Technology (NIST). *Stateless Hash-Based Digital Signature Standard*. Tech. rep. Federal Information Processing Standard (FIPS) 205. U.S. Department of Commerce, Aug. 2024. DOI: 10.6028/NIST.FIPS.205. URL: <https://csrc.nist.gov/pubs/fips/205/final>.
- [6] National Institute of Standards and Technology (NIST). *Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process*. Tech. rep. Sept. 2022. URL: <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf>.
- [7] Olivier Adjonyo, Arwa Alblooshi, Isaac Canales-Martinez, Jorge Chavez-Saab, Gilbert Dione, and Francisco Rodriguez henriquez. *PQ-SORT: Post-Quantum Signatures On-Ramp Tests*. Sept. 2025. URL: <https://pqsort.tii.ae/>.
- [8] National Institute of Standards and Technology (NIST). *SHA-3 standard : permutation-based hash and extendable-output functions*. Tech. rep. Federal Information Processing Standard (FIPS) 202. Washington, D.C.: National Institute of Standards and Technology (NIST), Aug. 2015. DOI: 10.6028/NIST.FIPS.202. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>.
- [9] Eslam G. AbdAllah, Yu Rang Kuang, and Changcheng Huang. “Advanced Encryption Standard New Instructions (AES-NI) Analysis: Security, Performance, and Power Consumption”. In: *Proceedings of the 2020 12th International Conference on Computer and Automation Engineering*. Sydney NSW Australia: ACM, Feb. 2020, pp. 167–172. DOI: 10.1145/3384613.3384648. URL: <https://dl.acm.org/doi/10.1145/3384613.3384648>.
- [10] Marco Baladi, Alessandro Barengi, Michele Battagliola, Sebastian Bitzer, Marco Gianvecchio, Patrick Karl, Felice Manganiello, Alessio Pavoni, Gerardo Pelosi, Federico Pintore, Paolo Santini, Jonas Schupp, Edoardo Signorini, Freeman Slaughter, Antonia Wachter-Zeh, and Violetta Weger. *CROSS crypto*. URL: <https://cross-crypto.com/>.

- [11] Marco Baldi, Alessandro Barenghi, Michele Battagliola, Sebastian Bitzer, Marco Gianvecchio, Patrick Karl, Felice Manganiello, Alessio Pavoni, Gerardo Pelosi, Paolo Santini, Jonas Schupp, Edoardo Signorini, Freeman Slaughter, Antonia Wachter-Zeh, and Violetta Weger. *CROSS: Codes and Restricted Objects Signature Scheme -Algorithm Specifications and Supporting Documentation*. Tech. rep. Jan. 2025. URL: <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/cross-spec-round2-web.pdf>.
- [12] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. *Status report on the first round of the additional digital signature schemes for the NIST post-quantum cryptography standardization process*. Tech. rep. NIST IR 8528. Gaithersburg, MD: National Institute of Standards and Technology (U.S.), Oct. 2024, NIST IR 8528. DOI: 10.6028/NIST.IR.8528. URL: <https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8528.pdf>.
- [13] Hanyu Wei, Wenqian Li, and Yunlei Zhao. “NeonCROSS: Vectorized Implementation of Post-Quantum Signature CROSS on Cortex-A72 and Apple M3”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2026.1 (Jan. 2026), pp. 250–272. DOI: 10.46586/tches.v2026.i1.250–272. URL: <https://tches.iacr.org/index.php/TCHES/article/view/12676>.
- [14] XKCP Team. *XKCP/XKCP*. July 2026. URL: <https://github.com/XKCP/XKCP>.
- [15] Guido Bertoni, Joan Daemen, Seth Hoeffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. *Software resources*. URL: <https://keccak.team/software.html>.
- [16] Mayo team. *MAYO*. URL: <https://pqmayo.org/>.
- [17] Matthias J Kannwischer, Ward Beullens, Fabio Campos, Sophia Celi, and Basil Hess. *MAYO Specification Document - Round 2*. Tech. rep. URL: <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/mayo-spec-round2-web.pdf>.
- [18] Yasufumi Hashimoto. “An improvement of algorithms to solve under-defined systems of multivariate quadratic equations”. In: *JSIAM Letters* 15 (2023), pp. 53–56. DOI: 10.14495/jsiaml.15.53.
- [19] Ryad Benadjila, Charles Bouillaguet, Thibauld Feneuil, and Matthieu Rivain. *MQOM: MQ on my Mind — Algorithm Specifications and Supporting Documentation (Version 2.0)*. Tech. rep. Feb. 2025. URL: <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/mqom-spec-round2-web.pdf>.
- [20] *MQOM*. URL: <https://mqom.org/index.html>.
- [21] Thibauld Feneuil. *Thibauld Feneuil’s site*. URL: <https://www.thibauld-feneuil.fr/>.
- [22] Ryad Benadjila, Charles Bouillaguet, Thibauld Feneuil, and Matthieu Rivain. *MQOM: MQ on my Mind - Version 2*. Mar. 2025. URL: https://www.thibauld-feneuil.fr/docs/talks/2025-03-13_PQ-TLS_mpcith-mqom2.pdf.
- [23] Hiroki Furue and Momonari Kudo. “Polynomial XL: A Variant of the XL Algorithm Using Macaulay Matrices over Polynomial Rings”. In: *Post-Quantum Cryptography*. Ed. by Markku-Juhani Saarinen and Daniel Smith-Tone. Cham: Springer Nature Switzerland, 2024, pp. 109–143. DOI: 10.1007/978-3-031-62746-0_6.
- [24] Matthieu Rivain. *Constructions for digital signatures Part III: Threshold-Computation-in-the-Head*. July 2024. URL: <https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/pqc-seminars/presentations/16-threshold-in-the-head-07022024.pdf>.
- [25] Ryad Benadjila, Charles Bouillaguet, Thibauld Feneuil, and Matthieu Rivain. *MQOM: MQ on my Mind — Algorithm Specifications and Supporting Documentation (Version 2.1)*. Tech. rep. Sept. 2025. URL: <https://mqom.org/docs/mqom-v2.1.pdf>.

- [26] SSE2NEON Contributors. *DLTcollab/sse2neon*. Aug. 2026. URL: <https://github.com/DLTcollab/sse2neon>.
- [27] *Intrinsics – Arm Developer*. URL: <https://developer.arm.com/architectures/instruction-sets/intrinsics/>.
- [28] *Intel® Intrinsics Guide*. Dec. 2024. URL: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>.
- [29] RBC Team. *rbc / rbc-lib · GitLab*. July 2024. URL: <http://gitlab.owndata.org/rbc/rbc-lib>.
- [30] *RBC: Rank-Based Cryptography Library*. URL: <https://rbc-lib.org/>.

Appendix A

Instructions

A.1 AES-NI instructions

Instruction	Intrinsic	Description
AESENC	<code>_mm_aesenc_si128</code>	Perform one round of an AES encryption flow
AESENCLAST	<code>_mm_aesenclast_si128</code>	Perform the last round of an AES encryption flow
AESDEC	<code>_mm_aesdec_si128</code>	Perform one round of an AES decryption flow
AESDECLAST	<code>_mm_aesdeclast_si128</code>	Perform the last round of an AES decryption flow
AESKEYGENASSIST	<code>_mm_aeskeygenassist_si128</code>	Assist in AES round key generation
AESIMC	<code>_mm_aesimc_si128</code>	Assist in AES decryption round key generation. Applies Inverse Mix Columns to round keys.

Table A.1: AES-NI instructions

A.2 SHA-NI instructions

Instruction	Intrinsic	Description
SHA256MSG1	<code>_mm_sha256msg1_epu32</code>	Message schedule update (part 1).
SHA256MSG2	<code>_mm_sha256msg2_epu32</code>	Message schedule update (part 2).
SHA256RND2	<code>_mm_sha256rnds2_epu32</code>	Performs two SHA-256 rounds.

Table A.2: SHA-256 (SHA-2) x86

A.3 AES ARM instructions

Instruction	Intrinsic	Description
AESE	vaeseq_u8	AES single round encryption
AESD	vaesdq_u8	AES single round decryption.
AESMC	vaesmcq_u8	AES mix columns
AESIMC	vaesimcq_u8	AES inverse mix columns

Table A.3: AES ARM instructions

A.4 SHA ARM instructions

Family	Instruction	Intrinsic	Description
SHA-2	SHA256H	vsha256hq_u32	Performs SHA-256 rounds (part 1).
SHA-2	SHA256H2	vsha256h2q_u32	Performs SHA-256 rounds (part 2).
SHA-2	SHA256SU0	vsha256su0q_u32	Message schedule update SHA-256 (part 1).
SHA-2	SHA256SU1	vsha256su1q_u32	Message schedule update SHA-256 (part 2).
SHA-2	SHA512H	vsha512hq_u64	Performs SHA-512 rounds (part 1).
SHA-2	SHA512H2	vsha512h2q_u64	Performs SHA-512 rounds (part 2).
SHA-2	SHA512SU0	vsha512su0q_u64	Message schedule update SHA-512 (part 1).
SHA-2	SHA512SU1	vsha512su1q_u64	Message schedule update SHA-512 (part 2).
SHA-3	BCAX	vbcaxq_u64	Bit Clear and Exclusive OR.
SHA-3	EOR3	veor3q_u64	Three-way Exclusive OR.
SHA-3	RAX1	vrax1q_u64	Rotate and Exclusive OR.
SHA-3	XAR	vxarq_u64	Exclusive OR and Rotate.

Table A.4: ARM SHA instructions

Appendix B

Mersenne prime

Reduction methods A Mersenne prime is a prime of the form $2^k - 1$. For a modulus p , modular reduction consists in computing

$$r = x \bmod p.$$

- **Mersenne reduction.** A pseudo-Mersenne prime has a similar form $2^k - c$, where c is a small integer. For

$$p = 2^k - 1,$$

we have

$$2^k \equiv 1 \pmod{p}.$$

Writing

$$x = x_0 + 2^k x_1,$$

gives

$$x \bmod p \equiv x_0 + x_1 \pmod{p}.$$

For CROSS, $127 = 2^7 - 1$.

- **Pseudo-Mersenne (PMP) reduction.** For

$$p = 2^k - c,$$

with small c ,

$$2^k \equiv c \pmod{p}.$$

Therefore,

$$x = x_0 + 2^k x_1 \implies x \bmod p \equiv x_0 + c x_1 \pmod{p}.$$

For CROSS, $509 = 2^9 - 3$, hence

$$x \bmod 509 \equiv x_0 + 3x_1 \pmod{509}.$$

- **Barrett reduction.** A precomputed approximation

$$\mu = \left\lfloor \frac{2^k}{p} \right\rfloor$$

is used to approximate the quotient:

$$q \approx \frac{x\mu}{2^k}, \quad r = x - qp,$$

followed by a correction to obtain $r \in \mathbb{F}_p$.

- **Montgomery reduction.** For $R = 2^k$ with $\gcd(R, p) = 1$, values are represented as

$$\tilde{x} = xR \bmod p.$$

Multiplication is then performed as

$$\text{MontMul}(\tilde{x}, \tilde{y}) = \tilde{x}\tilde{y}R^{-1} \bmod p.$$

- **PMP hybrid.** For $p = 509$, PMP reductions are first used to reduce the size of the intermediate value, followed by a Barrett-like reduction to obtain

$$0 \leq r < 509.$$

Appendix C

Measured values

C.1 Size

C.1.1 Private Key size in Bytes

	1	3	5
CROSS-R-SDP (fast, balanced and small)	32	48	64
CROSS-R-SDP(G) (fast, balanced and small)	32	48	64
MAYO-1	24		
MAYO-2	24		
MAYO-3		32	
MAYO-5			40
MQOM2.1-gf2-(short, fast)-3r/5r	72	108	144
MQOM2.1-gf16-(short, fast)-3r/5r	88	132	180
MQOM2.1-gf256-(short, fast)-3r/5r	128	192	256

Table C.1: Private Key size in Bytes

C.1.2 Public Key size in Bytes

	1	3	5
CROSS-R-SDP (fast, balanced and small)	77	115	153
CROSS-R-SDP(G) (fast, balanced and small)	54	83	106
MAYO-1	1 420		
MAYO-2	4 912		
MAYO-3		2 986	
MAYO-5			5 554
MQOM2.1-gf2-(short, fast)-3r/5r	52	78	104
MQOM2.1-gf16-(short, fast)-3r/5r	60	90	122
MQOM2.1-gf256-(short, fast)-3r/5r	80	120	160

Table C.2: Public Key size in Bytes

C.1.3 Signature size in Bytes

	1	3	5
CROSS-R-SDP fast	18 432	41 406	74 590
CROSS-R-SDP balanced	13 152	29 853	53 527
CROSS-R-SDP small	12 432	28 391	50 818
CROSS-R-SDP(G) fast	11 980	26 772	48 102
CROSS-R-SDP(G) balanced	9 120	22 464	40 100
CROSS-R-SDP(G) small	8 960	20 452	36 454
MAYO-1	454		
MAYO-2	186		
MAYO-3		681	
MAYO-5			964
MQOM2.1-gf2-short-3r	2 868	6 388	11 764
MQOM2.1-gf16-short-3r	3 060	6 820	12 664
MQOM2.1-gf256-short-3r	3 540	7 900	14 564
MQOM2.1-gf2-fast-3r	3 212	7 576	13 412
MQOM2.1-gf16-fast-3r	3 484	8 224	14 708
MQOM2.1-gf256-fast-3r	4 164	9 844	17 444
MQOM2.1-gf2-short-5r	2 820	6 280	11 564
MQOM2.1-gf16-short-5r	2 916	6 496	12 014
MQOM2.1-gf256-short-5r	3 156	7 036	12 964
MQOM2.1-gf2-fast-5r	3 144	7 414	13 124
MQOM2.1-gf16-fast-5r	3 280	7 738	13 772
MQOM2.1-gf256-fast-5r	3 620	8 548	15 140

Table C.3: Signature size in Bytes

C.2 Overall Benchmark

C.2.1 Benchmark key generation in kilocycles

	Reference				AVX2	
	1	3	5	1	3	5
CROSS-R-SDP fast	50	94	189	36	69	107
CROSS-R-SDP balanced	49	94	184	30	88	115
CROSS-R-SDP small	47	100	212	32	81	107
CROSS-R-SDP(G) fast	24	38	63	16	30	48
CROSS-R-SDP(G) balanced	22	44	63	15	33	65
CROSS-R-SDP(G) small	21	37	60	16	31	52
MAYO-1	1952	0	0	85	0	0
MAYO-2	1575	0	0	45	0	0
MAYO-3	0	5479	0	0	187	0
MAYO-5	0	0	11608	0	0	421
MQOM2.1-gf2-short-3r	16942	80508	140780	1741	6969	12535
MQOM2.1-gf16-short-3r	4160	17920	36808	483	1605	3293
MQOM2.1-gf256-short-3r	3813	17733	30709	502	1714	3161

MQOM2.1-gf2-fast-3r	17688	78608	133836	1844	6628	11838
MQOM2.1-gf16-fast-3r	4170	19249	39344	433	1791	3619
MQOM2.1-gf256-fast-3r	3556	17269	30578	443	1908	3729
MQOM2.1-gf2-short-5r	16368	81614	137320	1742	7058	12474
MQOM2.1-gf16-short-5r	4252	17921	36922	481	1694	3290
MQOM2.1-gf256-short-5r	3949	17498	30316	503	1723	3109
MQOM2.1-gf2-fast-5r	16849	78190	139395	1851	6534	11946
MQOM2.1-gf16-fast-5r	4118	19140	39251	429	1788	3562
MQOM2.1-gf256-fast-5r	3684	17778	30583	440	1912	3759

Table C.4: Key generation in kilocycles, Reference and AVX2 implementations

	ARM Ref			NEON		
	1	3	5	1	3	5
CROSS-R-SDP fast	8	19	31	6	13	23
CROSS-R-SDP balanced	8	19	32	6	13	23
CROSS-R-SDP small	8	19	32	6	13	23
CROSS-R-SDP(G) fast	5	11	19	2	5	9
CROSS-R-SDP(G) balanced	5	11	19	2	5	9
CROSS-R-SDP(G) small	5	11	19	2	5	9
MAYO-1	5899	0	0	314	0	0
MAYO-2	6013	0	0	271	0	0
MAYO-3	0	16848	0	0	846	0
MAYO-5	0	0	40948	0	0	1974
MQOM2.1-gf2-short-3r	273	1404	2445	47	225	390
MQOM2.1-gf16-short-3r	70	307	640	13	46	93
MQOM2.1-gf256-short-3r	60	295	516	12	48	87
MQOM2.1-gf2-fast-3r	267	1337	2406	39	178	363
MQOM2.1-gf16-fast-3r	74	319	676	13	46	93
MQOM2.1-gf256-fast-3r	61	296	527	12	46	85
MQOM2.1-gf2-short-5r	273	1403	2445	47	224	393
MQOM2.1-gf16-short-5r	69	309	640	13	46	93
MQOM2.1-gf256-short-5r	60	295	518	12	47	87
MQOM2.1-gf2-fast-5r	262	1336	2406	39	179	362
MQOM2.1-gf16-fast-5r	74	319	679	13	46	92
MQOM2.1-gf256-fast-5r	61	297	525	12	46	85

Table C.5: Key generation in kilocycles, ARM Ref and NEON implementations

C.2.2 Benchmark signing algorithm in kilocycles

	Reference			AVX2		
	1	3	5	1	3	5
CROSS-R-SDP fast	1383	3826	5654	853	2102	3651
CROSS-R-SDP balanced	2777	6702	9605	1502	3378	7021
CROSS-R-SDP small	5119	10010	15586	3697	5337	9642
CROSS-R-SDP(G) fast	971	2294	3858	556	1391	2348
CROSS-R-SDP(G) balanced	2018	3021	5006	1371	1865	3050

CROSS-R-SDP(G) small	3961	5675	9010	2640	4012	5810
MAYO-1	2999	0	0	206	0	0
MAYO-2	2029	0	0	100	0	0
MAYO-3	0	8595	0	0	563	0
MAYO-5	0	0	17807	0	0	1058
MQOM2.1-gf2-short-3r	130631	542015	982858	13999	66061	155716
MQOM2.1-gf16-short-3r	122260	469382	693899	8429	40207	75366
MQOM2.1-gf256-short-3r	147497	591873	876294	9497	49494	88804
MQOM2.1-gf2-fast-3r	55561	230172	479341	8433	35191	91469
MQOM2.1-gf16-fast-3r	29563	124496	214543	3813	14377	32624
MQOM2.1-gf256-fast-3r	36284	147287	243638	4578	18974	40233
MQOM2.1-gf2-short-5r	132660	542535	980138	13857	65628	155860
MQOM2.1-gf16-short-5r	106216	395344	608368	8025	36339	69281
MQOM2.1-gf256-short-5r	125158	461762	703913	8543	42349	76881
MQOM2.1-gf2-fast-5r	53041	230283	480969	8376	35045	90370
MQOM2.1-gf16-fast-5r	26606	112787	198609	3670	13727	31792
MQOM2.1-gf256-fast-5r	30465	124359	213614	4450	17369	37542

Table C.6: Signing algorithm in kilocycles, Reference and AVX2 implementations

	ARM Ref			NEON		
	1	3	5	1	3	5
CROSS-R-SDP fast	650	2078	4115	261	652	1218
CROSS-R-SDP balanced	1105	3396	6642	476	1115	2003
CROSS-R-SDP small	2233	5113	10780	959	1668	3240
CROSS-R-SDP(G) fast	279	748	1650	135	299	559
CROSS-R-SDP(G) balanced	530	947	2025	290	409	724
CROSS-R-SDP(G) small	1059	1802	3627	575	770	1290
MAYO-1	10548	0	0	846	0	0
MAYO-2	7560	0	0	548	0	0
MAYO-3	0	29742	0	0	2254	0
MAYO-5	0	0	71470	0	0	5055
MQOM2.1-gf2-short-3r	2488	10328	19913	605	2824	7331
MQOM2.1-gf16-short-3r	2134	8247	12858	288	1303	2566
MQOM2.1-gf256-short-3r	2718	10630	16099	333	1581	2963
MQOM2.1-gf2-fast-3r	1069	4942	11285	357	1700	4735
MQOM2.1-gf16-fast-3r	570	2307	4432	132	523	1304
MQOM2.1-gf256-fast-3r	684	2829	5031	157	663	1494
MQOM2.1-gf2-short-5r	2478	10361	19896	606	2832	7325
MQOM2.1-gf16-short-5r	1832	7111	11316	280	1219	2459
MQOM2.1-gf256-short-5r	2138	8316	13040	308	1403	2740
MQOM2.1-gf2-fast-5r	1074	4943	11350	361	1705	4797
MQOM2.1-gf16-fast-5r	520	2096	4179	130	505	1285
MQOM2.1-gf256-fast-5r	583	2427	4501	152	632	1470

Table C.7: Signing in kilocycles, ARM Ref and NEON implementations

C.2.3 Benchmark verification in kilocycles

	Reference			AVX2		
	1	3	5	1	3	5
CROSS-R-SDP fast	732	2083	3252	469	1196	2190
CROSS-R-SDP balanced	1463	3300	5245	975	2322	4298
CROSS-R-SDP small	2947	4672	8577	2379	3489	7388
CROSS-R-SDP(G) fast	605	1402	2488	354	921	1474
CROSS-R-SDP(G) balanced	1236	1818	3101	843	1293	2070
CROSS-R-SDP(G) small	2417	3450	5457	1671	2304	4082
MAYO-1	2012	0	0	102	0	0
MAYO-2	1316	0	0	27	0	0
MAYO-3	0	5047	0	0	218	0
MAYO-5	0	0	10490	0	0	484
MQOM2.1-gf2-short-3r	130025	531961	979862	13025	62149	149460
MQOM2.1-gf16-short-3r	118190	443872	670506	7572	35812	69639
MQOM2.1-gf256-short-3r	147338	589520	870357	8523	45309	83747
MQOM2.1-gf2-fast-3r	52594	225678	465811	7781	34473	85878
MQOM2.1-gf16-fast-3r	25534	115357	190234	3049	13660	30593
MQOM2.1-gf256-fast-3r	34849	144244	238474	3863	18134	37649
MQOM2.1-gf2-short-5r	129166	544122	968074	12887	61498	148247
MQOM2.1-gf16-short-5r	101668	379250	589733	7202	32243	63607
MQOM2.1-gf256-short-5r	121105	452938	698405	7680	38458	72586
MQOM2.1-gf2-fast-5r	50067	223245	472158	7776	34480	87336
MQOM2.1-gf16-fast-5r	22667	101548	174568	2947	12747	29237
MQOM2.1-gf256-fast-5r	28715	124524	209030	3704	16526	35573

Table C.8: Verification algorithm in kilocycles, Reference and AVX2 implementations

	ARM Ref			NEON		
	1	3	5	1	3	5
CROSS-R-SDP fast	331	1035	2042	148	366	679
CROSS-R-SDP balanced	338	894	1670	244	522	902
CROSS-R-SDP small	540	1066	2014	474	748	1365
CROSS-R-SDP(G) fast	188	523	1189	78	172	320
CROSS-R-SDP(G) balanced	335	631	1366	162	228	380
CROSS-R-SDP(G) small	658	1164	2340	322	425	638
MAYO-1	4800	0	0	378	0	0
MAYO-2	3257	0	0	135	0	0
MAYO-3	0	10837	0	0	1043	0
MAYO-5	0	0	23657	0	0	2184
MQOM2.1-gf2-short-3r	2417	10140	19693	577	2791	7495
MQOM2.1-gf16-short-3r	2040	7911	12411	242	1165	2433
MQOM2.1-gf256-short-3r	2659	10413	15902	288	1446	2849
MQOM2.1-gf2-fast-3r	1029	4783	11026	335	1656	4694
MQOM2.1-gf16-fast-3r	495	2111	3958	99	477	1205
MQOM2.1-gf256-fast-3r	654	2750	4884	125	611	1391
MQOM2.1-gf2-short-5r	2437	10160	19680	576	2779	7418
MQOM2.1-gf16-short-5r	1751	6820	10886	233	1075	2324
MQOM2.1-gf256-short-5r	2095	8150	12825	262	1268	2608

MQOM2.1-gf2-fast-5r	1014	4781	11071	336	1659	4718
MQOM2.1-gf16-fast-5r	447	1904	3693	97	455	1174
MQOM2.1-gf256-fast-5r	538	2322	4330	119	570	1348

Table C.9: Verification algorithm in kilocycles, ARM Ref and NEON implementations

C.3 Internal Benchmark

C.3.1 CROSS

Key generation

	Instance	row 1 and 8	declaration	row 2	row 3	row 5	row 7 (row 4 and 6 are implicit here)	row 9
Cat. I	CROSS-R-SDP fast	242	25	979	31117	1849	8201	100
	CROSS-R-SDP balanced	165	17	721	22092	1320	5844	88
	CROSS-R-SDP small	186	20	782	24309	1418	6433	75
	CROSS-R-SDP(G) fast	184	19	769	10875	1043	2660	30
	CROSS-R-SDP(G) balanced	188	17	724	10499	978	2643	25
	CROSS-R-SDP(G) small	174	17	729	10248	960	2607	28
Cat. III	CROSS-R-SDP fast	419	24	1139	75181	2339	15366	147
	CROSS-R-SDP balanced	305	19	815	56284	1758	12213	107
	CROSS-R-SDP small	449	28	1254	83679	2571	17903	154
	CROSS-R-SDP(G) fast	409	22	1109	30745	1753	6151	60
	CROSS-R-SDP(G) balanced	398	23	1046	29039	1632	5809	48
	CROSS-R-SDP(G) small	403	23	1103	31340	1756	6137	63
Cat. V	CROSS-R-SDP fast	493	23	1124	127131	3343	3043	166
	CROSS-R-SDP balanced	514	22	1169	131489	3466	3091	173
	CROSS-R-SDP small	510	22	1174	130131	3422	2985	169
	CROSS-R-SDP(G) fast	517	26	1167	52035	1869	7620	75
	CROSS-R-SDP(G) balanced	704	21	1586	73638	2650	9103	99
	CROSS-R-SDP(G) small	569	24	1269	57714	2171	8353	86

Table C.10: Internal Cross Key generation, AVX2 implementation

	Instance	row 1 and 8	declaration	row 2	row 3	row 5	row 7 (row 4 and 6 are implicit here)	row 9
Cat. I	CROSS-R-SDP fast	25	1	115	4070	178	1266	28
	CROSS-R-SDP balanced	25	1	115	4086	178	1274	27
	CROSS-R-SDP small	25	1	118	4116	181	1293	25
	CROSS-R-SDP(G) fast	25	1	117	1820	294	245	13
	CROSS-R-SDP(G) balanced	26	1	116	1805	293	243	13
	CROSS-R-SDP(G) small	25	1	116	1807	295	244	13
Cat. III	CROSS-R-SDP fast	41	1	126	9558	217	3220	37
	CROSS-R-SDP balanced	42	1	126	9524	217	3220	38
	CROSS-R-SDP small	41	1	126	9674	217	3221	39
	CROSS-R-SDP(G) fast	41	1	126	3944	502	500	20
	CROSS-R-SDP(G) balanced	41	1	126	3931	501	490	20
	CROSS-R-SDP(G) small	40	1	126	3980	499	490	20
Cat. V	CROSS-R-SDP fast	53	1	138	17071	334	5603	45
	CROSS-R-SDP balanced	53	1	138	17295	335	5614	44
	CROSS-R-SDP small	54	1	137	17305	336	5610	44
	CROSS-R-SDP(G) fast	54	1	137	6916	885	1057	22
	CROSS-R-SDP(G) balanced	54	1	137	6925	882	1054	22
	CROSS-R-SDP(G) small	53	1	137	6917	882	1052	22

Table C.11: Internal Cross Key generation, CrossNEON implementation

Signing algorithm

	Instance	row 28 (preparation)	row 1	avx2 preparation	row 2	row 3	declaration	row 5 and 10 preparation	declaration 2	row 4 loop	row 11	row 12
Cat. I	CROSS-R-SDP fast	276	28133	544	314	14637	19	59	212	622961	30179	27760
	CROSS-R-SDP balanced	146	23521	481	264	160730	19	50	403	825924	167845	37344
	CROSS-R-SDP small	214	36942	842	437	515672	38	42	1356	2605898	537072	117280
	CROSS-R-SDP(G) fast	145	14427	390	299	14541	19	24	179	358196	28387	25828
	CROSS-R-SDP(G) balanced	118	12303	330	245	164403	20	45	326	524441	172370	38389
	CROSS-R-SDP(G) small	125	15260	482	337	405237	19	62	1048	1281818	422255	92843
Cat. III	CROSS-R-SDP fast	866	76933	1314	683	42750	22	42	683	1576427	86673	82777
	CROSS-R-SDP balanced	648	79334	1511	674	389934	38	46	1487	2607482	412893	140197
	CROSS-R-SDP small	642	79901	1565	692	594808	23	48	2497	3942126	628165	211943
	CROSS-R-SDP(G) fast	478	31245	708	539	36800	32	33	552	871409	73355	70854
	CROSS-R-SDP(G) balanced	411	31065	831	552	236511	20	45	813	1039838	250068	85172
	CROSS-R-SDP(G) small	437	37421	1055	679	542084	22	73	2075	2364487	576035	193968
Cat. V	CROSS-R-SDP fast	1884	140255	2986	846	78088	35	31	1392	2981483	162790	158532
	CROSS-R-SDP balanced	1364	140432	3197	832	568564	22	77	3057	4713265	608272	248902
	CROSS-R-SDP small	1276	145110	3208	909	943545	22	100	5327	7751939	1009594	404310
	CROSS-R-SDP(G) fast	1064	60124	1250	794	69329	23	37	1224	1640874	149205	144969
	CROSS-R-SDP(G) balanced	900	58195	1335	818	380869	22	61	1773	1882430	410994	168612
	CROSS-R-SDP(G) small	862	63291	1546	942	738527	22	86	3745	3644632	795603	323969

Table C.12: Internal CROSS signing algorithm, AVX2 implementation – Part 1

	Instance	row 13	row 14	row 15	row 16	row 17 loop	row 21	row 22	row 23	row 24	row 25 loop
Cat. I	CROSS-R-SDP fast	858	762	889	2271	9096	111578	5831	1343	1350	15857
	CROSS-R-SDP balanced	724	661	752	2272	12987	150184	6965	3028	4812	8094
	CROSS-R-SDP small	1156	1006	1210	6547	36236	474961	21996	8355	10923	12661
	CROSS-R-SDP(G) fast	819	725	886	2245	57642	56687	5454	1257	1182	6022
	CROSS-R-SDP(G) balanced	759	611	784	2517	87815	84313	6909	2906	4668	3145
	CROSS-R-SDP(G) small	921	758	978	5705	205183	205292	17120	5335	8481	4160
Cat. III	CROSS-R-SDP fast	1165	978	1897	3225	15517	322794	10517	2354	2299	34918
	CROSS-R-SDP balanced	1169	983	2035	4749	26704	546024	18075	8780	10241	24513
	CROSS-R-SDP small	1240	988	2073	7434	39748	832819	28148	10917	14261	23842
	CROSS-R-SDP(G) fast	977	796	1688	3478	132173	146912	8783	2129	1903	13967
	CROSS-R-SDP(G) balanced	997	887	1696	3689	156779	177248	10729	6292	7439	10289
	CROSS-R-SDP(G) small	1228	1017	2089	7392	344117	403894	22798	8226	11612	10752
Cat. V	CROSS-R-SDP fast	1353	1042	2170	4627	26149	622598	16668	3328	3157	71881
	CROSS-R-SDP balanced	1340	1067	2173	6437	42051	982881	24266	9745	13795	47651
	CROSS-R-SDP small	1334	1113	2301	10763	70428	1605674	48472	16717	19970	45630
	CROSS-R-SDP(G) fast	1244	1024	2114	5315	214121	305474	15235	3144	2886	28338
	CROSS-R-SDP(G) balanced	1278	967	2069	5524	252489	354408	17464	9641	11012	20405
	CROSS-R-SDP(G) small	1354	1089	2197	9683	465800	682895	32295	13162	16185	19533

Table C.13: Internal CROSS signing algorithm, AVX2 implementation – Part 2

	Instance	row 28 (preparation)	row 1	avx2 preparation	row 2	row 3	declaration	row 5 and 10 preparation	declaration 2	row 4 loop	row 11	row 12
Cat. I	CROSS-R-SDP fast	13	4265	1038	37	2280	46	3	8	201746	4287	3787
	CROSS-R-SDP balanced	9	4288	1041	39	35419	80	3	14	331078	35545	6173
	CROSS-R-SDP small	9	4310	1046	42	71959	188	4	30	669696	72294	12667
	CROSS-R-SDP(G) fast	9	2224	356	39	2335	31	3	9	103653	4171	3718
	CROSS-R-SDP(G) balanced	6	2207	349	40	35562	52	3	18	180142	35801	6243
	CROSS-R-SDP(G) small	6	2217	362	44	71608	120	4	34	359071	71879	12512
Cat. III	CROSS-R-SDP fast	30	9776	2324	64	5937	112	3	19	514486	10945	10015
	CROSS-R-SDP balanced	21	9807	2323	64	53565	165	3	30	827264	53846	15873
	CROSS-R-SDP small	20	9786	2327	63	81479	336	3	49	1244619	81962	24068
	CROSS-R-SDP(G) fast	19	4510	744	64	5412	59	2	16	221180	10675	9362
	CROSS-R-SDP(G) balanced	16	4514	747	63	37358	68	3	22	265122	37674	11099
	CROSS-R-SDP(G) small	14	4512	747	66	71694	130	2	41	505594	72085	21221
Cat. V	CROSS-R-SDP fast	58	17381	4097	84	10396	200	2	33	968964	19190	17796
	CROSS-R-SDP balanced	39	17398	4092	84	72300	408	3	61	1545618	73072	28339
	CROSS-R-SDP small	38	17770	4093	95	117548	804	5	124	2511975	118804	46141
	CROSS-R-SDP(G) fast	35	7670	1322	84	9644	106	2	29	422539	19031	16664
	CROSS-R-SDP(G) balanced	28	7684	1324	84	50337	124	2	36	500874	50703	19708
	CROSS-R-SDP(G) small	26	7706	1326	85	90922	280	3	66	903683	91989	35655

Table C.14: Internal CROSS signing algorithm, CrossNEON implementation – Part 1

	Instance	row 13	row 14	row 15	row 16	row 17 loop	row 21	row 22	row 23	row 24	row 25 loop
Cat. I	CROSS-R-SDP fast	115	99	129	290	15437	20405	749	81	70	5600
	CROSS-R-SDP balanced	116	101	134	358	25347	33671	1092	323	533	2878
	CROSS-R-SDP small	116	102	134	685	51750	69861	2260	558	905	2375
	CROSS-R-SDP(G) fast	119	103	137	310	5915	10673	733	75	68	2556
	CROSS-R-SDP(G) balanced	118	103	135	394	10383	18503	1116	384	510	1355
	CROSS-R-SDP(G) small	118	102	137	749	20893	37765	2089	574	985	1117
Cat. III	CROSS-R-SDP fast	129	104	233	358	34511	50678	1152	134	120	13046
	CROSS-R-SDP balanced	128	104	229	529	55440	82101	1918	493	785	6943
	CROSS-R-SDP small	128	104	231	816	83739	125874	2851	698	1088	5914
	CROSS-R-SDP(G) fast	130	104	233	448	12885	25153	1111	124	117	5912
	CROSS-R-SDP(G) balanced	127	103	230	482	15450	30222	1336	464	577	3873
	CROSS-R-SDP(G) small	130	103	229	817	29573	58334	2328	597	1015	2736
Cat. V	CROSS-R-SDP fast	140	107	246	498	62274	92200	1740	171	165	23150
	CROSS-R-SDP balanced	139	107	247	698	99191	148924	2363	657	1033	12584
	CROSS-R-SDP small	141	109	251	1141	161432	245078	4052	956	1626	10345
	CROSS-R-SDP(G) fast	142	106	247	590	23307	45655	1632	164	146	11040
	CROSS-R-SDP(G) balanced	140	105	245	632	27759	54318	1904	509	754	6831
	CROSS-R-SDP(G) small	141	105	248	1088	50300	99158	3237	735	1199	4891

Table C.15: Internal CROSS signing algorithm, CrossNEON implementation – Part 2

Verification algorithm

	Instance	row 1	preparation avx2	unpack syndrome	row 4	row 5	row 6	row 7	row 8	declaration
Cat. I	CROSS-R-SDP fast	24643	559	110	706	866	1937	4090	442	224
	CROSS-R-SDP balanced	21316	467	99	612	730	2061	5626	88509	363
	CROSS-R-SDP small	32094	779	163	950	1116	6104	20333	367011	1337
	CROSS-R-SDP(G) fast	12003	393	124	704	878	2091	3945	400	183
	CROSS-R-SDP(G) balanced	10567	417	111	626	754	2231	5344	92352	449
	CROSS-R-SDP(G) small	11296	436	117	667	829	4574	11492	275891	843
Cat. III	CROSS-R-SDP fast	74833	1269	219	972	1981	3083	9149	1163	777
	CROSS-R-SDP balanced	76661	1418	254	1023	2017	4616	17595	206561	1478
	CROSS-R-SDP small	72676	1387	259	985	1970	6948	25430	381591	2307
	CROSS-R-SDP(G) fast	25783	702	141	732	1561	3140	6324	670	486
	CROSS-R-SDP(G) balanced	29013	879	175	848	1786	3717	8633	85935	791
	CROSS-R-SDP(G) small	32190	957	224	925	1939	6773	17553	344891	1988
Cat. V	CROSS-R-SDP fast	127581	2595	273	984	2044	4260	13727	2593	1446
	CROSS-R-SDP balanced	132471	2880	290	1059	2168	6216	20763	296734	2908
	CROSS-R-SDP small	149153	3098	315	1134	2288	10708	38414	672254	5323
	CROSS-R-SDP(G) fast	53234	1218	208	980	2006	4848	11713	1191	1291
	CROSS-R-SDP(G) balanced	58924	1306	251	994	2207	5597	15044	155402	1650
	CROSS-R-SDP(G) small	57468	1571	224	986	2166	9764	26923	466265	3696

Table C.16: Internal CROSS verification algorithm, AVX2 implementation – Part 1

	Instance	row 9 loop	row 22	row 23	row 24	row 25	row 26
Cat. I	CROSS-R-SDP fast	297321	29063	26295	794	106806	68
	CROSS-R-SDP balanced	533710	82422	37204	689	149268	103
	CROSS-R-SDP small	1703403	140684	108790	1014	440547	136
	CROSS-R-SDP(G) fast	225266	28050	25238	824	55016	92
	CROSS-R-SDP(G) balanced	397760	86201	38509	708	85727	100
	CROSS-R-SDP(G) small	881545	100482	82192	764	182780	83
Cat. III	CROSS-R-SDP fast	851083	89637	84678	1117	331487	156
	CROSS-R-SDP balanced	1735980	227812	141706	1168	556107	114
	CROSS-R-SDP small	2602434	229858	201981	1160	794382	176
	CROSS-R-SDP(G) fast	523170	67987	65392	896	136132	66
	CROSS-R-SDP(G) balanced	775718	178872	87004	1019	182270	139
	CROSS-R-SDP(G) small	1764166	191902	180484	1163	377473	167
Cat. V	CROSS-R-SDP fast	1508896	154145	148866	1229	584204	124
	CROSS-R-SDP balanced	2979150	307226	242432	1444	956365	169
	CROSS-R-SDP small	5358381	350327	410129	1330	1630339	207
	CROSS-R-SDP(G) fast	960197	144739	139525	1180	293220	132
	CROSS-R-SDP(G) balanced	1377224	290427	178442	1298	376707	179
	CROSS-R-SDP(G) small	2584058	299066	313217	1311	659713	139

Table C.17: Internal CROSS verification algorithm, AVX2 implementation – Part 2

	Instance	row 1	preparation avx2	unpack syndrome	row 4	row 5	row 6	row 7	row 8	declaration
Cat. I	CROSS-R-SDP fast	4069	1038	22	99	127	293	746	42	10
	CROSS-R-SDP balanced	4012	1048	22	100	127	356	1083	19184	19
	CROSS-R-SDP small	4002	1037	21	99	128	677	2308	53943	28
	CROSS-R-SDP(G) fast	1852	357	11	100	127	305	707	39	12
	CROSS-R-SDP(G) balanced	1824	356	12	99	128	386	1085	19462	19
	CROSS-R-SDP(G) small	1807	360	14	99	127	740	2044	54233	38
Cat. III	CROSS-R-SDP fast	9442	2327	34	101	223	355	1134	83	24
	CROSS-R-SDP balanced	9463	2328	31	101	223	526	1950	26812	38
	CROSS-R-SDP small	9448	2321	30	103	226	810	2842	53500	55
	CROSS-R-SDP(G) fast	3964	742	17	101	221	444	1121	89	20
	CROSS-R-SDP(G) balanced	3934	745	20	101	224	477	1295	13029	23
	CROSS-R-SDP(G) small	3970	745	16	101	224	816	2285	48077	48
Cat. V	CROSS-R-SDP fast	16914	4075	39	106	240	502	1710	152	36
	CROSS-R-SDP balanced	16910	4080	39	107	242	692	2291	37601	57
	CROSS-R-SDP small	17325	4099	41	107	239	1137	3992	81915	108
	CROSS-R-SDP(G) fast	6881	1322	21	105	238	590	1601	113	37
	CROSS-R-SDP(G) balanced	6921	1321	23	105	240	629	1932	18888	37
	CROSS-R-SDP(G) small	6881	1326	20	105	240	1088	3267	58811	69

Table C.18: Internal CROSS verification algorithm, CrossNEON implementation – Part 1

	Instance	row 9 loop	row 22	row 23	row 24	row 25	row 26
Cat. I	CROSS-R-SDP fast	112350	4277	3812	118	21266	1
	CROSS-R-SDP balanced	161655	17540	6186	117	34579	4
	CROSS-R-SDP small	308684	19415	12545	116	70299	3
	CROSS-R-SDP(G) fast	57845	4257	3800	120	10987	3
	CROSS-R-SDP(G) balanced	98903	17044	6227	118	18681	4
	CROSS-R-SDP(G) small	198255	19150	12498	117	37701	7
Cat. III	CROSS-R-SDP fast	279916	11052	10034	129	52041	2
	CROSS-R-SDP balanced	352183	28558	15910	129	83287	3
	CROSS-R-SDP small	498404	30155	24048	131	127055	3
	CROSS-R-SDP(G) fast	119739	10738	9359	129	25820	1
	CROSS-R-SDP(G) balanced	139976	25762	11106	130	30896	2
	CROSS-R-SDP(G) small	262568	25761	21194	129	58842	3
Cat. V	CROSS-R-SDP fast	524831	19220	17842	143	94670	2
	CROSS-R-SDP balanced	625335	37608	28248	142	150049	5
	CROSS-R-SDP small	924989	39718	46079	142	245155	4
	CROSS-R-SDP(G) fast	226461	19130	16671	142	46731	2
	CROSS-R-SDP(G) balanced	242020	33636	19646	142	54757	2
	CROSS-R-SDP(G) small	395999	35389	35563	140	99881	3

Table C.19: Internal CROSS verification algorithm, CrossNEON implementation – Part 2

C.3.2 MAYO

Key generation

Instance	declaration	row 2	row 5	row 6	row 7	row 9-12	row 14-16	row 18-20	cleanup
MAYO-1	47	705	1969	17	71	58826	27284	263	299
MAYO-2	96	561	2654	16	49	17751	22942	591	703
MAYO-3	73	616	3017	16	79	128724	59057	459	1216
MAYO-5	101	858	3611	16	87	243104	155903	950	2582

Table C.20: Internal MAYO Key generation, AVX2 implementation

Instance	declaration	row 2	row 5	row 6	row 7	row 9-12	row 14-16	row 18-20	cleanup
MAYO-1	1	25	89	1	1	3987	2847	12	22
MAYO-2	6	24	141	1	2	2052	3827	39	35
MAYO-3	3	27	143	1	2	9414	9395	26	54
MAYO-5	7	43	199	1	4	19017	25171	53	104

Table C.21: Internal MAYO Key generation, NEON implementation

Signing algorithm

Instance	declaration	expand sk	row 8	declaration 2	row 9	row 10	row 11	row 14 loop	row 43-45	row 46	cleanup
MAYO-1	16	96971	699	115	610	640	595	87734	5234	64	15
MAYO-2	16	54851	833	55	667	709	698	50004	2720	36	16
MAYO-3	16	231870	791	311	736	690	654	176274	39534	96	15
MAYO-5	21	699179	1118	698	1282	1481	773	401051	76714	189	18

Table C.22: Internal MAYO signing algorithm, AVX2 implementation

Instance	declaration	expand sk	row 8	declaration 2	row 9	row 10	row 11	row 14 loop	row 43-45	row 46	cleanup
MAYO-1	1	8317	35	6	27	33	34	9563	951	2	118
MAYO-2	1	6878	35	2	26	33	33	4388	810	1	87
MAYO-3	1	25609	37	12	35	34	34	22926	1790	3	411
MAYO-5	1	64070	43	27	51	64	34	45748	3032	6	855

Table C.23: Internal MAYO signing algorithm, NEON implementation

Verification algorithm

Instance	declarations and decode	expand pk	row 1-7	row 16	row 17	row 11-13	row 20-26	row 29-31
MAYO-1	2766	54301	16	694	613	63	35225	45
MAYO-2	1899	17354	16	685	587	28	6247	19
MAYO-3	9848	127522	16	827	654	112	75724	74
MAYO-5	29654	260208	17	936	726	184	204952	47

Table C.24: Internal MAYO verification algorithm, AVX2 implementation

Instance	declarations and decode	expand pk	row 1-7	row 16	row 17	row 11-13	row 20-26	row 29-31
MAYO-1	108	3827	1	35	34	3	4539	1
MAYO-2	82	1992	1	34	34	1	907	1
MAYO-3	300	9318	1	36	35	5	13725	1
MAYO-5	792	18786	1	39	36	7	29606	1

Table C.25: Internal MAYO verification algorithm, NEON implementation

C.3.3 MQOMv2.1

Key generation

	Instance	declarations	allocation	sanity checks	row 1-2	row 3	row 4 (loop)	row 7	row 8	cleanup
Cat. I	mqom2_gf2_short_r3	30	2000	25	1377	1221413	73701	61	26	1166
	mqom2_gf16_short_r3	25	1150	22	1177	245903	82578	25	26	12654
	mqom2_gf256_short_r3	25	1546	25	1303	339502	62892	55	27	15048
	mqom2_gf2_fast_r3	24	11746	26	1342	1678716	66933	57	25	79423
	mqom2_gf16_fast_r3	21	851	21	932	152438	99957	24	22	5830
	mqom2_gf256_fast_r3	25	1408	26	1197	248036	94434	37	28	8901
	mqom2_gf2_short_r5	25	2025	25	1368	1222407	74960	57	25	1223
	mqom2_gf16_short_r5	24	1211	24	1204	274522	90442	62	32	15159
	mqom2_gf256_short_r5	28	1407	25	1234	342230	63273	31	29	14947
	mqom2_gf2_fast_r5	25	11619	24	1360	1696672	68547	41	26	81601
	mqom2_gf16_fast_r5	32	1018	23	1035	175034	110182	39	26	7265
	mqom2_gf256_fast_r5	25	1436	25	1211	256176	96694	34	32	9317
Cat. III	mqom2_cat3_gf2_short_r3	21	1902	21	1249	4249699	228796	58	29	1258
	mqom2_cat3_gf16_short_r3	24	1740	24	1346	890014	260888	38	40	1258
	mqom2_cat3_gf256_short_r3	21	1420	21	1112	880196	143263	24	23	1111
	mqom2_cat3_gf2_fast_r3	22	2026	22	1230	4218409	190919	47	28	1377
	mqom2_cat3_gf16_fast_r3	24	1841	25	1402	1005255	317097	40	58	1327
	mqom2_cat3_gf256_fast_r3	22	1706	25	1165	1031894	217164	52	25	1241
	mqom2_cat3_gf2_short_r5	21	1979	21	1300	4311849	232628	64	34	1323
	mqom2_cat3_gf16_short_r5	25	10833	26	1387	1101798	265947	44	33	40488
	mqom2_cat3_gf256_short_r5	25	2076	23	1261	962119	153833	25	24	1194
	mqom2_cat3_gf2_fast_r5	22	2009	24	1285	4160737	186492	43	22	1327
	mqom2_cat3_gf16_fast_r5	25	1933	25	1368	1005177	317092	70	31	1301
	mqom2_cat3_gf256_fast_r5	22	1596	22	1210	1020802	212284	78	27	1220
Cat. V	mqom2_cat5_gf2_short_r3	20	1976	20	1166	7104391	524349	83	34	1443
	mqom2_cat5_gf16_short_r3	22	1821	24	1285	1532133	564522	57	39	1343
	mqom2_cat5_gf256_short_r3	22	1792	27	2091	1744023	344522	29	55	1304
	mqom2_cat5_gf2_fast_r3	22	2265	22	1314	7467320	453984	52	25	1530
	mqom2_cat5_gf16_fast_r3	22	1897	23	1348	1734719	616991	71	34	1334
	mqom2_cat5_gf256_fast_r3	23	2142	23	2143	2197695	465892	72	26	1462
	mqom2_cat5_gf2_short_r5	21	2011	21	1173	7137541	534032	49	29	1399
	mqom2_cat5_gf16_short_r5	22	1907	23	1340	1591276	583306	55	28	1397
	mqom2_cat5_gf256_short_r5	23	2016	23	2116	1864206	363898	55	46	1431
	mqom2_cat5_gf2_fast_r5	25	2266	22	1286	7540807	454781	48	35	1726
	mqom2_cat5_gf16_fast_r5	22	1845	23	1337	1783192	637862	78	30	1435
	mqom2_cat5_gf256_fast_r5	23	2113	23	2134	2244011	477674	42	33	1545

Table C.26: Internal MQOMv2.1 Key generation, AVX2 implementation

	Instance	declarations	allocation	sanity checks	row 1-2	row 3	row 4 (loop)	row 7	row 8	cleanup
Cat. I	mqom2_cat1_gf2_short_r3	1	37	1	70	44343	2305	6	1	42
	mqom2_cat1_gf16_short_r3	1	27	1	56	7220	5066	1	1	169
	mqom2_cat1_gf256_short_r3	1	31	1	63	7878	4155	1	1	181
	mqom2_cat1_gf2_fast_r3	1	149	1	56	36952	1202	2	1	885
	mqom2_cat1_gf16_fast_r3	1	16	1	53	6070	6578	1	1	129
	mqom2_cat1_gf256_fast_r3	1	15	1	53	7015	4488	1	1	126
	mqom2_cat1_gf2_short_r5	1	34	1	72	44546	2304	1	1	39
	mqom2_cat1_gf16_short_r5	1	22	1	53	7311	4935	1	1	180
	mqom2_cat1_gf256_short_r5	1	20	1	54	7884	4107	1	1	170
	mqom2_cat1_gf2_fast_r5	1	156	1	55	36930	1208	1	1	894
	mqom2_cat1_gf16_fast_r5	1	16	1	53	6063	6562	1	1	126
	mqom2_cat1_gf256_fast_r5	1	17	1	53	7027	4458	1	1	134
Cat. III	mqom2_cat3_gf2_short_r3	1	45	1	73	212063	12328	1	1	44
	mqom2_cat3_gf16_short_r3	1	36	1	68	30530	15039	7	1	42
	mqom2_cat3_gf256_short_r3	1	36	1	74	34513	12786	7	1	41
	mqom2_cat3_gf2_fast_r3	1	45	1	69	169186	5268	6	1	35
	mqom2_cat3_gf16_fast_r3	1	19	1	54	26865	18550	1	1	18
	mqom2_cat3_gf256_fast_r3	1	20	1	54	31605	13949	1	1	21
	mqom2_cat3_gf2_short_r5	1	50	1	69	212174	12676	5	1	46
	mqom2_cat3_gf16_short_r5	1	38	1	65	30388	15316	1	1	36
	mqom2_cat3_gf256_short_r5	1	34	1	73	34561	12692	7	1	39
	mqom2_cat3_gf2_fast_r5	1	45	1	66	169590	4745	6	1	47
	mqom2_cat3_gf16_fast_r5	1	20	1	55	26975	18458	1	1	16
	mqom2_cat3_gf256_fast_r5	1	17	1	54	31652	13948	1	1	16
Cat. V	mqom2_cat5_gf2_short_r3	1	56	1	68	355688	31792	7	1	55
	mqom2_cat5_gf16_short_r3	1	37	1	74	55393	37725	1	1	31
	mqom2_cat5_gf256_short_r3	1	36	1	115	57875	28794	7	1	36
	mqom2_cat5_gf2_fast_r3	1	50	1	74	331953	28203	1	1	50
	mqom2_cat5_gf16_fast_r3	2	26	1	70	49750	42661	2	1	29
	mqom2_cat5_gf256_fast_r3	1	20	1	102	54581	30100	1	1	22
	mqom2_cat5_gf2_short_r5	1	56	1	69	352638	31547	7	1	51
	mqom2_cat5_gf16_short_r5	1	37	1	73	55025	36781	1	1	26
	mqom2_cat5_gf256_short_r5	1	37	1	125	57693	28936	1	6	40
	mqom2_cat5_gf2_fast_r5	1	54	1	69	333906	28416	4	1	41
	mqom2_cat5_gf16_fast_r5	1	20	1	54	49731	42632	1	1	19
	mqom2_cat5_gf256_fast_r5	1	33	1	111	54405	30045	4	1	31

Table C.27: Internal MQOMv2.1 Key generation, NEON implementation

Signing algorithm

	Instance	declarations	row 1-2	row 5	signature preparation	row 6	row 7	row 8	row 8 continued	row 9	row 10	row 11	return success
Cat. I	mqom2_cat1_gf2_short_r3	25	26	1179	25	4449557	5822265	2655	2706	1359	131462	3010	0
	mqom2_cat1_gf16_short_r3	31	24	1101	54	4083909	1251788	2217	2623	1115	107800	2044	0
	mqom2_cat1_gf256_short_r3	25	31	1229	28	5351926	1775283	3436	3891	1989	127257	2646	0
	mqom2_cat1_gf2_fast_r3	27	27	1349	25	926275	5559233	3296	3519	1333	260469	2812	0
	mqom2_cat1_gf16_fast_r3	21	23	904	35	719411	1206826	2162	3207	1057	194038	1213	0
	mqom2_cat1_gf256_fast_r3	24	28	1161	66	1000355	2104742	4490	5626	1993	246957	2151	0
	mqom2_cat1_gf2_short_r5	25	30	1281	52	4383640	5887477	2332	2553	1392	132215	2920	0
	mqom2_cat1_gf16_short_r5	24	31	1122	62	4382978	1404424	1711	2188	1243	118866	2227	0
	mqom2_cat1_gf256_short_r5	28	28	1169	26	4589398	1805922	1809	2332	2050	129155	2438	0
	mqom2_cat1_gf2_fast_r5	25	28	1383	62	901303	5613688	2548	3544	1367	263725	2905	0
	mqom2_cat1_gf16_fast_r5	22	25	1011	23	775294	1382803	1792	2880	1055	218265	1596	0
	mqom2_cat1_gf256_fast_r5	24	27	1181	47	904097	2169727	2222	3329	1998	258733	2175	0
Cat. III	mqom2_cat3_gf2_short_r3	21	22	1204	37	15779917	22410250	4387	4935	1841	1641010	7084	0
	mqom2_cat3_gf16_short_r3	24	26	1198	34	21351268	5561658	6025	6925	2076	1971743	6244	0
	mqom2_cat3_gf256_short_r3	20	26	1029	28	21710125	5780281	7410	8401	1660	1599239	5425	0
	mqom2_cat3_gf2_fast_r3	22	23	1244	23	3390791	19374016	6037	6608	1914	11914	6470	0
	mqom2_cat3_gf16_fast_r3	24	87	1296	51	4345443	6254557	8682	8779	2150	12366	6213	0
	mqom2_cat3_gf256_fast_r3	24	27	1219	60	4785403	7530934	12015	12026	1853	10378	5870	0
	mqom2_cat3_gf2_short_r5	21	75	1196	25	15745067	22834333	4120	3975	1833	1683092	7634	0
	mqom2_cat3_gf16_short_r5	27	31	1318	48	18937419	5993065	4351	4340	2122	2015685	6893	0
	mqom2_cat3_gf256_short_r5	21	25	1154	34	19000013	6286775	3606	3902	1802	1743359	5793	0
	mqom2_cat3_gf2_fast_r5	22	24	1271	39	3290395	19284179	5141	5658	1950	11831	6670	0
	mqom2_cat3_gf16_fast_r5	24	27	1395	25	3803459	6310231	5525	6254	2079	12433	5778	0
	mqom2_cat3_gf256_fast_r5	23	26	1157	22	3768367	7258004	4526	5283	1867	10151	5458	0
Cat. V	mqom2_cat5_gf2_short_r3	20	21	1422	27	24897323	63856622	7131	6984	2377	33928	6951	0
	mqom2_cat5_gf16_short_r3	22	25	1276	27	31937047	15424942	9599	10384	2579	36063	8435	0
	mqom2_cat5_gf256_short_r3	22	25	1321	26	39288763	17620341	14920	15115	2684	37144	7840	0
	mqom2_cat5_gf2_fast_r3	23	23	1471	22	5215913	56084843	9900	10461	2651	16087	9115	0
	mqom2_cat5_gf16_fast_r3	22	26	1478	23	6068983	15082902	13871	13785	2713	15306	9779	0
	mqom2_cat5_gf256_fast_r3	23	29	1359	34	7775823	19971582	21370	21035	2766	17156	9404	0
	mqom2_cat5_gf2_short_r5	20	21	1371	21	25315324	64298626	5841	6174	2468	33683	7767	0
	mqom2_cat5_gf16_short_r5	24	26	1300	30	29281540	16015280	6016	6694	2705	37079	8048	0
	mqom2_cat5_gf256_short_r5	23	28	1316	31	34003695	18971332	6089	6719	2796	39494	10586	0
	mqom2_cat5_gf2_fast_r5	22	24	1432	24	5258690	56068791	8858	8213	2693	16072	9572	0
	mqom2_cat5_gf16_fast_r5	22	25	1448	25	5566778	15667232	8368	8223	2729	15676	9728	0
	mqom2_cat5_gf256_fast_r5	23	29	1353	38	6427596	19849023	8562	9123	2793	17266	10895	0

Table C.28: Internal MQOMv2.1 signing algorithm, AVX2 implementation

	Instance	declarations	row 1-2	row 5	signature preparation	row 6	row 7	row 8	row 8 continued	row 9	row 10	row 11	return success
Cat. I	mqom2_cat1_gf2_short_r3	1	1	61	1	184689	414893	81	119	76	4510	104	0
	mqom2_cat1_gf16_short_r3	1	1	54	2	195101	88680	114	159	58	4509	64	0
	mqom2_cat1_gf256_short_r3	1	1	53	1	217259	110872	158	207	103	4521	59	0
	mqom2_cat1_gf2_fast_r3	1	1	59	1	36512	311220	119	165	56	8902	52	0
	mqom2_cat1_gf16_fast_r3	1	1	51	1	39260	83330	116	211	55	8774	27	0
	mqom2_cat1_gf256_fast_r3	1	1	52	1	42964	103990	209	306	102	8961	28	0
	mqom2_cat1_gf2_short_r5	1	2	57	5	186203	416401	80	120	78	4430	99	0
	mqom2_cat1_gf16_short_r5	1	1	52	1	185424	89118	61	108	57	4477	48	0
	mqom2_cat1_gf256_short_r5	1	1	53	1	190510	112193	61	108	103	4459	61	0
	mqom2_cat1_gf2_fast_r5	1	2	58	1	37119	315121	72	161	56	8928	46	0
	mqom2_cat1_gf16_fast_r5	1	1	53	1	37301	83587	66	161	55	8839	26	0
	mqom2_cat1_gf256_fast_r5	1	1	51	1	37998	105177	63	160	102	8821	30	0
Cat. III	mqom2_cat3_gf2_short_r3	1	1	74	1	742475	2011924	200	322	132	71358	239	0
	mqom2_cat3_gf16_short_r3	1	1	59	6	829518	403372	268	372	122	71670	396	0
	mqom2_cat3_gf256_short_r3	1	1	59	1	1007010	497733	463	582	124	71451	308	0
	mqom2_cat3_gf2_fast_r3	1	1	71	6	148367	1550072	305	418	129	274	244	0
	mqom2_cat3_gf16_fast_r3	1	1	51	1	164428	357732	417	461	103	186	80	0
	mqom2_cat3_gf256_fast_r3	1	1	52	1	199612	463834	696	746	104	190	81	0
	mqom2_cat3_gf2_short_r5	1	1	69	1	745078	2013669	192	255	125	71323	248	0
	mqom2_cat3_gf16_short_r5	1	1	60	1	740590	407349	176	217	125	71860	268	0
	mqom2_cat3_gf256_short_r5	1	1	60	6	832855	505295	175	218	128	71926	278	0
	mqom2_cat3_gf2_fast_r5	1	1	68	5	149361	1550966	264	362	122	279	243	0
	mqom2_cat3_gf16_fast_r5	1	1	51	1	146807	357006	218	311	104	187	78	0
	mqom2_cat3_gf256_fast_r5	1	1	54	1	164001	465564	218	315	103	183	85	0
Cat. V	mqom2_cat5_gf2_short_r3	1	1	93	5	1114987	6206752	420	473	173	1287	413	0
	mqom2_cat5_gf16_short_r3	1	1	64	1	1230673	1328230	545	648	175	1207	478	0
	mqom2_cat5_gf256_short_r3	1	1	66	1	1461168	1506375	878	976	181	1255	437	0
	mqom2_cat5_gf2_fast_r3	1	1	76	1	206170	4519348	569	638	180	441	432	0
	mqom2_cat5_gf16_fast_r3	1	1	58	5	231164	1071194	776	831	181	378	235	0
	mqom2_cat5_gf256_fast_r3	1	1	53	1	272004	1221930	1244	1280	153	349	216	0
	mqom2_cat5_gf2_short_r5	1	1	95	3	1107552	6211263	334	410	182	1266	410	0
	mqom2_cat5_gf16_short_r5	1	1	61	1	1116465	1347580	278	383	176	1224	441	0
	mqom2_cat5_gf256_short_r5	1	1	64	6	1233472	1510805	292	383	167	1230	497	0
	mqom2_cat5_gf2_fast_r5	1	1	85	6	205943	4544362	468	506	170	438	387	0
	mqom2_cat5_gf16_fast_r5	1	1	52	1	207839	1072846	418	470	153	336	217	0
	mqom2_cat5_gf256_fast_r5	1	1	58	4	225319	1243765	424	477	167	363	198	0

Table C.29: Internal MQOMv2.1 signing algorithm, NEON implementation

Verification algorithm

	Instance	declaration	row 1	row 2	row 3	row 4	row 5	row 6	row 7	row 8 and 9	row 10	row 11	row 12
Cat. I	mqom2_cat1_gf2_short_r3	41	31	25	1215	1264	1182	42	3982248	5630536	4820	274	0
	mqom2_cat1_gf16_short_r3	22	37	22	953	1019	998	35	3767862	1134182	4742	186	0
	mqom2_cat1_gf256_short_r3	25	29	24	1162	1992	1179	33	4820174	1683313	7510	276	0
	mqom2_cat1_gf2_fast_r3	51	33	24	1200	1228	1103	62	832364	5437963	6542	354	0
	mqom2_cat1_gf16_fast_r3	21	36	22	903	949	894	38	661804	1065811	5105	155	0
	mqom2_cat1_gf256_fast_r3	27	30	24	1166	1909	1109	27	921777	1882030	10337	193	0
	mqom2_cat1_gf2_short_r5	24	51	24	1234	1287	1185	35	3895873	5663288	4521	312	0
	mqom2_cat1_gf16_short_r5	23	74	23	1039	1197	1128	37	3952000	1266406	3553	231	0
	mqom2_cat1_gf256_short_r5	24	39	24	1181	1982	1101	42	4085190	1686474	3727	220	0
	mqom2_cat1_gf2_fast_r5	54	72	24	1233	1192	1200	51	814947	5475499	5734	456	0
	mqom2_cat1_gf16_fast_r5	23	30	23	1013	1047	983	33	713138	1199865	4248	165	0
	mqom2_cat1_gf256_fast_r5	25	30	25	1177	1902	1153	56	829260	1937875	5113	210	0
Cat. III	mqom2_cat3_gf2_short_r3	27	42	21	1414	1759	1091	25	15302675	21969970	8817	445	0
	mqom2_cat3_gf16_short_r3	45	48	24	1536	1995	1239	34	20664878	5154187	12362	346	0
	mqom2_cat3_gf256_short_r3	36	29	20	1257	1654	1012	22	21260388	5549035	15943	230	0
	mqom2_cat3_gf2_fast_r3	33	24	21	1017	1789	1098	34	3290370	19188287	12355	329	0
	mqom2_cat3_gf16_fast_r3	33	81	25	1109	1976	1206	26	4221325	5780274	16983	399	0
	mqom2_cat3_gf256_fast_r3	24	33	22	1044	1775	1023	27	4569252	7122551	24092	264	0
	mqom2_cat3_gf2_short_r5	21	39	21	1382	1773	1061	23	15262723	22298484	7765	354	0
	mqom2_cat3_gf16_short_r5	24	40	24	1678	1988	1209	37	18299857	5575653	8178	633	0
	mqom2_cat3_gf256_short_r5	21	32	22	1449	1760	1062	42	18369786	5999804	7207	250	0
	mqom2_cat3_gf2_fast_r5	24	74	22	1037	1852	1122	39	3183924	19122719	10210	337	0
	mqom2_cat3_gf16_fast_r5	29	32	25	1194	1997	1173	26	3665009	5687695	11332	284	0
	mqom2_cat3_gf256_fast_r5	29	38	21	1027	1762	1047	52	3639779	6762838	9439	224	0
Cat. V	mqom2_cat5_gf2_short_r3	23	41	20	1122	2249	1052	31	22668145	62377217	13914	505	0
	mqom2_cat5_gf16_short_r3	27	33	21	1118	2485	1129	28	29310446	14506276	19718	350	0
	mqom2_cat5_gf256_short_r3	29	28	23	1156	2507	1136	26	36755004	17683390	30497	380	0
	mqom2_cat5_gf2_fast_r3	28	56	22	1118	2503	1131	27	4779762	52806145	20587	498	0
	mqom2_cat5_gf16_fast_r3	26	35	22	1086	2560	1141	38	5558592	14235461	27148	380	0
	mqom2_cat5_gf256_fast_r3	25	34	23	1124	2651	1196	39	7249571	19026737	43759	468	0
	mqom2_cat5_gf2_short_r5	20	59	20	933	2311	1056	24	23013178	62489716	11319	491	0
	mqom2_cat5_gf16_short_r5	22	61	22	1122	2603	1164	33	26743117	15232078	11715	389	0
	mqom2_cat5_gf256_short_r5	24	44	23	1170	2598	1174	33	31148576	18598030	12133	418	0
	mqom2_cat5_gf2_fast_r5	22	44	22	1128	2517	1126	40	4797082	53436583	16574	489	0
	mqom2_cat5_gf16_fast_r5	22	35	22	1063	2592	1107	24	5068451	14377268	16405	374	0
	mqom2_cat5_gf256_fast_r5	26	42	23	1239	2645	1190	41	5929395	18941160	16406	456	0

Table C.30: Internal MQOMv2.1 verification algorithm, AVX2 implementation

	Instance	declaration	row 1	row 2	row 3	row 4	row 5	row 6	row 7	row 8 and 9	row 10	row 11	row 12
Cat. I	mqom2_cat1_gf2_short_r3	1	1	1	57	77	69	1	146654	429034	191	8	0
	mqom2_cat1_gf16_short_r3	1	1	1	53	62	57	1	159503	82154	265	5	0
	mqom2_cat1_gf256_short_r3	1	1	1	51	102	54	1	181390	105954	358	3	0
	mqom2_cat1_gf2_fast_r3	1	1	1	52	56	54	1	29486	304934	272	4	0
	mqom2_cat1_gf16_fast_r3	1	1	1	51	54	54	1	32530	66259	310	2	0
	mqom2_cat1_gf256_fast_r3	1	1	1	50	102	53	1	36361	87759	502	2	0
	mqom2_cat1_gf2_short_r5	1	1	1	58	77	71	1	148783	425566	187	13	0
	mqom2_cat1_gf16_short_r5	1	1	1	50	56	54	1	149540	82530	161	3	0
	mqom2_cat1_gf256_short_r5	1	1	1	51	103	54	1	158900	106493	162	7	0
	mqom2_cat1_gf2_fast_r5	1	1	1	51	55	54	1	29870	305733	222	3	0
	mqom2_cat1_gf16_fast_r5	1	1	1	51	54	53	1	30405	66840	214	2	0
	mqom2_cat1_gf256_fast_r5	1	1	1	50	102	54	1	31272	87525	213	1	0
Cat. III	mqom2_cat3_gf2_short_r3	1	1	1	62	120	73	1	681984	2107637	472	4	0
	mqom2_cat3_gf16_short_r3	1	1	1	59	124	71	1	771937	390757	624	6	0
	mqom2_cat3_gf256_short_r3	1	1	1	63	126	72	7	951747	495649	1012	3	0
	mqom2_cat3_gf2_fast_r3	1	1	1	60	125	71	1	136700	1516969	660	15	0
	mqom2_cat3_gf16_fast_r3	1	1	1	51	104	54	1	153785	321891	848	2	0
	mqom2_cat3_gf256_fast_r3	1	1	1	51	104	53	1	188745	420910	1413	2	0
	mqom2_cat3_gf2_short_r5	1	1	1	66	126	72	7	688447	2091040	429	11	0
	mqom2_cat3_gf16_short_r5	1	1	1	55	129	69	1	681272	393023	366	12	0
	mqom2_cat3_gf256_short_r5	1	1	1	64	124	72	1	771796	493476	379	11	0
	mqom2_cat3_gf2_fast_r5	1	1	1	63	122	68	1	138046	1516408	577	14	0
	mqom2_cat3_gf16_fast_r5	1	1	1	51	103	54	1	136474	316572	512	3	0
	mqom2_cat3_gf256_fast_r5	1	1	1	51	103	54	1	153378	416240	513	1	0
Cat. V	mqom2_cat5_gf2_short_r3	1	1	1	63	163	70	6	990241	6498772	831	14	0
	mqom2_cat5_gf16_short_r3	1	1	1	63	169	77	1	1119306	1311585	1124	7	0
	mqom2_cat5_gf256_short_r3	1	1	1	62	172	71	6	1345754	1503830	1795	11	0
	mqom2_cat5_gf2_fast_r3	1	1	1	63	164	73	1	184030	4503678	1211	6	0
	mqom2_cat5_gf16_fast_r3	1	1	1	58	171	71	1	208545	993083	1558	7	0
	mqom2_cat5_gf256_fast_r3	1	1	1	51	153	55	1	249602	1145771	2496	10	0
	mqom2_cat5_gf2_short_r5	1	1	1	64	168	74	1	992098	6425766	689	18	0
	mqom2_cat5_gf16_short_r5	1	1	1	61	173	71	1	997780	1323539	626	11	0
	mqom2_cat5_gf256_short_r5	1	1	1	62	175	69	1	1110496	1501723	641	9	0
	mqom2_cat5_gf2_fast_r5	1	1	1	62	169	70	1	183329	4503086	985	11	0
	mqom2_cat5_gf16_fast_r5	1	1	1	51	151	56	1	186704	984741	855	9	0
	mqom2_cat5_gf256_fast_r5	1	1	1	52	150	55	1	205180	1143620	859	3	0

Table C.31: Internal MQOMv2.1 verification algorithm, NEON implementation

UNIVERSITÉ CATHOLIQUE DE LOUVAIN
École polytechnique de Louvain

Rue Archimède, 1 bte L6.11.01, 1348 Louvain-la-Neuve, Belgique | www.uclouvain.be/epl